Python Lecture Note

2. Python 클래스와 객체

Dong Hyun KIM

2 Python 클래스와 객체

2.1 Introduction - 클래스와 객체 모델을 왜 공부하는가?

학습목표

Python의 클래스와 객체 모델을 공부해야 하는 이유를

이해하고 핵심 질문을 정의한다.

이번 학습의 출발점은 다음과 같은 PyTorch 모델 코드다.

```
import torch.nn as nn


class MyModel(nn.Module):

    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)
```

이 코드는 PyTorch를 이용해 간단한 신경망 모델을 정의한 것이다.

코드의 기능을 기준으로 설명하면 다음과 같다.

1. MyModel이라는 클래스를 정의한다.
2. nn.Module을 상속한다.
3. __init__()에서 신경망 계층을 정의한다.
4. forward()에서 입력 데이터에 대한 연산을 정의한다.

이 정도의 설명만으로도 코드의 목적은 대략적으로 파악할 수 있다.

하지만 코드의 기능을 이해하는 것과 Python이 해당 코드를 실행하는 원리를 이해하는 것은 서로 다른 문제다.

예를 들어 다음과 같은 질문을 생각해 볼 수 있다.

코드	핵심 질문
`class MyModel:`	클래스 정의를 실행하면 무엇이 생성되는가?
`MyModel()`	클래스 이름 뒤에 ()를 붙이면 어떤 과정이 시작되는가?
`__init__(self)`	왜 인스턴스를 생성할 때 자동으로 호출되는가?
`self`	무엇을 참조하며 누가 전달하는가?
`self.linear = ...`	생성한 객체는 어디에 저장되는가?
`nn.Module`	상속하면 어떤 기능을 사용할 수 있는가?
`super().__init__()`	상위 클래스의 초기화가 왜 필요한가?
`model(x)`	모델 인스턴스를 함수처럼 호출할 수 있는 이유는 무엇인가?
`__call__()`	Python은 왜 이 이름을 특별하게 처리하는가?

이러한 질문에 순서대로 답하는 것을 목표로 한다.

2.1.1 메서드와 self에 대한 의문

다음 코드를 살펴보자.

```
class Person:

    def introduce(self):
        print("Hello")


p = Person()

p.introduce()
```

실행 결과:

```
Hello
```

여기서 `introduce()` 메서드는 다음과 같이 정의되어 있다.

```
def introduce(self):
```

분명히 `self`라는 매개변수가 하나 존재한다.

그런데 실제 호출에서는 아무런 인수도 전달하지 않았다.

```
p.introduce()
```

그럼에도 정상적으로 실행된다.

왜 그럴까?

Python에서는 인스턴스를 통해 일반적인 인스턴스 메서드에 접근하면 해당 인스턴스가 연결된 **바운드 메서드(Bound Method)**를 얻기 때문이다.

따라서 다음 두 호출은 이 예제에서 같은 효과를 가진다.

```
p.introduce()
```

```
Person.introduce(p)
```

여기서 `self`는 Python의 예약어가 아니다.

메서드에 전달되는 인스턴스를 받기 위해 관례적으로 사용하는 매개변수 이름이다.

즉, `self`를 이해하려면 다음 개념을 함께 이해해야 한다.

1. 함수도 객체라는 사실
2. 클래스에 정의된 함수와 인스턴스 메서드의 관계
3. 인스턴스를 통한 속성 접근
4. 바운드 메서드와 인수 전달

이 내용은 2.5. 메서드와 `self`에서 자세히 학습한다.

2.1.2 인스턴스 생성과 초기화에 대한 의문

다음 코드를 살펴보자.

```
class Person:
    def __init__(self, name):
```

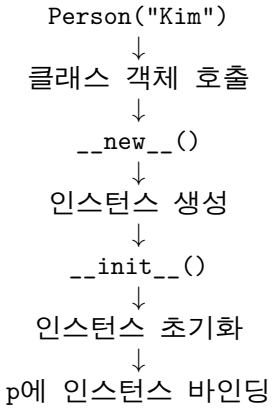
```
        self.name = name

p = Person("Kim")
```

일반적으로 `__init__()`은 객체를 생성할 때 실행되는 함수라고 설명한다.

하지만 엄밀하게 말하면 `__init__()`은 객체 자체를 생성하는 메서드가 아니다.

Python의 일반적인 인스턴스 생성 과정은 다음과 같다.



이것은 일반적인 클래스 생성 과정의 개념도이며,

실제로는 `__new__()`가 해당 클래스의 인스턴스를 반환하는지 등에 따라

`__init__()` 호출 여부가 달라질 수 있다.

여기서 두 메서드의 역할을 구분해야 한다.

메서드	주요 역할
`__new__()`	인스턴스 생성
`__init__()`	생성된 인스턴스 초기화

그리고 다음 코드에서는

```
self.name = name
```

인스턴스에 `name`이라는 속성을 설정한다.

따라서 다음 질문들이 발생한다.

1. __new__()와 __init__()은 왜 분리되어 있는가?
2. Python은 이 메서드들을 어떤 규칙으로 호출하는가?
3. self는 어느 시점에 전달되는가?
4. self.name은 어디에 저장되는가?
5. __init__()은 어떤 값을 반환해야 하는가?

이 내용은 2.6. __new__와 __init__에서 자세히 학습한다.

2.1.3 특수 메서드와 Callable 객체

다음 코드를 살펴보자.

```
class Calculator:
    def __call__(self, x):
        return x * 2

calc = Calculator()

result = calc(10)

print(result)
```

실행 결과:

```
20
```

여기서 calc는 함수가 아니다.

Calculator 클래스의 인스턴스다.

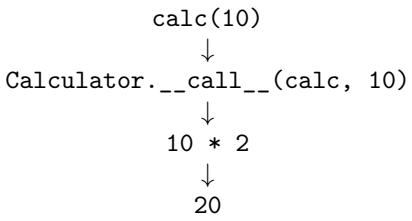
그런데 다음 코드가 정상적으로 실행된다.

```
calc(10)
```

이것이 가능한 이유는 Calculator 클래스에 __call__() 이라는 특수 메서드가 정의되어 있기 때문이다.

Python은 인스턴스가 호출될 때 해당 타입의 __call__()을 이용한다.

따라서 이 예제에서는 다음과 같은 호출 관계를 생각할 수 있다.



이러한 객체를 Callable 객체라고 한다.

이 개념은 PyTorch 모델에서도 사용된다.

```
model = MyModel()

output = model(x)
```

여기서 `model`은 `MyModel`의 인스턴스다.

하지만 `nn.Module`이 제공하는 호출 메커니즘을 상속받았기 때문에 함수처럼 호출할 수 있다.

PyTorch는 이 호출 과정에서 `forward()`를 실행하도록 구성되어 있다.

따라서 `__call__()`을 이해하면 다음 두 코드의 관계도 설명할 수 있다.

```
model(x)

model.forward(x)
```

두 코드 모두 순전파 계산을 수행할 수 있지만, 동일한 호출 경로를 거치는 것은 아니다.

`model(x)`는 PyTorch의 모듈 호출 메커니즘을 거치므로 일반적으로 권장되는 방식이다.

이 내용은 2.10. 특수 메서드에서 자세히 학습한다.

2.1.4 상속은 왜 사용하는가?

다음 코드를 살펴보자.

```
class MyModel(nn.Module):
    ...
```

여기서 MyModel은 nn.Module을 상속한다.

상속은 일반적으로 기존 클래스의 기능을 재사용하는 방법으로 소개된다.

하지만 상속의 목적은 단순한 코드 재사용에만 있지 않다.

상속을 통해 기존 클래스가 제공하는 메서드와 동작 규약을 활용하거나,

필요한 부분을 재정의하여 새로운 클래스를 구성할 수 있다.

예를 들어 nn.Module을 상속받은 모델에서는 다음과 같은 메서드들을 사용할 수 있다.

```
model.parameters()

model.state_dict()

model.to(device)

model.train()

model.eval()
```

우리는 이러한 메서드를 MyModel 내부에 직접 정의하지 않았다.

그런데도 사용할 수 있는 이유는 상속 관계를 통해 해당 기능들이 제공되기 때문이다.

또한 nn.Module은 하위 모듈과 파라미터를 등록하고 관리하는 기능도 제공한다.

따라서 다음 코드에서

```
self.linear = nn.Linear(10, 5)
```

nn.Linear 인스턴스를 모델의 하위 모듈로 등록하는 동작이 이루어진다.

이러한 구조를 이해하려면 다음 개념이 필요하다.

1. 상속 관계
2. 메서드 탐색
3. 메서드 오버라이딩
4. super()

5. MRO(Method Resolution Order)

6. 클래스가 제공하는 객체 관리 기능

이 내용은 2.8. 상속과 메서드 오버라이딩, 2.9. super()와 MRO에서 자세히 학습한다.

2.1.5 최종 학습 목표

이번 학습의 최종 목표는 Python 클래스의 문법을 단순히 암기하는 것이 아니다.

다음 코드를 다시 살펴보자.

```
import torch.nn as nn


class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)

model = MyModel()

output = model(x)
```

여기서 x는 적절한 입력 Tensor가 미리 정의되어 있다고 가정한다.

이번 학습을 마친 뒤에는 다음 과정을 스스로 설명할 수 있어야 한다.

클래스 정의

```
class MyModel(nn.Module):
```

Python이 클래스 객체를 생성하고 MyModel이라는 이름에 바인딩하는 과정

인스턴스 생성

```
model = MyModel()
```

클래스 객체가 호출되고 인스턴스가 생성되는 과정

인스턴스 초기화

```
def __init__(self):
```

생성된 인스턴스가 초기화되는 과정

상속된 기능 활용

```
super().__init__()
```

상속 관계에서 `super()`가 다음 메서드를 탐색하고 호출하는 과정

인스턴스 속성 설정

```
self.linear = nn.Linear(10, 5)
```

생성한 객체가 인스턴스에 연결되고 Pytorch에 하위 모듈로 등록되는 과정

Callable 객체 호출

```
output = model(x)
```

인스턴스 호출이 `nn.Module`의 호출 메커니즘을 거쳐 `forward()`를 실행하는 과정

이 과정을 이해하면 PyTorch 모델을 단순히 아키텍처를 구현한 코드로 보는 것이 아니라, Python 객체 모델을 활용하여 구성된 실행가능한 시스템으로 이해할 수 있게 된다.

2.2 클래스도 객체인가? - class 정의를 실행하면 무엇이 생성되는가?

이번에는 Python의 `class`문을 단순히 클래스를 선언하는 문법으로 이해하는 것이 아니라, Python이 `class`문을 실행할 때 어떤 객체를 생성하고, 그 객체를 어떤 이름에 바인딩하는지 살펴본다.

함수와 데코레이터에서 다음 개념을 학습했을 것이다.

```
def add(a, b):
```

```
    return a + b
```

이 코드를 실행하면 Python은 함수 객체를 생성하고 add라는 이름에 바인딩한다.

클래스도 비슷한 관점에서 이해할 수 있다.

```
class Person:
    pass
```

이 코드를 실행하면 Python은 **클래스 객체(Class Object)**를 생성하고 Person이라는 이름에 바인딩한다.

다만 함수 객체와 클래스 객체는 서로 다른 종류의 객체이며, 생성 과정과 역할에도 차이가 있다.

이번에는 그 차이를 순서대로 살펴본다.

2.2.1 개념 정의 - Python에서 클래스도 객체다

Python에서는 정수, 문자열, 리스트, 함수뿐 아니라 클래스도 객체로 취급한다.

다음 코드를 살펴보자.

```
class Person:
    pass

print(Person)
print(type(Person))
```

실행 결과:

```
<class '__main__.Person'>
<class 'type'>
```

첫 번째 출력은 Person이라는 이름이 클래스 객체를 참조하고 있다는 사실을 보여준다.

두 번째 출력은 해당 클래스 객체의 타입이 type이라는 사실을 보여준다.

여기서 중요한 구분이 있다.

클래스 객체(class object)

```
Person
```

Person은 클래스 객체를 참조하는 이름이다.

인스턴스 객체(Instance Object)

```
Person()
```

Person()은 클래스 객체를 호출하는 표현식이다.

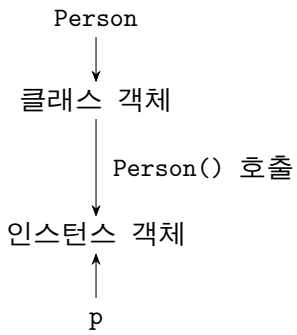
기본적인 클래스에서는 이 호출을 통해 새로운 인스턴스가 생성된다.

인스턴스를 참조하는 이름

```
p = Person()
```

생성된 인스턴스 객체를 p라는 이름에 바인딩한다.

따라서 다음과 같이 구분할 수 있다.



여기서 Person과 p는 모두 이름이지만, 참조하는 객체의 종류가 다르다.

표현식	의미	참조하거나 반환하는 객체
`Person`	클래스 이름을 평가	클래스 객체
`Person()`	클래스 객체 호출	일반적으로 새로운 인스턴스 객체
`p = Person()`	반환된 객체를 이름에 바인딩	p가 인스턴스를 참조
`type(Person)`	클래스 객체의 타입 확인	`type`
`type(p)`	인스턴스 객체의 타입 확인	`Person`

이 관계를 실제 코드로 확인해 보자.

```
class Person:
    pass

p = Person()

print(type(Person))
print(type(p))

print(isinstance(p, Person))
```

실행 결과:

```
<class 'type'>
<class '__main__.Person'>
True
```

여기서 `isinstance(p, Person)`은 `p`가 `Person` 클래스의 인스턴스인지 확인하는 표현식이다.

상속 관계까지 고려하므로 단순히 `type(p) is Person`과 항상 같은 검사는 아니다.

핵심은 다음과 같다.

> Python에서 클래스는 객체를 만들기 위한 정의이면서 동시에,
>
> Python 프로그램 안에서 참조하고 전달할 수 있는 객체이기도 하다.

2.2.2 class문은 단순한 선언이 아니라 실행되는 문장이다

이 부분이 이번 학습에서 가장 중요하다.

Python의 class문은 프로그램 실행 과정에서 평가되고 실행된다.

예를 들어 다음 코드를 보자.

```
print("A")

class Person:
    print("B")

print("C")
```

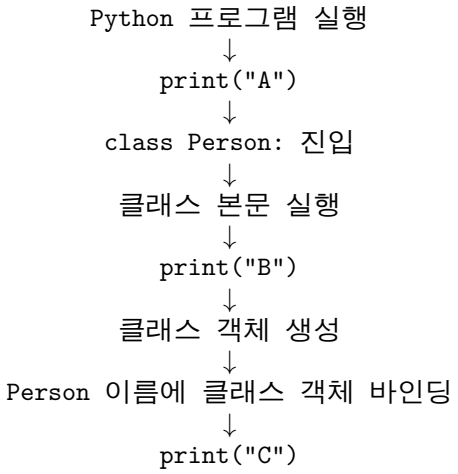
실행 결과:

```
A
B
C
```

왜 이런 결과가 발생할까?

class Person: 아래의 본문은 인스턴스를 생성할 때 실행되는 것이 아니라, 클래스 정의문 자체가 실행될 때 실행되기 때문이다.

실행 흐름을 살펴보자.



따라서 클래스 본문에 작성된 일반적인 문장은 클래스 정의 시점에 실행된다.

반면 다음 코드는 조금 다르다.

```
class Person:

    print("클래스 본문 실행")

    def introduce(self):
        print("메서드 실행")


p = Person()
```

실행 결과:

```
클래스 본문 실행
```

`Person()`을 호출했지만 `introduce()` 내부의 출력문은 실행되지 않았다.

왜냐하면 `def introduce(self):`는 함수 본문을 즉시 실행하는 것이 아니라, 함수 객체를 만들어 클래스의 네임스페이스에 등록하는 과정이기 때문이다.

`introduce()`의 본문은 해당 메서드가 호출될 때 실행된다.

```
p.introduce()
```

실행 결과:

```
메서드 실행
```

여기서 함수 정의와 클래스 정의 사이의 차이점이 드러난다.

구분	정의문이 실행될 때	함수·메서드 본문 실행 시점
`def`	함수 객체 생성 및 이름 바인딩	함수 호출 시
`class`	클래스 본문 실행 후 클래스 객체 생성 및 이름 바인딩	클래스 본문의 메서드는 별도 호출 시

클래스 본문 실행과 인스턴스 생성은 서로 다른 사건이라는 점을 구분해야 한다.

2.2.3 클래스 객체는 어떻게 만들어지는가?

이번에는 클래스 객체가 만들어지는 과정을 조금 더 자세히 살펴보자.

다음 코드를 사용하겠다.

```
class Person:

    species = "Human"

    def introduce(self):
        print("Hello")
```

Python은 이 클래스 정의를 처리할 때 대략 다음 과정을 거친다.

클래스 본문에서 실행하기 위한 네임스페이스 준비

클래스는 본문에서 정의되는 이름들을 관리할 공간을 준비한다.

클래스 본문 실행

```
species = "Human"
```

문자열 객체 "Human"을 species라는 이름에 연결한다.

```
def introduce(self):
    print("Hello")
```

함수 객체를 생성하고 introduce라는 이름에 연결한다.

클래스 객체 생성

클래스 본문의 실행 결과와 클래스 이름, 상속 정보 등을 사용해 클래스 객체를 생성한다.

클래스 객체를 Person이라는 이름에 바인딩

생성된 클래스 객체가 현재 네임스페이스의 Person이라는 이름에 연결된다.

이 과정을 개념적으로 나타내면 다음과 같다.

```
class Person:
    species = "Human"
```

```
def introduce(self):
    print("Hello")

        ↓

클래스 본문 실행

        ↓

클래스 네임 스페이스
    species   → "Human"
    introduce → 함수 객체

        ↓

클래스 객체 생성

        ↓

Person → 클래스 객체
```

실제 클래스 생성 과정에는 메타클래스, 상속 처리, 클래스 네임스페이스 준비 등의 절차가 더 포함되지만, 지금은 클래스 본문 실행 → 클래스 객체 생성 → 이름 바인딩이라는 구조를 이해하는 것이 중요하다.

클래스 네임스페이스 확인하기

클래스 객체가 생성된 뒤에는 다음과 같이 클래스의 속성을 확인할 수 있다.

```python
class Person:

    species = "Human"

    def introduce(self):
        print("Hello")


print(Person.__dict__)
```

`Person.__dict__`에는 클래스가 관리하는 속성들이 들어 있다.

개념적으로는 다음과 같은 구조다.

```
Person.__dict__

{
    "__module__": "__main__",
    "species": "Human",
```

```
    "introduce": <function ...>,
    "__dict__": ...,
    "__weakref__": ...,
    "__doc__": None
}
```

실제 출력 형태는 Python 버전과 실행 환경에 따라 달라질 수 있으며, 일반적인 클래스의 `__dict__`는 읽기 전용 매핑 뷰인 `mappingproxy`로 제공된다.

여기서 중요한 것은 `species`와 `introduce`가 클래스의 속성으로 존재한다는 사실이다.

특히 `introduce`는 함수 객체라는 점을 기억하자.

```
print(Person.__dict__["introduce"])
```

이 표현식은 클래스 네임스페이스에 저장된 함수 객체를 확인한다.

이후 2.5에서는 이 함수 객체가 인스턴스를 통해 접근될 때 어떻게 바운드 메서드로 연결되는지 살펴볼 것이다.

2.2.4 클래스 객체도 이름을 바꾸거나 다른 변수에 전달할 수 있다

이전에 함수도 일급 객체라는 개념을 공부했다.

```
def add(a, b):
    return a + b

another_add = add
```

이 코드는 함수 객체를 복사하는 것이 아니라, 동일한 함수 객체를 다른 이름으로도 참조하도록 만드는 것이다.

클래스 객체도 마찬가지다.

```
class Person:
    pass

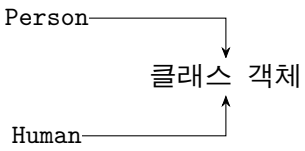
Human = Person

print(Human is Person)
```

실행 결과:

```
True
```

여기서 Human과 Person은 동일한 클래스 객체를 참조한다.



따라서 다음 코드도 가능하다.

```
p1 = Person()
p2 = Human()

print(type(p1) is type(p2))
```

실행 결과:

```
True
```

두 이름은 같은 클래스 객체를 참조하므로 같은 클래스의 인스턴스를 생성한다.

클래스 객체를 함수의 인수로 전달하기

클래스 객체는 함수의 인수로도 전달할 수 있다.

```
class Person:
    pass

def create_instance(cls):
    return cls()

p = create_instance(Person)

print(type(p))
```

실행 결과:

```
<class '__main__.Person'>
```

여기서 실행 흐름을 분석해 보자.

Person 평가

```
create_instance(Person)
```

Person이라는 이름을 평가해 클래스 객체를 가져온다.

매개변수 cls에 바인딩

```
def create_instance(cls):
```

cls는 전달된 클래스 객체를 참조한다.

클래스 객체 호출

```
return cls()
```

cls가 참조하는 클래스 객체를 호출해 인스턴스를 생성한다.

인스턴스 반환

생성된 인스턴스가 반환되어 p에 바인딩된다.

이 구조는 이후 머신러닝 프로젝트에서 유용하게 활용된다.

예를 들어 모델 클래스를 함수에 전달해 원하는 모델을 생성할 수 있다.

```
def build_model(model_class, *args, **kwargs):
    return model_class(*args, **kwargs)
```

이 함수는 특정 모델 클래스에 종속되지 않는다.

전달받은 클래스 객체를 호출해 모델 인스턴스를 생성한다.

즉, 클래스 객체 자체를 데이터처럼 전달하고 사용할 수 있다.

2.2.5 type은 무엇인가?

앞에서 다음 결과를 확인했다.

```
class Person:
    pass
```

```
print(type(Person))
```

실행 결과:

```
<class 'type'>
```

여기서 자연스럽게 다음 질문이 발생한다.

클래스 객체의 타입이 왜 type일까?

Python에서 type은 일반적인 클래스 객체를 생성하는 기본 메타클래스(Metaclass)다.

메타클래스는 클래스 객체의 생성과 동작을 정의하는 클래스라고 이해하면 된다.

다음 관계를 살펴보자.

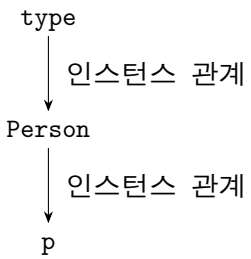
```
class Person:
    pass

p = Person()

print(type(p))
print(type(Person))
print(type(type))
```

```
<class '__main__.Person'>
<class 'type'>
<class 'type'>
```

이 관계를 구조적으로 표현하면 다음과 같다.



여기서 화살표는 상속 관계가 아니라 타입과 인스턴스의 관계를 나타낸다.

- p는 Person의 인스턴스다.
- Person은 type의 인스턴스다.
- type 자체도 type의 인스턴스다.

마지막 관계는 처음 보면 조금 특이할 수 있다.

하지만 Python의 객체 모델에서는 type이 자기 자신을 타입으로 갖는 구조가 성립한다.

type으로 클래스 객체를 직접 생성하기

type은 인수를 하나 전달하면 객체의 타입을 확인하는 용도로 사용할 수 있다.

```
type(10)
```

하지만 세 개의 인수를 전달하면 클래스 객체를 생성하는 데 사용할 수도 있다.

```
Person = type(
    "Person",
    (),
    {"species": "Human"}
)
```

이 코드는 기본적인 경우 다음 클래스 정의와 유사한 결과를 만든다.

```
class Person:
    species = "Human"
```

세 인수의 의미는 다음과 같다.

인수	의미
"Person"	생성할 클래스의 이름
()	상속할 기본 클래스들의 튜플
{"species": "Human"}	클래스의 속성을 담은 딕셔너리

실제로 확인해 보자.

```
Person = type(
    "Person",
    (),
```

```
    {"species": "Human"}
)

print(Person)
print(type(Person))
print(Person.species)
```

실행 결과:

```
<class '__main__.Person'>
<class 'type'>
Human
```

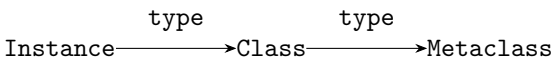
이것은 class문이 단순히 텍스트 수준의 선언이 아니라, 실행 결과로 클래스 객체를 만들어 낸다는 사실을 보여주는 예제다.

다만 일반적인 class문과 type()을 직접 호출하는 방식이 모든 상황에서 완전히 동일한 것은 아니다.

클래스 정의문에는 클래스 본문 실행, 메타클래스 선택, __class__ 참조 처리 등의 추가적인 의미가 있기 때문이다.

현재 단계에서는 메타클래스를 직접 작성하는 방법까지 공부할 필요는 없다.

우선 다음 관계만 이해하면 충분하다.



2.2.6 함수 객체와 클래스 객체의 공통점과 차이

이번에는 함수 객체와 클래스 객체를 비교해 보자.

함수 정의

```
def add(a, b):
    return a + b
```

이 코드를 실행하면 함수 객체가 생성되고 add라는 이름에 바인딩된다.

add → 함수 객체

클래스 정의

```
class Calculator:
    pass
```

이 코드를 실행하면 클래스 객체가 생성되고 `Calculator`라는 이름에 바인딩된다.

`Calculator` → 클래스 객체

두 객체 모두 Python 프로그램 안에서 참조하거나 다른 함수에 전달할 수 있다.

하지만 호출했을 때의 기본적인 역할은 다르다.

```
add(1, 2)
```

함수 객체를 호출하면 함수 본문이 실행된다.

반면 다음 코드에서는

```
Calculator()
```

클래스 객체를 호출해 일반적으로 새로운 인스턴스를 생성하고 초기화한다.

정리하면 다음과 같다.

구분	함수 객체	클래스 객체
정의 문법	`def`	`class`
정의 결과	함수 객체	클래스 객체
일반적인 타입	`function`	`type`
이름에 바인딩	가능	가능
인수로 전달	가능	가능
반환값으로 사용	가능	가능
`()` 호출	함수 본문 실행	일반적으로 인스턴스 생성 및 초기화
주요 역할	연산과 동작 수행	인스턴스의 구조와 동작 정의

여기서 주의할 점이 있다.

함수와 클래스 모두 호출 가능한 객체라는 공통점이 있지만,

모든 Callable 객체가 함수이거나 클래스인 것은 아니다.

예를 들어 `__call__()`을 구현한 인스턴스 역시 Callable 객체가 될 수 있다.

이 내용은 2.10에서 다시 연결할 것이다.

2.2.7 PyTorch와의 연결 - 모델 클래스도 객체다

이제 처음에 살펴봤던 PyTorch 코드로 돌아가 보자.

```
import torch.nn as nn


class MyModel(nn.Module):

    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)
```

여기서 다음 표현식에 주목하자.

```
MyModel
```

이것은 모델 인스턴스가 아니다.

모델 클래스를 나타내는 클래스 객체다.

반면 다음 코드는

```
model = MyModel()
```

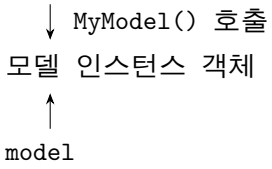
`MyMdodel` 클래스 객체를 호출해 모델 인스턴스를 생성한다.

따라서 다음과 같이 구분할 수 있다.

`MyModel`

↓

모델 클래스 객체



이 차이는 머신러닝 코드를 작성할 때 중요하다.

예를 들어 다음 함수를 보자.

```
def create_model(model_class):
    return model_class()
```

이 함수에는 클래스 객체를 전달한다.

```
model = create_model(MyModel)
```

반대로 이미 생성된 모델 인스턴스를 전달하는 경우도 있다.

```
def count_parameters(model):
    return sum(
        p.numel()
        for p in model.parameters()
    )
```

이 함수는 모델 클래스가 아니라 모델 인스턴스를 전달받는 것을 전제로 한다.

```
model = MyModel()

count = count_parameters(model)
```

따라서 PyTorch 코드를 읽을 때는 다음을 구분해야 한다.

1. 클래스 객체를 전달하는가?
2. 클래스 객체를 호출해 인스턴스를 생성하는가?
3. 이미 생성된 인스턴스를 전달하는가?
4. 인스턴스를 Callable 객체로 호출하는가?

이 구분이 가능해지면 Hugging Face나 PyTorch에서 클래스를 인수로 전달하거나 모델을 동적으로 생성하는 코드도 훨씬 이해하기 쉬워질 것이다.

2.2.8 핵심 정리

이번에는 Python에서 클래스도 객체라는 사실을 중심으로 class 정의문의 실행 과정을 살펴보았다.

핵심 내용을 정리하면 다음과 같다.

> 클래스도 Python 객체다.

```
class Person:
    pass
```

이 코드를 실행하면 클래스 객체가 생성되고 Person이라는 이름에 바인딩된다.

> 클래스 정의문은 실행된다.

클래스 본문은 클래스 정의 시점에 실행되며, 그 결과가 클래스 객체를 구성하는 데 사용된다.

> 클래스 객체와 인스턴스 객체는 다르다.

```
Person      # 클래스 객체
Person()    # 일반적으로 인스턴스 생성
```

> 클래스 객체도 다른 이름에 바인딩하거나 함수에 전달할 수 있다.

```
Huamn = Person
```

이 경우 두 이름은 동일한 클래스 객체를 참조한다.

> 일반적인 클래스 객체의 타입은 type이다.

```
type(Person)
```

type은 Python에서 기본적인 메타클래스 역할을 한다.

> 클래스 객체를 호출하면 일반적으로 인스턴스 생성 과정이 시작된다.

```
p = Person()
```

인스턴스 생성과 초기화의 구체적인 과정은 이후 `__new__()`와 `__init__()`을 학습하면서 자세히 살펴본다.

PyTorch의 모델 클래스 역시 클래스 객체다.

```
class MyModel(nn.Module):
    ...
```

여기서 `MyModel`은 클래스 객체이며, `MyModel()`을 호출하면 일반적으로 모델 인스턴스가 생성된다.

반드시 이해해야 할 결론

이전에 함수 객체를 공부하며 다음 구조를 이해했다.

$$\texttt{def} \rightarrow \texttt{Function Object} \rightarrow \texttt{Name Binding}$$

이번에는 여기에 대응하는 새로운 구조를 배웠다.

$$\texttt{class} \rightarrow \texttt{Class Body Execution} \rightarrow \texttt{Class Object} \rightarrow \texttt{Name Binding}$$

즉, Python의 `class`는 클래스 객체를 생성하는 실행 가능한 문장이며,

생성된 클래스 객체는 다른 Python 객체들과 마찬가지로 이름에 바인딩하고 전달하고 호출할 수가 있다.

이것이 이번 학습에서 가장 중요한 개념이다.

다음 2.3. 클래스와 인스턴스에서는

이 개념을 기반으로 `Person`과 `Person()`이 구체적으로 어떻게 다른지,

클래스 객체를 호출했을 때 인스턴스가 생성되고 이름에 바인딩되는 과정을 더 자세히 살펴볼 것이다.

2.3 클래스와 인스턴스 — `Person`과 `Person()`은 어떻게 다른가?

2.2에서는 Python의 `class`문이 단순한 선언이 아니라, 클래스 본문을 실행하고 클래스 객체를 생성한 뒤 이름에 바인딩하는 문장이라는 사실을 학습했다.

이번 2.3에서는 생성된 클래스 객체를 호출하면 어떤 일이 발생하는지 살펴본다.

특히 다음 세 표현식의 차이를 정확하게 이해하는 것이 목표다.

```
Person
Person()
p = Person()
```

겉으로 보기에는 () 의 유무와 대입문 정도의 차이지만,

Python의 객체 모델에서는 서로 다른 동작을 의미한다.

이번 학습에서는 다음 질문에 답할 것이다.

1. 클래스와 인스턴스는 정확히 무엇이 다른가?
2. Person과 Person()은 각각 무엇을 의미하는가?
3. 인스턴스가 생성될 때 클래스 객체도 복사되는가?
4. 하나의 클래스로 여러 인스턴스를 생성하면 어떤 관계가 형성되는가?
5. 인스턴스는 자신의 클래스를 어떻게 알 수 있는가?
6. is, ==, type(), isinstance()는 어떤 차이가 있는가?
7. 이러한 구조가 Pytorch 모델에서는 어떻게 사용되는가?

2.3.1 개념 정의 - 클래스와 인스턴스

먼저 가장 기본적인 코드부터 살펴보자.

```
class Person:
    pass
```

이 코드를 실행하면 Python은 Person이라는 클래스 객체를 생성한다.

여기서 아직 Person의 인스턴스를 생성한 것은 아니다.

이후 다음 코드를 실행한다고 가정하자.

```
p = Person()
```

이때 일반적인 클래스에서는 새로운 인스턴스 객체가 생성된다.

따라서 두 객체를 구분해야 한다.

클래스 객체(Class Object)

클래스 객체는 인스턴스의 생성과 동작을 정의하는 객체다.

```
Person
```

Person이라는 이름은 클래스 객체를 참조한다.

인스턴스 객체(Instance Object)

인스턴스는 클래스 객체를 기반으로 생성된 개별 객체다.

```
Person()
```

이 표현식은 클래스 객체를 호출한다.

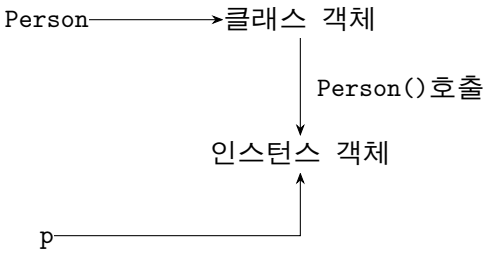
일반적인 경우 새로운 인스턴스를 생성하고 반환한다.

인스턴스를 참조하는 이름

```
p = Person()
```

오른쪽 표현식의 평가 결과를 왼쪽 이름 p에 바인딩한다.

따라서 전체 관계는 다음과 같다.



여기서 중요한 사실은 클래스 객체와 인스턴스 객체가 별도로 존재한다는 것이다.

클래스는 인스턴스를 생성하기 위한 정의이지만, Python에서는 그 정의 자체도 하나의 객체다.

구분	클래스 객체	인스턴스 객체
예시	`Person`	`Person()`의 반환값
일반적인 타입	`type`	`Person`
생성 시점	`class` 정의문 실행 시	클래스 호출 시
주요 역할	인스턴스의 구조와 동작 정의	개별 객체의 상태와 동작 표현
여러 개 생성 가능 여부	클래스 정의를 다시 실행하면 새 클래스 객체 생성 가능	같은 클래스에서 여러 인스턴스 생성 가능

여기서 인스턴스가 반드시 개별적인 상태를 가져야 하는 것은 아니다.

```
class Person:
    pass
```

이처럼 속성이 없는 클래스의 인스턴스도 정상적인 인스턴스 객체다.

2.3.2 Person과 Person()의 차이

이제 두 표현식을 비교해 보자.

```
Person
```

```
Person()
```

이전에 함수 객체를 공부하면서 다음 두 표현식의 차이를 학습했다.

```
add
```

```
add()
```

`add`는 함수 객체를 참조하고, `add()`는 함수 객체를 호출하는 표현식이다.

클래스도 이와 비슷하다.

Person - 클래스 객체 참조

```
class Person:
    pass

print(Person)
```

출력:

```
<class '__main__.Person'>
```

여기서는 클래스 객체를 참조할 뿐, 새로운 인스턴스를 생성하지 않는다.

Person() - 클래스 객체 호출

```
p = Person()
```

()는 앞에 있는 호출 가능한 객체를 호출하는 문법이다.

따라서 Python은 Person이 참조하는 클래스 객체를 호출한다.

일반적인 클래스 객체 호출은 인스턴스 생성과 초기화 과정으로 이어진다.

p = Person() - 반환값 바인딩

```
p = Person()
```

이 대입문을 개념적으로 나누면 다음과 같다.

1. Person 이름 평가

↓

2. 클래스 객체 획득

↓

3. 클래스 객체 호출

↓

4. 인스턴스 생성 및 초기화

↓

5. 호출 결과 반환

↓

6. p 이름에 반환된 객체 바인딩

중요한 것은 p가 객체를 생성하는 것이 아니라는 점이다.

객체 생성은 오른쪽의 Person() 호출 과정에서 이루어진다.

왼쪽의 p는 그 결과를 참조하는 이름일 뿐이다.

이전에 학습한 이름과 객체의 바인딩 개념이 그대로 적용되는 것이다.

2.3.3 하나의 클래스에서 여러 인스턴스 생성하기

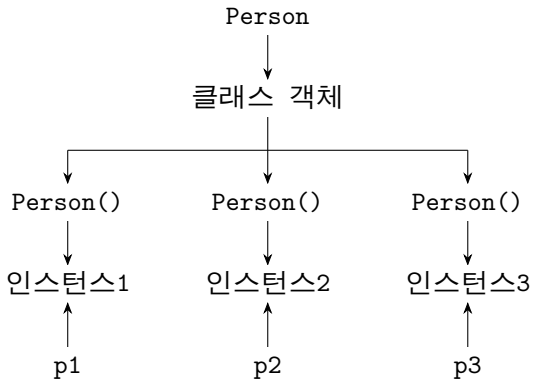
다음 코드를 살펴보자.

```
class Person:
    pass

p1 = Person()
p2 = Person()
p3 = Person()
```

여기서 Person 클래스 객체는 하나다.

하지만 Person()을 세 번 호출했으므로 일반적인 생성 과정에서는 서로 다른 인스턴스 세 개가 만들어진다.



실제로 확인해 보자.

```
print(p1 is p2)
print(p2 is p3)
print(p1 is p3)
```

출력:

```
False
```

```
False
False
```

is는 두 표현식이 동일한 객체를 참조하는지 검사한다.

각각 새로운 인스턴스를 생성했기 때문에 결과가 모두 False다.

그런데 다음 코드는 어떨까?

```
p1 = Person()
p2 = p1

print(p1 is p2)
```

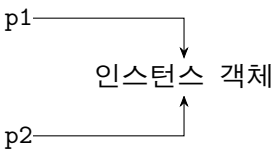
출력:

```
True
```

이번에는 Person()을 한 번만 호출했다.

p2 = p1은 새로운 인스턴스를 생성하는 코드가 아니다.

기존 인스턴스에 새로운 이름을 바인딩한다.



이 구조는 이전에 살펴본 이름과 객체의 관계와 동일하다.

이름을 새로 만든다고 객체가 새로 생성되는 것은 아니다.

2.3.4 인스턴스가 생성되면 클래스도 복사되는가?

여기서 중요한 질문이 하나 생긴다.

```
p1 = Person()
p2 = Person()
```

이렇게 인스턴스를 두 개 생성하면 Person 클래스 객체도 두 개로 복사될까?

아니다.

일반적인 인스턴스 생성에서는 클래스 객체가 복사되지 않는다.

두 인스턴스는 동일한 클래스 객체를 자신의 타입으로 가진다.

다음 코드로 확인할 수 있다.

```
class Person:
    pass

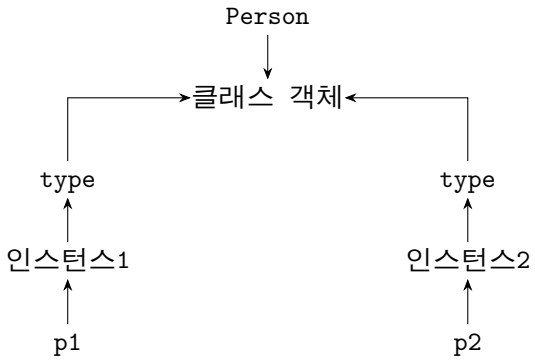
p1 = Person()
p2 = Person()

print(type(p1) is Person)
print(type(p2) is Person)
print(type(p1) is type(p2))
```

출력:

```
True
True
True
```

관계를 표현하면 다음과 같다.



두 인스턴스는 각각 독립적인 객체이지만 동일한 클래스 객체를 타입으로 참조한다.

이 구조가 중요한 이유는 메서드와 클래스 속성 때문이다.

예를 들어 다음 클래스를 보자.

```
class Person:
```

```
    def introduce(self):
        print("Hello")
```

인스턴스를 여러 개 생성해도 일반적으로 introduce 함수 객체가 인스턴스마다 복사되는 것은 아니다.

```
p1 = Person()
p2 = Person()
```

introduce 함수 객체는 클래스의 네임스페이스에 존재한다.

```
print(Person.__dict__["introduce"])
```

각 인스턴스에서 다음처럼 접근할 수 있다.

```
p1.introduce()
p2.introduce()
```

Python은 인스턴스를 통해 메서드에 접근할 때 클래스의 함수 객체를 찾아 해당 인스턴스에 바인딩된 메서드로 제공한다.

이 과정은 2.5에서 자세히 학습한다.

현재 단계에서는 다음 사실을 기억하면 충분하다.

인스턴스가 여러 개 존재하더라도

클래스에 정의된 메서드가 인스턴스마다 복사되는 것은 아니다.

2.3.5 인스턴스는 자신의 클래스를 어떻게 알 수 있는가?

앞에서 다음 코드를 사용했다.

```
type(p1)
```

그렇다면 Python은 p1이 Person의 인스턴스라는 사실을 어떻게 알까?

Python 객체는 자신의 타입에 대한 정보를 가진다.

일반적인 Python 객체에서는 __class__ 속성을 통해 해당 정보를 확인할 수 있다.

```
class Person:
    pass

p = Person()

print(p.__class__)
```

출력:

```
<class '__main__.Person'>
```

다음 코드도 확인해 보자.

```
print(p.__class__ is Person)
```

출력:

```
True
```

따라서 다음 두 표현식은 이 예제에서 같은 클래스 객체를 반환한다.

```
type(p)
p.__class__
```

다만 엄밀하게는 두 표현식의 동작이 항상 완전히 동일한 것은 아니다.

`type(p)`는 객체의 실제 타입을 확인하는 반면,

`p.__class__`는 속성 접근을 통해 얻는 값이므로 사용자 정의 클래스의 특수한 속성 처리에 영향을 받을 수 있다.

일반적인 Python 클래스를 공부하는 현재 단계에서는 둘 다 인스턴스와 클래스의 관계를 확인하는 방법으로 이해해도 충분하다.

타입과 인스턴스 관계

다음 코드를 다시 보자.

```
class Person:
    pass

p = Person()
```

각 표현식의 타입을 확인하면 다음과 같다.

```
print(type(p))
pirnt(type(Person))
```

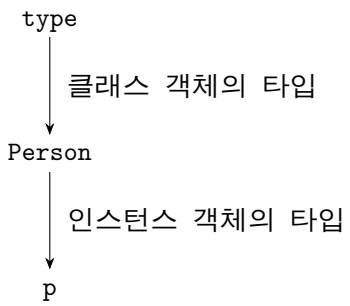
출력:

```
<class '__main__.Person'>
<class 'type'>
```

즉,

```
type(p) = Person
type(Person) = type
```

이 관계는 2.2에서 배운 내용과 연결된다.



여기서 다시 강조할 점은 이 관계가 상속 관계가 아니라 타입과 인스턴스의 관계라는 것이다.

상속은 이후 2.8에서 별도로 학습한다.

2.3.6 객체의 동일성과 동등성 - is와 ==

클래스와 인스턴스를 이해할 때 반드시 구분해야 하는 개념이 있다.

바로 객체의 동일성(Identity)과 동등성(Equality)이다.

이전에 객체와 바인딩을 공부하면서 살펴봤던 개념이기도 하다.

`is - 동일한 객체인가?`

```
class Person:
    pass
```

```
p1 = Person()
p2 = Person()
p3 = p1

print(p1 is p2)
print(p1 is p3)
```

출력:

```
False
True
```

is는 두 이름이 같은 객체를 참조하는지 확인한다.

```
== - 두 객체가 동등한가?
```

```
print(p1 == p2)
print(p1 == p3)
```

출력:

```
False
True
```

이 예제에서는 Person 클래스에 동등성 비교를 위한 별도의 메서드를 정의하지 않았다.

따라서 기본적인 객체 비교 동작에 따라 서로 다른 인스턴스는 동등하지 않은 것으로 평가된다.

하지만 ==의 동작은 클래스에서 __eq__()를 정의하여 변경할 수 있다.

다음 예제를 보자.

```
class Person:
    def __init__(self, name):
        self.name = name

    def __eq__(self, other):
        if not isinstance(other, Person):
            return NotImplemented

        return self.name == other.name
```

이제 다음 코드를 실행해 보자.

```
p1 = Person("Kim")
p2 = Person("Kim")

print(p1 is p2)
print(p1 == p2)
```

출력:

```
False
True
```

왜 이런 결과가 발생할까?

p1과 p2는 서로 다른 인스턴스 객체다.

따라서 is는 False다.

하지만 __eq__()에서는 두 인스턴스의 name 속성이 같은지 비교하도록 정의했다.

따라서 ==는 True가 된다.

정리하면 다음과 같다.

연산자	비교 기준
is	두 표현식이 동일한 객체를 참조하는가?
==	객체가 정의한 동등성 비교 결과는 무엇인가?

특히 머신러닝에서는 Tensor 객체의 ==가 원소별 비교를 수행한다는 점도 기억해 둘 필요가 있다.

따라서 Python 객체의 동일성 비교와 Tensor 값 비교는 구분해서 사용해야 한다.

2.3.7 type()과 isinstance()의 차이

이번에는 객체의 타입을 검사하는 두 방법을 비교해 보자.

type() - 객체의 실제 타입 확인

```
class Person:
    pass

p = Person()
print(type(p))
```

출력:

```
<class '__main__.Person'>
```

정확히 Person 클래스의 인스턴스인지 확인하려면 다음처럼 작성할 수 있다.

```
print(type(p) is Person)
```

출력:

```
True
```

isinstance() - 클래스 관계를 고려한 검사

```
print(isinstance(p, Person))
```

출력:

```
True
```

그런데 상속이 등장하면 차이가 발생한다.

```
class Person:
    pass

class Student(Person):
    pass

s = Student()
```

여기서 Student는 Person을 상속받는다.

다음 결과를 확인해 보자.

```
print(type(s) is Person)
print(type(s) is Student)

print(isinstance(s, Person))
print(isinstance(s, Student))
```

출력:

```
False
True
True
True
```

왜 이런 결과가 발생할까?

s의 실제 타입은 Student다.

따라서 다음 비교는 False다.

```
type(s) is Person
```

하지만 Student는 Person을 상속하므로 s는 Person의 인스턴스이기도 하다.

따라서 다음 비교는 True다.

```
isinstance(s, Person)
```

정리하면 다음과 같다.

표현식	의미
type(s) is Person	실제 타입이 정확히 Person인가?
type(s) is Student	실제 타입이 정확히 Student인가?
isinstance(s, Person)	Person 또는 그 하위 클래스의 인스턴스인가?
isinstance(s, Student)	Student 또는 그 하위 클래스의 인스턴스인가?

isinstance()에는 추상 기반 클래스나 사용자 정의 인스턴스 검사 규칙 등 추가적인 동작도 있지만,

현재는 상속 관계를 고려한다는 점을 이해하면 충분하다.

이 개념은 이후 PyTorch에서 중요하게 사용된다.

2.3.8 클래스 정의를 다시 실행하면 어떻게 되는가?

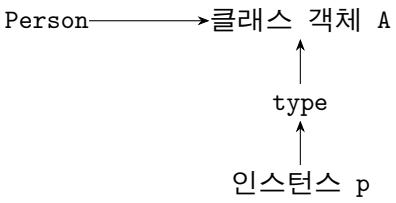
이번에는 이름과 객체의 바인딩 관점에서 조금 더 깊게 살펴보자.

다음 코드를 실행한다고 가정하자.

```
class Person:
    pass

p = Person()
```

이 시점의 관계는 다음과 같다.



그런데 같은 이름으로 클래스를 다시 정의하면 어떻게 될까?

```
class Person:
    pass
```

Python은 기존 클래스 객체를 수정하는 것이 아니라, 새로운 클래스 객체를 생성하고

Person이라는 이름을 그 객체에 다시 바인딩한다.

따라서 기존 인스턴스 p는 이전 클래스 객체를 타입으로 유지한다.

실제로 확인해 보자.

```
class Person:
    pass

p = Person()

OldPerson = Person

class Person:
    pass
```

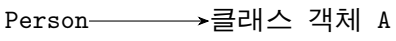
```
print(OldPerson is Person)
print(type(p) is OldPerson)
print(type(p) is Person)
```

출력:

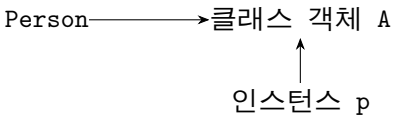
```
False
True
False
```

실행 과정을 나타내면 다음과 같다.

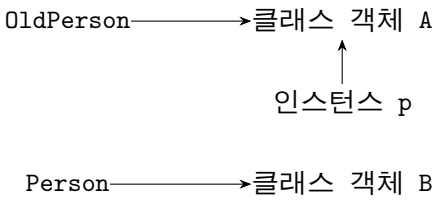
처음 클래스 정의 후



인스턴스 생성 후



클래스 재정의 후



여기서 Person이라는 이름은 새로운 클래스 객체 B를 참조하지만,

기존 인스턴스 p의 타입은 여전히 클래스 객체 A다.

이 개념은 Jupyter Notebook에서 클래스를 여러 번 정의할 때 특히 중요하다.

예를 들어 Notebook에서 모델 클래스를 정의한 셀을 다시 실행하면 새로운 클래스 객체가 만들어질 수 있다.

하지만 이전에 생성한 모델 인스턴스가 자동으로 새로운 클래스의 인스턴스로 바뀌는 것은 아니다.

따라서 다음과 같은 상황이 발생할 수 있다.

```
model = MyModel()
```

이후 MyModel 클래스 정의 셀을 다시 실행하더라도 기존 model은

이전 클래스 정의를 기반으로 생성된 인스턴스다.

이런 경우에는 새 클래스 정의를 반영하려면 일반적으로 모델 인스턴스를 다시 생성해야 한다.

2.3.9 PyTorch와의 연결 - 모델 클래스와 모델 인스턴스

이제 PyTorch 코드에 적용해 보자.

```
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10,5)

    def forward(self,x):
        return self.linear(x)
```

이 코드를 실행하면 MyModel이라는 클래스 객체가 생성된다.

아직 모델 인스턴스를 생성한 것은 아니다.

다음 코드를 실행하면

```
model = MyModel()
```

MyModel 클래스 객체가 호출되고 모델 인스턴스가 생성된다.

여기서 중요한 차이가 있다.

모델 클래스

MyModel

모델의 구조와 동작을 정의하는 클래스 객체다.

모델 인스턴스

```
model = MyModel()
```

실제 nn.Linear 계층과 파라미터를 포함하는 모델 객체다.

여러 모델 인스턴스 생성

```
model1 = MyModel()
model2 = MyModel()
```

두 모델은 같은 클래스의 인스턴스지만 서로 다른 객체다.

```
print(model1 is model2)
```

출력:

```
False
```

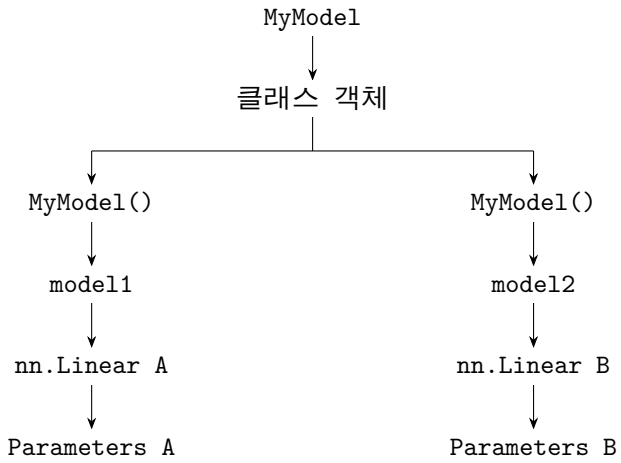
또한 각각 nn.Linear 계층도 별도로 생성된다.

```
print(model1.linear is model2.linear)
```

출력:

```
False
```

따라서 일반적으로 두 모델은 서로 독립적인 파라미터를 가진다.



여기서 중요한 것은 클래스 객체는 공유하지만,

인스턴스 내부에 생성된 계층과 파라미터는 별도로 존재한다는 점이다.

이는 딥러닝 실험에서 중요한 의미를 가진다.

예를 들어 동일한 모델 아키텍처를 사용하면서 서로 다른 초기 파라미터를 가진 모델을 여러 개 생성할 수 있다.

```
model1 = MyModel()
model2 = MyModel()
```

두 모델은 같은 아키텍처를 가지지만, 별도의 파라미터 객체를 가진다.

초기 가중치의 실제 값이 서로 다른지는 난수 상태와 초기화 방식에 따라 달라진다.

이러한 구조는 여러 모델을 비교하거나 서로 다른 학습 실험을 수행할 때 활용된다.

2.3.10 핵심 정리

이번 학습에서는 클래스 객체와 인스턴스 객체의 관계를 살펴보았다.

핵심 내용을 정리하면 다음과 같다.

> 클래스 객체와 인스턴스 객체는 서로 다른 객체다.

```
Person
```

```
Person()
```

Person은 클래스 객체를 참조하고, Person()은 클래스 객체를 호출하는 표현식이다.

> 인스턴스는 클래스 객체를 호출하여 생성할 수 있다.

```
p = Person()
```

일반적인 클래스에서는 새로운 인스턴스가 생성되고 p에 바인딩된다.

> 하나의 클래스에서 여러 인스턴스를 생성할 수 있다.

```
p1 = Person()
p2 = Person()
```

일반적으로 두 인스턴스는 서로 다른 객체다.

> 인스턴스를 생성한다고 클래스 객체가 복사되는 것은 아니다.

```
type(p1) is type(p2)
```

같은 클래스에서 생성한 인스턴스들은 동일한 클래스 객체를 타입으로 가진다.

> 이름의 재바인딩은 객체의 재생성과 다르다.

```
p2 = p1
```

이 코드는 새로운 인스턴스를 생성하지 않는다.

> is와 ==는 서로 다른 비교 연산이다.

```
p1 is p2
p1 == p2
```

is는 객체의 동일성을 검사하고, ==는 객체의 동등성을 검사한다.

> type()과 isinstance()는 검사 목적이 다르다.

```
type(p) is Person
```

```
isinstance(p, Person)
```

전자는 실제 타입을 정확히 비교하고, 후자는 일반적으로 상속 관계까지 고려한다.

> 클래스를 다시 정의하면 새로운 클래스 객체가 생성된다.

기존 인스턴스는 이전 클래스 객체를 자신의 타입으로 유지한다.

> PyTorch 모델도 클래스와 인스턴스를 구분해야 한다.

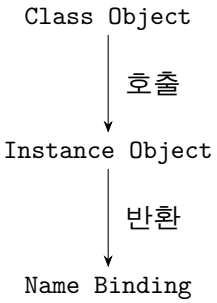
```
MyModel
MyModel()
```

`MyModel`은 모델 클래스 객체이고,

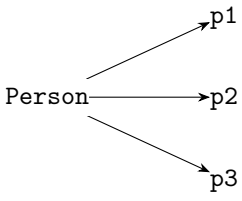
`MyModel()`은 일반적으로 실제 계층과 파라미터를 가진 모델 인스턴스를 생성한다.

> 2.3에서 반드시 이해해야 할 결론

이번 학습의 핵심 구조는 다음과 같다.



그리고 하나의 클래스 객체를 기반으로 여러 인스턴스가 생성될 수 있다.



여기서 `Person`은 클래스 객체이고,

`p1`, `p2`, `p3`는 각각 해당 클래스의 인스턴스를 참조하는 이름이다.

클래스는 인스턴스의 타입과 동작을 정의하고, 인스턴스는 그 클래스를 기반으로 생성된 개별 객체다.

다음 2.4. 인스턴스 속성과 네임스페이스에서는

지금까지 구분한 클래스 객체와 인스턴스 객체를 기반으로

`self.name`이 실제로 어디에 저장되는지,

`__dict__`가 어떤 역할을 하는지,

Python이 속성을 어떻게 찾아가는지 살펴볼 것이다.

2.4 인스턴스 속성과 네임스페이스 - `self.name`은 어디에 저장되는가?

2.2에서는 `class`문을 실행하면 클래스 객체가 생성된다는 사실을 학습했고,

2.3에서는 클래스 객체와 인스턴스 객체가 서로 다른 객체라는 사실을 확인했다.

이번 2.4에서는 한 단계 더 나아가 인스턴스가 자신의 데이터를 어떻게 보관하고,

Python이 그 데이터에 어떻게 접근하는지 살펴보자.

특히 다음 코드에 집중할 것이다.

```
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Kim")
```

우리는 일반적으로 `self.name = name`을 인스턴스에 속성을 저장하는 코드라고 설명한다.

그런데 여기서 몇 가지 질문이 생긴다.

1. `self.name`에서 `self`와 `name`은 각각 무엇인가?

2. `name`이라는 속성은 실제로 어디에 저장되는가?

3. `p.name`과 `name`은 왜 다른가?

4. p1과 p2가 서로 다른 값을 저장할 수 있는 이유는 무엇인가?

5. p.name을 읽을 때 Python은 어디에서 값을 찾는가?

6. 클래스 속성과 인스턴스 속성이 같은 이름을 가지면 어떻게 되는가?

7. 이 구조가 PyTorch의 self.linear와 어떻게 연결되는가?

이번 학습의 핵심은 이름, 객체, 네임스페이스, 속성 접근을 하나의 구조로 연결하는 것이다.

2.4.1 개념 정의 - 속성과 네임스페이스

먼저 두 용어를 구분한다.

속성(attribute)은 객체에 연결된 이름으로, 점(.)을 사용하는 속성 접근 문법을 통해 읽거나 설정할 수 있는 값이다.

네임스페이스(Namespace)는 이름과 객체 사이의 바인딩을 관리하는 공간이다.

예를 들어 다음 코드를 보자.

```
name = "Kim"
```

여기서 name은 현재 네임스페이스에 존재하는 이름이다.

반면 다음 코드는 다르다.

```
p.name = "Kim"
```

여기서 name은 객체 p의 속성 이름이다.

두 표현식은 모두 "Kim"이라는 문자열 객체를 참조할 수 있지만, 이름을 관리하는 위치와 접근 방법이 다르다.

표현식	의미	이름을 찾는 방식
name	현재 실행 환경의 일반적인 이름	스코프 및 이름 탐색 규칙
p.name	p 객체의 name 속성	객체의 속성 접근 규칙
Person.name	Person 클래스 객체의 name 속성	클래스 객체의 속성 접근 규칙

따라서 name과 p.name은 같은 이름을 공유하더라도 별개의 바인딩일 수 있다.

직접 확인해 보자.

```
name = "Lee"

class Person:
    pass

p = Person()
p.name = "Kim"

print(name)
print(p.name)
```

실행 결과:

```
Lee
Kim
```

왜 다른 값이 출력될까?

name은 현재 모듈의 전역 네임스페이스에 있는 이름이고,

p.name은 인스턴스 객체에 연결된 속성이기 때문이다.

두 이름은 서로 다른 공간에서 관리된다.

2.4.2 인스턴스 속성은 어디에 저장되는가? - __dict__

Python의 일반적인 사용자 정의 클래스 인스턴스는

속성들을 __dict__라는 딕셔너리에 저장한다.

다음 코드를 보자.

```
class Person:
    pass

p = Person()

p.name = "Kim"
p.age = 30

print(p.__dict__)
```

실행 결과:

```
{'name': 'Kim', 'age': 30}
```

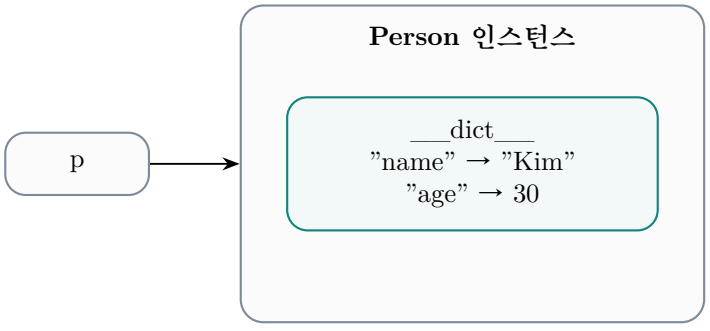
여기서 중요한 사실을 알 수 있다.

우리는 다음처럼 속성을 설정했다.

```
p.name = "Kim"
p.age = 30
```

그런데 인스턴스의 `__dict__`를 확인하니 이름과 값이 딕셔너리 형태로 관리되고 있다.

개념적으로 다음과 같은 구조다.



일반적인 사용자 정의 인스턴스의 속성 저장 구조를 단순화한 그림

여기서 `p`는 인스턴스 객체를 참조하는 이름이고, 인스턴스는 자신의 속성 정보를 `__dict__`에 보관한다.

따라서 일반적인 경우 다음 두 표현식으로 같은 값을 확인할 수 있다.

```
print(p.name)
print(p.__dict__["name"])
```

실행 결과:

```
Kim
Kim
```

하지만 두 표현식이 내부적으로 항상 동일하게 처리되는 것은 아니다.

`p.name`은 Python의 속성 접근 규칙을 따르고, `p.__dict__["name"]`은 딕셔너리에 직접 접근한다.

이 차이는 이후 속성 탐색과 디스크립터를 이해할 때 중요해진다.

또한 모든 객체가 `__dict__`를 가지는 것은 아니다.

예를 들어 `__slots__`를 사용하는 클래스에서는 인스턴스 속성을 다른 방식으로 관리할 수 있다.

현재는 일반적인 사용자 정의 클래스에서 인스턴스 속성이 `__dict__`에 저장된다는 원리를 이해하면 충분하다.

2.4.3 `self.name = name`의 정확한 의미

이제 처음의 코드로 돌아가 보자.

```
class Person:
    def __init__(self, name):
        self.name = name
```

이 코드에서 `self.name = name`을 이해하려면 왼쪽과 오른쪽을 구분해야 한다.

```
self.name = name
#  왼쪽       오른쪽
```

오른쪽 `name`

`__init__()` 함수의 매개변수 이름이다.

함수가 호출될 때 전달된 객체를 참조하도록 바인딩된다.

왼쪽 self

현재 초기화 중인 인스턴스 객체를 참조하는 매개변수다.

왼쪽 self.name

self가 참조하는 인스턴스에 name이라는 속성을 설정하라는 의미다.

예를 들어 다음 코드를 실행한다고 생각해 보자.

```
p = Person("Kim")
```

일반적인 인스턴스 생성 과정에서 __init__()이 호출될 때는 개념적으로 다음과 같은 인수 전달이 이루어진다.

```
Person.__init__(새로_생성된_인스턴스, "Kim")
```

따라서 __init__() 내부에서는 다음과 같은 관계가 성립한다.

self → 새로 생성된 Person 인스턴스

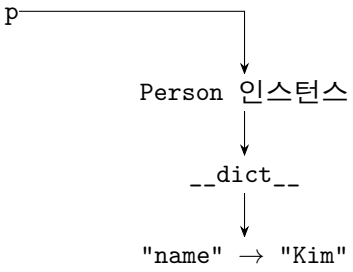
name → "Kim" 문자열 객체

이 상태에서 다음 코드가 실행된다.

```
self.name = name
```

오른쪽의 name을 평가해 "Kim" 객체를 가져오고, self가 참조하는 인스턴스에 name 속성을 설정하는 것이다.

실행 후에는 다음과 같은 구조가 형성된다.



실제로 확인해 보자.

```
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Kim")

print(p.__dict__)
```

실행 결과:

```
{'name': 'Kim'}
```

> 매개변수 name과 인스턴스 속성 self.name은 다른 바인딩이다

이 점이 특히 중요하다.

다음 코드를 보자.

```
class Person:
    def __init__(self, name):
        self.name = name
        name = "Lee"

p = Person("Kim")

print(p.name)
```

실행 결과:

```
Kim
```

왜 "Lee"가 출력되지 않을까?

name="Lee"는 지역 변수 name을 다른 문자열 객체에 재바인딩하는 코드이기 때문이다.

이미 설정한 self.name 속성을 변경하는 코드가 아니다.

반대로 다음처럼 작성하면 결과가 달라진다.

```
class Person:
    def __init__(self, name):
        self.name = name
        self.name = "Lee"

p = Person("Kim")
```

```
print(p.name)
```

실행 결과:

```
Lee
```

이번에는 self.name 속성 자체를 다시 설정했기 때문이다.

즉,

$$\texttt{name} \neq \texttt{self.name}\text{이라는 바인딩}$$

두 표현식이 같은 객체를 참조할 수는 있지만, 동일한 이름 저장 위치를 의미하지는 않는다.

2.4.4 인스턴스마다 서로 다른 속성을 가질 수 있는 이유

2.3에서 하나의 클래스로 여러 인스턴스를 생성할 수 있다는 사실을 배웠다.

```
class Person:
    pass

p1 = Person()
p2 = Person()
```

이때 p1과 p2는 서로 다른 인스턴스 객체다.

따라서 일반적인 클래스에서는 각 인스턴스가 자신의 속성 저장 공간을 가질 수 있다.

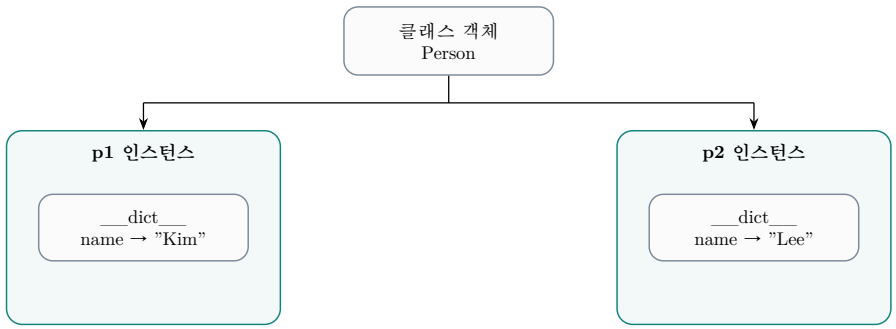
```
p1.name = "Kim"
p2.name = "Lee"

print(p1.__dict__)
print(p2.__dict__)
```

실행 결과:

```
{'name': 'Kim'}
{'name': 'Lee'}
```

관계를 나타내면 다음과 같다.



두 인스턴스는 동일한 클래스에 속하지만 별도의 속성 저장 공간을 가진다.

여기서 클래스 객체는 하나지만, 인스턴스는 두 개다.

그리고 각 인스턴스는 서로 다른 `__dict__`를 가지고 있다.

따라서 다음 코드도 가능하다.

```
p1.age = 30
p2.city = "Seoul"

print(p1.__dict__)
print(p2.__dict__)
```

실행 결과:

```
{'name': 'Kim', 'age': 30}
{'name': 'Lee', 'city': 'Seoul'}
```

이 예제에서 `p1`은 `age` 속성을 가지고 있지만 `city` 속성은 없다.

반면 `p2`는 `city` 속성을 가지고 있지만 `age` 속성은 없다.

즉, 동일한 클래스의 인스턴스라고 해서 반드시 동일한 인스턴스 속성을 가져야 하는 것은 아니다.

다만 실무에서는 객체의 상태를 예측하기 쉽도록 `__init__()`에서 필요한 인스턴스 속성을 일관되게 초기화하는 경우가 많다.

2.4.5 인스턴스 속성은 언제 생성되는가?

앞에서는 `__init__()` 내부에서 속성을 생성하였다.

```
class Person:

    def __init__(self, name):
        self.name = name
```

하지만 Python에서는 반드시 `__init__()` 안에서만 인스턴스 속성을 생성해야 하는 것은 아니다.

다음 코드도 가능하다.

```
class Person:
    pass

p = Person()

p.name = "Kim"
p.age = 30
```

`Person` 클래스에는 `name`이나 `age`를 정의하지 않았지만, 인스턴스에 속성을 설정할 수 있다.

왜냐하면 일반적인 Python 인스턴스는 동적으로 속성을 추가할 수 있기 때문이다.

다음 코드를 확인해 보자.

```
print(p.__dict__)
```

실행 결과:

```
{'name': 'Kim', 'age': 30}
```

따라서 속성의 생성 시점은 반드시 인스턴스 생성 시점과 일치하지 않는다.

다만 다음 두 상황은 구분해야 한다.

인스턴스 생성

```
p = Person()
```

새로운 인스턴스 객체가 생성되는 과정이다.

인스턴스 속성 설정

```
p.name = "Kim"
```

이미 존재하는 인스턴스에 속성을 설정하는 과정이다.

이 둘은 서로 다른 동작이다.

속성이 없는 상태에서 읽으면?

```
class Person:
    pass

p = Person()

print(p.name)
```

이 코드는 오류를 발생시킨다.

```
AttributeError: 'Person' object has no attribute 'name'
```

반대로 속성을 설정한 후에는 읽을 수 있다.

```
p.name = "Kim"

print(p.name)
```

실행 결과:

```
Kim
```

이처럼 속성은 객체가 생성되었다는 이유만으로 자동으로 존재하는 것이 아니다.

해당 속성이 실제로 설정되어 있거나, 클래스 등에서 속성 탐색을 통해 발견될 수 있어야 한다.

2.4.6 Python은 p.name을 어떻게 처리하는가?

이제 중요한 질문으로 넘어간다.

```
p.name
```

Python은 이 표현식을 어떻게 처리할까?

단순히 `p.__dict__["name"]`을 실행한다고 생각할 수 있지만, 실제 속성 접근 규칙은 그보다 복잡하다.

일반적인 사용자 정의 인스턴스에서는 `object.__getattribute__()`가 기본적인 속성 접근 동작을 제공한다.

여기에는 인스턴스의 속성뿐 아니라 클래스 속성, 상속 관계, 디스크립터 등이 관여할 수 있다.

우선 가장 단순한 경우부터 살펴본다.

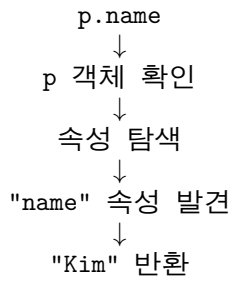
```
class Person:
    pass

p = Person()
p.name = "Kim"

print(p.name)
```

이 예제에서는 인스턴스의 `__dict__`에 `"name"`이 존재하므로 해당 값을 읽을 수 있다.

개념적으로 다음과 같이 생각할 수 있다.



하지만 클래스에도 같은 이름의 속성이 있다면 어떻게 될까?

```
class Person:
    name = "Unknown"

p = Person()

print(p.name)
```

실행 결과:

```
Unknown
```

이번에는 인스턴스에 직접 name 속성을 설정하지 않았다.

그런데도 p.name으로 값을 읽을 수 있다.

왜냐하면 Python이 인스턴스에서 속성을 찾지 못했을 때 클래스에서도 속성을 탐색할 수 있기 때문이다.

다음 코드를 확인해 보자.

```
print(p.__dict__)
print(Person.__dict__["name"])
```

실행 결과:

```
{}
Unknown
```

2.4.7 클래스 속성과 인스턴스 속성이 같은 이름을 가지면?

다음 코드를 보자.

```
class Person:
    name = "Unknown"

p = Person()

print(p.name)
```

실행 결과:

```
Unknown
```

여기까지는 클래스 속성에서 값을 읽었다.

그런데 다음 코드를 실행하면 어떻게 될까?

```
p.name = "Kim"
```

```
print(p.name)
print(Person.name)
```

실행 결과:

```
Kim
Unknown
```

왜 이런 결과가 발생할까?

`p.name = "Kim"`은 일반적인 속성 설정 규칙에 따라 인스턴스의 `name` 속성을 생성하기 때문이다.

클래스 속성 `Person.name`을 수정한 것이 아니다.

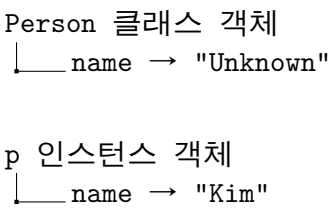
실제로 확인해 보자.

```
print(p.__dict__)
print(Person.__dict__["name"])
```

실행 결과:

```
{'name': 'Kim'}
Unknown
```

따라서 다음과 같은 구조가 형성된다.



이때 `p.name`은 인스턴스 속성 `"Kim"`을 반환하고, `Person.name`은 클래스 속성 `"Unknown"`을 반환한다.

이러한 현상을 일반적으로

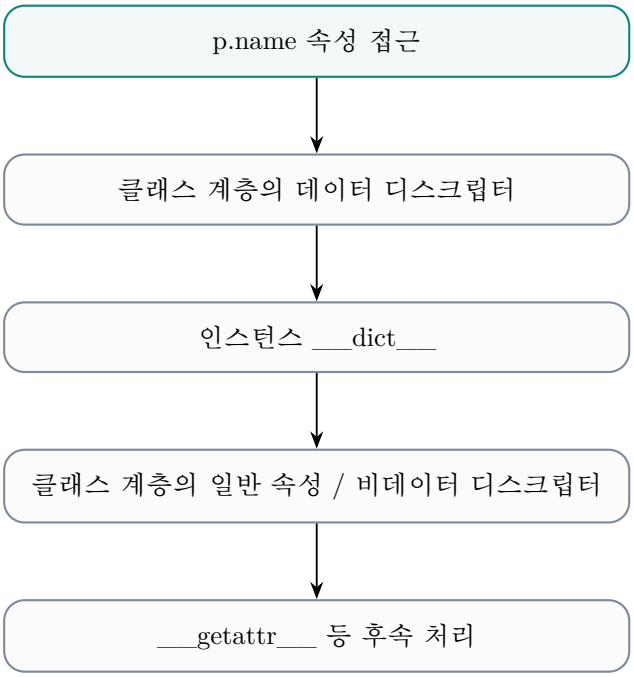
인스턴스 속성이 클래스 속성을 가린다 (Shadowing)고 설명한다.

다만 이 규칙에는 중요한 예외가 있다.

클래스에 같은 이름의 **데이터 디스크립터(Data Descriptor)**가 존재하면

인스턴스 `__dict__`보다 우선할 수 있다.

따라서 일반적인 속성 탐색 순서를 조금 더 정확하게 표현하면 다음과 같다.



일반적인 인스턴스 속성 읽기의 개념적 우선순위.
각 단계에서 속성이 발견되면 그에 맞는 처리가 이루어진다.

이 도식은 기본적인 `object.__getattribute__()` 동작을 단순화한 것이다.

클래스에서 속성 접근을 직접 재정의하면 동작이 달라질 수 있다.

지금 단계에서는 인스턴스 속성과 클래스 속성이 별도의 저장 공간에 존재하며, Python은 정해진 규칙에 따라 속성을 탐색한다는 점을 이해하면 충분하다.

2.4.8 `getattr()`, `setattr()`, `delattr()` - 속성을 동적으로 다루기

지금까지는 점(.)을 사용해 속성에 접근했다.

```
p.name
p.name = "Kim"
```

그런데 Python에서는 속성 이름을 문자열로 전달해 접근할 수도 있다.

이를 위해 다음 내장 함수를 제공한다.

함수	역할
`getattr(obj, name)`	객체의 속성 읽기
`setattr(obj, name, value)`	객체의 속성 설정
`delattr(obj, name)`	객체의 속성 삭제
`hasattr(obj, name)`	속성 접근 가능 여부 확인

예제를 살펴본다.

```
class Person:
    pass

p = Person()

setattr(p, "name", "Kim")

print(getattr(p, "name"))
```

실행 결과:

```
Kim
```

다음 두 코드는 일반적인 경우 같은 효과를 가진다.

```
p.name = "kim"

setattr(p, "name", "Kim")
```

마찬가지로 다음 두 코드는 같은 속성을 읽는다.

```
p.name

getattr(p, "name")
```

기본값을 지정할 수도 있다.

```
print(getattr(p, "age", 0))
```

실행 결과:

```
0
```

age 속성이 존재하지 않으므로 기본 값 0을 반환한다.

반면 다음 코드는 오류가 발생할 수 있다.

```
print(getattr(p, "age"))
```

getattr()에 기본값을 전달하지 않았기 때문이다.

다음 코드는 일반적인 인스턴스에서 name 속성을 삭제한다.

```
delattr(p, "name")

print(p.__dict__)
```

실행 결과:

```
{}
```

단, delattr() 역시 Python의 속성 삭제 규칙을 사용하므로 항상 단순한 딕셔너리 삭제와 동일한 것은 아니다.

왜 이런 함수들이 필요할까?

속성 이름이 실행 중에 결정되는 상황에서 유용하기 때문이다.

```
attribute_name = "name"

value = getattr(p, attribute_name, None)
```

여기서는 어떤 속성을 읽을지 문자열 변수로 지정할 수 있다.

이런 방식은 설정 객체, 데이터 변환, 프레임워크 내부 코드 등에서 활용된다.

2.4.9 Mutable 객체를 인스턴스 속성에 저장하면?

이번에는 이전에 학습한 Mutable / Immutable 개념과 연결해 보자.

```
class Person:
    def __init__(self, name):
        self.name = name
```

```
        self.hobbies = []
```

여기서 `self.hobbies`에는 리스트 객체가 저장된다.

정확히는 인스턴스 속성이 리스트 객체를 참조하게 되는 것이다.

```
p = Person("Kim")
p.hobbies.append("Reading")
```

이때 `append()`는 리스트 객체를 변경하는 메서드다.

따라서 다음 결과를 얻을 수 있다.

```
print(p.hobbies)
```

실행 결과:

```
['Reading']
```

그런데 다음 두 코드는 서로 다른 동작이다.

```
p.hobbies.append("Music")
```

```
p.hobbies = ["Music"]
```

첫 번째 코드는 기존 리스트 객체를 변경한다.

두 번째 코드는 새로운 리스트 객체를 생성하고 `p.hobbies` 속성이 그 객체를 참조하도록 재설정한다.

이 차이를 `id()`로 확인할 수 있다.

```
p = Person("Kim")

before = id(p.hobbies)

p.hobbies.append("Reading")

after_append = id(p.hobbies)

p.hobbies = ["Music"]

after_rebind = id(p.hobbies)

print(before == after_append)
```

```
print(before == after_rebind)
```

실행 결과:

```
True
False
```

첫 번째는 기존 리스트 객체를 변경했기 때문에 객체의 동일성이 유지된다.

두 번째는 새 리스트를 만들어 속성을 재바인딩했기 때문에 다른 객체를 참조한다.

이것은 우리가 앞서 함수 인수 전달, 리스트 등을 공부하며 살펴본 원리와 같다.

> 서로 다른 인스턴스의 리스트는 공유될까?

```
p1 = Person("Kim")
p2 = Person("Lee")

print(p1.hobbies is p2.hobbies)
```

실행 결과:

```
False
```

왜냐하면 `__init__()`이 호출될 때마다 다음 표현식이 새 리스트 객체를 생성하기 때문이다.

```
self.hobbies = []
```

따라서 각 인스턴스는 별도의 리스트를 참조한다.

반대로 클래스 속성으로 리스트를 정의하면 공유 문제가 발생할 수 있다.

```
class Person:
    hobbies = []


p1 = Person()
p2 = Person()

p1.hobbies.append("Reading")
```

```
print(p2.hobbies)
```

실행 결과:

```
['Reading']
```

두 인스턴스가 같은 클래스 속성의 리스트 객체를 참조하기 때문이다.

이 문제는 2.7. 클래스 속성과 인스턴스 속성에서 더 깊게 분석할 것이다.

2.4.10 PyTorch와의 연결 - self.linear는 어디에 저장되는가?

이제 PyTorch 모델 코드를 살펴보자.

```
import torch.nn as nn


class MyModel(nn.Module):

    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)
```

여기서 다음 표현식에 주목하자.

```
self.linear = nn.Linear(10, 5)
```

일반적인 Python 클래스였다면 `self.linear`는 인스턴스 속성으로 저장되는 경우가 많다.

하지만 PyTorch의 `nn.Module`에서는 조금 더 특별한 처리가 이루어진다.

`nn.Module`은 `__setattr__()`을 재정의하여, `nn.Module`의 하위 모듈이 인스턴스 속성으로 설정될 때 이를 등록하도록 구성되어 있다.

따라서 `self.linear`는 일반적인 인스턴스 딕셔너리에 단순히 저장되는 것이 아니라, PyTorch가 관리하는 하위 모듈 저장 구조에 등록된다.

실제로 확인해 보자.

```
model = MyModel()

print(model._modules)
```

개념적인 출력:

```
{'linear': Linear(in_features=10, out_features=5, bias=True)}
```

이때 `_modules`는 PyTorch가 하위 모듈을 관리하기 위해 사용하는 내부 구조다.

다음 코드를 통해 등록된 하위 모듈에 접근할 수 있다.

```
print(model.linear)
```

출력:

```
Linear(in_features=10, out_features=5, bias=True)
```

여기서 중요한 질문이 생긴다.

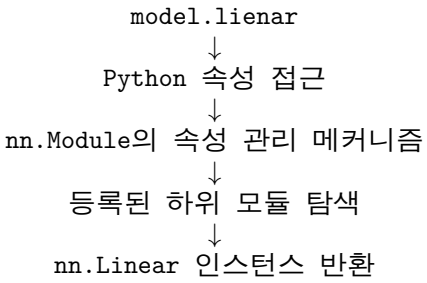
`model.linear`가 일반적인 `model.__dict__`에 직접 저장되지 않았는데도 왜 접근할 수 있을까?

그 이유는 `nn.Module`이 속성 접근과 설정을 위한 특수 메서드를 활용하기 때문이다.

일반적인 Python 객체에서는 속성 접근이 `__getattribute__()`를 통해 처리되고, 속성을 찾지 못한 경우 `__getattr__()`이 호출될 수 있다.

PyTorch의 `nn.Module`은 이러한 메커니즘을 이용해 등록된 하위 모듈과 파라미터에 접근할 수 있도록 구성되어 있다.

따라서 다음과 같은 관계가 형성된다.



이 구조 덕분에 PyTorch는 모델 내부의 하위 모듈과 파라미터를 체계적으로 관리할 수 있다.

예를 들어 다음 메서드들이 등록된 객체들을 활용한다.

```
model.parameters()
model.state_dict()
model.to("cpu")
```

여기서 핵심은 다음과 같다.

PyTorch의 self.linear도

Python의 속성 접근 문법을 사용하지만,

실제 저장과 탐색 과정은

nn.Module이 정의한 객체 관리 규칙에 의해 확장된다.

따라서 Python의 기본적인 속성 접근 규칙을 이해하면 PyTorch의 모델 등록 구조도 이해하기 쉬워진다.

2.4.11 핵심 정리

이번 학습에서는 인스턴스 속성과 네임스페이스 관계를 살펴보았다.

속성은 객체에 연결된 이름이다.

```
p.name
```

여기서 name은 p 객체의 속성 이름이다.

일반적인 사용자 정의 인스턴스는 __dict__에 속성을 저장한다.

```
p.__dict__
```

인스턴스에 직접 설정된 일반적인 속성들을 확인할 수 있다.

일반 변수와 인스턴스 속성은 다른 바인딩이다.

```
name = "Kim"

self.name = name
```

오른쪽 name은 지역 이름이고, 왼쪽 self.name은 인스턴스 속성이다.

각 인스턴스는 독립적인 속성 저장 공간을 가질 수 있다.

```
p1.name = "Kim"
p2.name = "Lee"
```

두 인스턴스의 속성은 서로 독립적으로 설정할 수 있다.

인스턴스 속성은 __init__() 밖에서도 설정할 수 있다.

```
p.age = 30
```

일반적인 Python 인스턴스는 동적으로 속성을 추가할 수 있다.

속성 접근은 단순한 딕셔너리 조회보다 복잡하다.

```
p.name
```

Python은 클래스 속성, 디스크립터, 상속 관계 등을 고려해 속성을 탐색한다.

getattr(), setattr(), delattr()로 속성을 동적으로 다룰 수 있다.

```
getattr(p, "name")
setattr(p, "age", 30)
delattr(p, "age")
```

Mutable 객체를 속성에 저장할 때는 변경과 재바인딩을 구분해야 한다.

```
p.hobbies.append("Reading")
p.hobbies = ["Music"]
```

첫 번째는 기존 객체의 변경이고, 두 번째는 속성의 재바인딩이다.

PyTorch는 Python의 속성 관리 메커니즘을 확장한다.

```
self.linear = nn.Linear(10, 5)
```

nn.Module은 하위 모듈을 등록하고 관리하기 위해 속성 설정 및 접근 메커니즘을 활용한다.

2.4.12 2.4에서 반드시 이해해야 할 결론

이번 학습에서 가장 중요한 관계는 다음과 같다.

Instance Object → Attribute Namespace → Name-Object Binding

일반적인 Python 인스턴스에서는 이 구조를 다음처럼 확인할 수 있다.

```
p.__dict__
```

그리고 다음 표현식은

```
sefl.name = name
```

지역 이름 name이 참조하는 객체를, self가 참조하는 인스턴스의 name 속성에 연결하는 동작이다.

이것이 2.4의 핵심이다.

2.5 메서드와 self - 인스턴스 메서드는 어떻게 호출되는가?

2.4에서는 인스턴스 속성과 네임스페이스를 학습했다.

특히 self.name = name이 지역 변수 name의 값을 인스턴스 속성에 연결하는 동작이라는 점을 확인했다.

이번 2.5에서는 클래스에 정의된 함수가 인스턴스를 통해 호출될 때 어떤 일이 발생하는지 살펴볼 것이다.

이번 학습의 출발점은 다음 코드다.

```
class Person:
    def introduce(self):
        print("Hello")

p = Person()
p.introduce()
```

여기서 가장 중요한 질문은 하나다.

`introduce()`에는 `self`라는 매개변수가 있는데, 왜 `p.introduce()`에서는 인수를 전달하지 않아도 되는가?

이 질문에 답하려면 함수 객체, 속성 탐색, 바운드 메서드(Bound Method), 인수 바인딩을 연결해서 이해해야 한다.

2.5.1 개념 정의 - 함수와 메서드는 무엇이 다른가?

먼저 일반 함수를 살펴보자.

```
def introduce():
    print("Hello")
```

이 코드를 실행하면 Python은 함수 객체를 생성하고 `introduce`라는 이름에 바인딩한다.

```
introduce()
```

이 표현식은 `introduce`라는 이름이 참조하는 함수 객체를 호출하는 것이다.

그렇다면 다음 코드는 어떨까?

```
class Person:
    def introduce(self):
        print("Hello")
```

여기서도 `def`가 실행되면 함수 객체가 생성된다.

다만 그 함수 객체가 모듈의 전역 이름이 아니라 클래스의 네임스페이스에 `introduce`라는 이름으로 저장된다는 차이가 있다.

```
print(Person.__dict__["introduce"])
```

출력 예시:

```
<function Person.introduce at 0x...>
```

즉, `Person` 클래스에 정의된 `introduce`도 본래는 함수 객체다.

그런데 이 함수를 인스턴스를 통해 접근하면 Python의 메서드 바인딩 규칙이 적용된다.

표현식	얻는 객체
`Person.__dict__["introduce"]`	클래스에 저장된 원래 함수 객체
`Person.introduce`	이 예제에서는 같은 함수 객체
`p.introduce`	p에 바인딩된 메서드 객체
`p.introduce()`	바운드 메서드를 호출한 결과

여기서 중요한 구분은 클래스에 함수가 저장되어 있다는 사실과, 인스턴스를 통해 그 함수에 접근할 때 메서드 바인딩이 일어난다는 사실이다.

2.5.2 메서드는 왜 필요한가?

다음과 같이 사람의 이름을 출력하는 일반 함수를 생각해 보자.

```
def introduce(person):
    print(f"Hello, {person.name}")

class Person:
    pass

p = Person()
p.name = "Kim"

introduce(p)
```

실행 결과:

```
Hello, Kim
```

여기서는 `introduce()`가 어떤 사람의 정보를 사용할지 알 수 있도록 `p`를 인수로 전달했다.

그런데 클래스에 함수를 정의하면 다음과 같이 작성할 수 있다.

```
class Person:
    def introduce(self):
        print(f"Hello, {self.name}")

p = Person()
```

```
p.name = "Kim"

p.introduce()
```

실행 결과:

```
Hello, Kim
```

이 코드는 p라는 객체에 연결된 동작을 호출한다는 의미를 명확하게 표현한다.

객체지향 프로그래밍에서는 이렇게 객체의 상태와 그 상태를 사용하는 동작을 하나의 인터페이스로 구성할 수 있다.

다만 Python에서는 클래스에 정의된 모든 함수가 반드시 인스턴스의 상태를 사용해야 하는 것은 아니다.

인스턴스 상태가 필요 없는 경우에는 정적 메서드 등을 사용할 수도 있으며 이 내용은 2.11에서 다룬다.

2.5.3 self는 무엇인가?

다음 코드를 다시 보자.

```
class Person:
    def introduce(self):
        print(self.name)

p = Person()
p.name = "Kim"

p.introduce
```

여기서 self는 메서드 호출의 대상이 되는 인스턴스를 전달받는 매개변수다.

self 자체는 Python의 예약어가 아니다.

예를 들어 다음 코드도 문법적으로 가능하다.

```
class Person:
    def introduce(this):
        print(this.name)

p = Person()
```

```
p.name = "Kim"

p.introduce()
```

결과는 동일하게 Kim이 출력된다.

하지만 Python에서는 인스턴스 메서드의 첫 번째 매개변수 이름으로 self를 사용하는 것이 관례이다.

따라서 특별한 이유가 없다면 반드시 이 관례를 따르는 것이 좋다.

> self는 클래스 객체를 의미하는가?

아니다.

```
class Person:
    def introduce(self):
        print(self)
        print(type(self))

p = Person()
p.introduce()
```

출력 예시:

```
<__main__.Person object at 0x...>
<class '__main__.Person'>
```

self는 Person 클래스 객체가 아니라 p가 참조하는 인스턴스 객체다.

다음 코드로 동일성을 확인할 수도 있다.

```
class Person:
    def check(self, other):
        print(self is other)


p = Person()
p.check(p)
```

실행 결과:

```
True
```

이 호출에서 self와 other는 모두 동일한 인스턴스 객체를 참조한다.

다만 두 매개변수는 별개의 지역 이름이다.

2.5.4 `p.introduce()`는 어떻게 실행되는가?

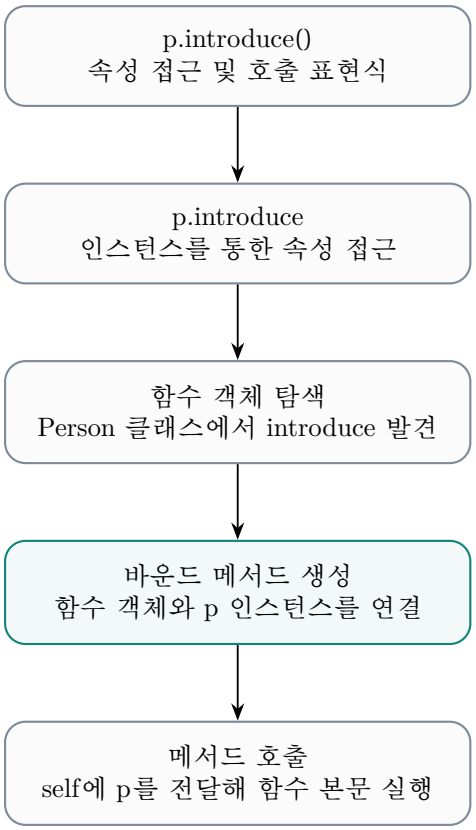
이제 이번 학습의 핵심으로 들어가 보자.

```
class Person:
    def introduce(self):
        print("Hello")

p = Person()

p.introduce()
```

Python은 `p.introduce()`를 처리할 때 다음 과정을 거친다.



일반적인 인스턴스 메서드 호출 과정을 단순화한 그림

이 과정에서 중요한 부분은 세 번째와 네 번째 단계다.

Python은 클래스에서 `introduce` 함수 객체를 찾은 뒤, 인스턴스 `p`와 연결된 바운드 메서드를 제공한다.

그 결과 `p.introduce()`를 호출하면 `p`가 첫 번째 인수로 전달되는 효과가 발생한다.

따라서 다음 두 호출은 이 예제에서 동일한 동작을 수행한다.

```
p.introduce()

Person.introduce(p)
```

두 번째 코드는 클래스에 저장된 함수 객체를 직접 호출하면서 `p`를 명시적으로 전달한 것이다.

첫 번째 코드는 Python이 인스턴스와 함수를 연결한 바운드 메서드를 통해 호출한 것이다.

직접 확인해 보자.

```
class Person:
    def introduce(self):
        print("Hello")

p = Person()
print(Person.introduce)
print(p.introduce)
```

출력 예시:

```
<function Person.introduce at 0x...>
<bound method Person.introduce of <__main__.Person object at 0x...>>
```

여기서 두 번째 출력에 주목한다.

```
bound method
```

이 것이 바로 바운드 메서드다.

2.5.5 바운드 메서드의 내부 구조 - __self__와 __func__

바운드 메서드는 단순히 함수 이름을 다르게 표현한 것이 아니다.

원래 함수 객체와 연결된 인스턴스 객체를 함께 참조하는 별도의 메서드 객체다.

다음 코드를 보자.

```
class Person:
    def introduce(self):
        print(self.name)

p = Person()
p.name = "Kim"

method = p.introduce
```

이제 method가 무엇을 참조하는지 확인해 보자.

```
print(method.__self__ is p)
print(method.__func__ is Person.introduce)
```

실행 결과:

```
True
True
```

각 속성의 의미는 다음과 같다.

속성	의미
method.__self__	메서드에 연결된 인스턴스 객체
method.__func__	실제 호출할 원래 함수 객체

이를 그림으로 나타내면 다음과 같다.



바운드 메서드는 호출 대상 인스턴스와 원래 함수 객체를 참조한다.

따라서 다음 코드를 실행하면

```
method()
```

개념적으로 다음과 같은 호출이 이루어진다.

```
method.__func__(method.__self__)
```

즉,

```
Person.introduce(p)
```

와 같은 효과를 갖는 것이다.

여기서 이전에 배운 함수 객체와 이름의 바인딩 개념이 다시 등장한다.

method는 바운드 메서드 객체를 참조하는 이름일 뿐이다.

> 바운드 메서드는 변수에 저장할 수도 있다.

```
method = p.introduce

method()
```

실행 결과:

```
Kim
```

이것이 가능한 이유는 바운드 메서드 역시 Callable 객체이기 때문이다.

또한 method가 인스턴스 p를 참조하고 있으므로, 이후에 p라는 이름을 다른 객체에 재바인딩하더라도 기존 바운드 메서드는 원래 인스턴스에 연결되어

있다.

```
original = p
method = p.introduce

p = Person()
p.name = "Lee"

method()
```

실행 결과:

```
Kim
```

method는 새로운 p가 아니라 원래 인스턴스를 참조하기 때문이다.

2.5.6 메서드 바인딩은 왜 발생하는가? - 디스크립터 프로토콜

여기서 조금 더 깊게 들어가 보자.

앞에서 Python은 인스턴스를 통해 함수에 접근할 때 바운드 메서드를 제공한다고 설명하였다.

그런데 Python은 어떻게 클래스에 저장된 함수 객체를 바운드 메서드로 변환할까?

그 핵심에는 **디스크립터 프로토콜**(Descriptor Protocol)이 있다.

Python의 일반적인 함수 객체는 __get__() 메서드를 제공한다.

클래스에 저장된 함수 객체에 인스턴스를 통해 접근하면, 속성 탐색 과정에서 이 __get__()이 활용되어 바운드 메서드가 만들어진다.

다음 코드를 살펴보자.

```
class Person:
    def introduce(self):
        print("Hello")

p = Person()

func = Person.__dict__["introduce"]
```

여기서 func는 클래스에 저장된 원래 함수 객체다.

다음 코드를 실행해 보자.

```
method = func.__get__(p, Person)

print(method)
```

출력 예시:

```
<bound method Person.introduce of <__main__.Person object at 0x...>>
```

이제 다음 두 표현식을 비교해 보자.

```
p.introduce

func.__get__(p, Person)
```

이 예제에서는 둘 다 p에 연결된 바운드 메서드를 얻는 효과가 있다.

실제로 확인해 보자.

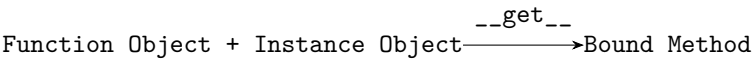
```
method = func.__get__(p, Person)

print(method.__self__ is p)
print(method.__func__ is func)
```

실행 결과:

```
True
True
```

따라서 기본적인 함수 메서드 바인딩 구조를 다음처럼 이해할 수 있다.



여기서 주의할 점은 Python이 모든 속성에 대해 무조건 바운드 메서드를 생성하는 것은 아니라는 것이다.

클래스에 저장된 일반적인 함수 객체가 디스크립터 프로토콜을 제공하기 때문에 이런 동작이 가능한 것이다.

또한 인스턴스에 함수를 직접 저장한 경우에는 동작이 달라진다.

```
def greet(self):
```

```
    print("Hello")

class Person:
    pass

p = Person()
p.greet = greet
```

이제 다음 코드를 실행하면 어떻게 될까?

```
p.greet()
```

이 경우에는 일반적으로 TypeError가 발생한다.

왜냐하면 greet 함수 객체가 인스턴스의 __dict__에 직접 저장되어 있고, 인스턴스 속성으로 꺼낸 함수에는 클래스에 정의된 함수처럼 자동 바인딩이 적용되지 않기 때문이다.

따라서 다음처럼 직접 인스턴스를 전달해야 한다.

```
p.greet(p)
```

이 차이는 매우 중요하다.

함수 객체가 어디에 저장되어 있고 어떤 속성 탐색 경로를 통해 접근되는지에 따라 메서드 바인딩 여부가 달라질 수 있다는 뜻이다.

디스크립터 프로토콜 자체는 이후 심화 학습에서 더 자세히 다룰 수 있지만, 지금은 일반적인 메서드의 바인딩 원리를 이해하는 데 집중하면 된다.

2.5.7 self와 인수 전달 - 매개변수는 어떻게 바인딩되는가?

이번에는 인수가 여러 개인 메서드를 살펴보자.

```
class Calculator:
    def add(self, a, b):
        return a + b

calc = Calculator()

result = calc.add(10,20)

print(result)
```

실행 결과:

```
30
```

여기서 add()는 세 개의 매개변수를 정의했다.

```
def add(self, a, b):
```

하지만 호출할 때는 두 개의 인수만 전달했다.

```
calc.add(10,20)
```

왜 가능할까?

calc.add가 이미 calc 인스턴스에 바인딩된 메서드이기 때문이다.

개념적으로 다음과 같이 대응한다.

```
calc.add(10,20)
```

```
Calculator.add(calc, 10, 20)
```

따라서 매개변수 바인딩은 다음과 같이 이루어진다.

매개변수	전달되는 객체
self	calc 인스턴스
a	정수 객체 10
b	정수 객체 20

이 구조는 일반 함수의 인수 전달 규칙과 동일하다.

다만 바운드 메서드에서는 연결된 인스턴스가 첫 번째 인수로 제공된다는 차이가 있을 뿐이다.

self도 일반적인 매개변수다

따라서 다음 코드도 가능하다.

```
Calculator.add(calc, 10, 20)
```

이때는 클래스에서 함수 객체에 직접 접근했으므로 calc를 명시적으로 전달해야 한다.

반대로 다음 코드는 오류가 발생한다.

```
Calculator.add(10,20)
```

self, a, b에 필요한 인수 세 개 중 두 개만 전달했기 때문이다.

이 예제에서는 self라는 이름 자체가 특별한 기능을 갖는 것이 아니라, 인스턴스 메서드 바인딩 규칙과 함수의 일반적인 인수 전달 규칙이 결합되어 동작한다는 점을 기억해야 한다.

2.5.8 인스턴스 메서드는 클래스에 저장되는가, 인스턴스에 저장되는가?

다음 코드를 보자.

```
class Person:
    def introduce(self):
        print("Hello")


p1 = Person()
p2 = Person()
```

여기서 introduce 함수 객체는 어디에 저장되어 있을까?

일반적으로 클래스의 네임스페이스에 저장된다.

```
print("introduce" in Person.__dict__)
print("introduce" in p1.__dict__)
print("introduce" in p2.__dict__)
```

실행 결과:

```
True
False
False
```

즉, p1과 p2의 인스턴스 딕셔너리에는 introduce 함수가 직접 저장되어 있지 않다.

하지만 다음 코드는 모두 가능하다.

```
p1.introduce()
p2.introduce()
```

왜냐하면 Python이 인스턴스에서 속성을 찾을 때 클래스에서도 속성을 탐색하고, 클래스에 정의된 함수를 인스턴스에 바인딩해 제공하기 때문이다.

다음 코드를 확인해 보자.

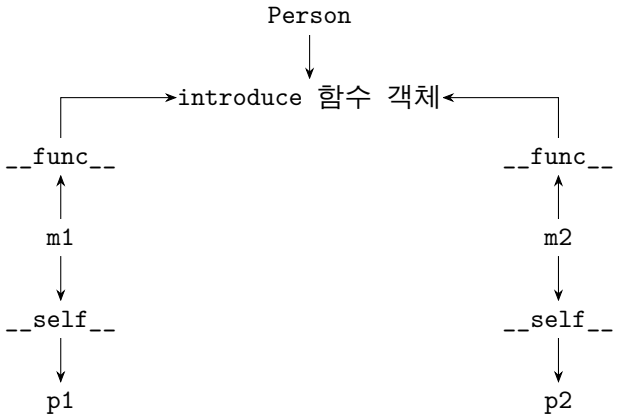
```
m1 = p1.introduce
m2 = p2.introduce

print(m1.__func__ is m2.__func__)
print(m1.__self__ is m2.__self__)
```

실행 결과:

```
True
False
```

두 바운드 메서드는 같은 함수 객체를 참조하지만, 서로 다른 인스턴스에 연결되어 있다.



이 구조 덕분에 인스턴스마다 동일한 메서드 코드를 복사해서 저장할 필요가 없다.

하나의 함수 정의를 여러 인스턴스가 각자의 상태와 결합하여 사용할 수 있다.

2.5.9 메서드 안에서 다른 메서드를 호출할 때도 self가 필요할까?

다음 코드를 살펴보자.

```
class Calculator:
    def add(self, a, b):
```

```
        return a + b

    def calculate(self, a, b):
        return self.add(a,b)


calc = Calculator()

print(calc.calculate(10,20))
```

실행 결과:

```
30
```

여기서 `calculate()` 내부에서는 다음 코드를 사용했다.

```
self.add(a,b)
```

왜 단순히 `add(a,b)`라고 작성하지 않을까?

Python의 일반적인 이름 탐색 규칙에서는 메서드 내부의 단순 이름 `add`를 자동으로 클래스 속성에서 찾아주지 않기 때문이다.

다음처럼 작성하면

```
class Calculator:
    def add(self, a, b):
        return a + b

    def calculate(self, a, b):
        return add(a, b)
```

별도의 `add` 이름이 지역 또는 전역 등 일반적인 이름 탐색 경로에 정의되어 있지 않다면 `NameError`가 발생한다.

클래스에 `add`라는 메서드가 정의되어 있다는 사실만으로 메서드 본문에서 `add`라는 단순 이름이 자동으로 연결되는 것은 아니다.

따라서 다음처럼 작성해야 한다.

```
self.add(a,b)
```

여기서는 `self`가 참조하는 인스턴스를 통해 `add` 속성을 탐색하고 바운드 메서드를 호출하는 것이다.

이 내용은 이전에 공부한 LEGB 규칙과 클래스 네임스페이스의 차이를 연결해 주는 중요한 사례다.

2.5.10 PyTorch와의 연결 - `forward(self, x)`의 의미

이제 PyTorch 코드에 적용해 보자.

```
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10,5)

    def forward(self, x):
        return self.linear(x)

model = MyModel()
x = torch.randn(2, 10)

output = model(x)
```

여기서 다음 메서드에 집중해 보자.

```
def forward(self, x):
    return self.linear(x)
```

`forward()`는 `MyModel` 클래스에 정의된 인스턴스 메서드다.

따라서 다음처럼 직접 호출할 수도 있다.

```
output = model.forward(x)
```

이 호출은 개념적으로 다음과 대응한다.

```
output = MyModel.forward(model, x)
```

여기서 매개변수 바인딩은 다음과 같다.

매개변수	전달되는 객체
self	model 인스턴스
x	입력 Tensor 객체

따라서 forward() 내부의 다음 표현식에서

```
self.linear(x)
```

self는 model 인스턴스를 참조한다.

그리고 self.linear는 해당 모델에 등록된 nn.Linear 하위 모듈을 찾아 반환한다.

그다음 nn.Linear 인스턴스를 호출해 연산을 수행하는 것이다.

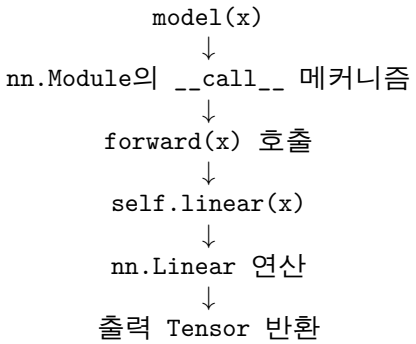
> 그렇다면 model(x)는 왜 가능한가?

```
output = model(x)
```

이 코드는 forward()를 직접 호출하는 것과는 다른 경로를 거친다.

nn.Module은 인스턴스를 함수처럼 호출할 수 있도록 __call__ 메커니즘을 제공한다.

따라서 개념적인 실행 흐름은 다음과 같다.



실제 PyTorch의 __call__ 메커니즘은 Hook 처리 등 추가적인 기능을 포함하므로 단순히 forward()만 호출하는 구조는 아니다.

따라서 일반적으로는 다음 방식을 사용해야 한다.

```
output = model(x)
```

이 구조를 정확히 이해하려면 앞으로 학습할 두 개념이 더 필요하다.

- 상속과 `super()` - `nn.Module`이 제공하는 기능을 어떻게 물려받는가?
- 특수 메서드 - `__call__()`이 왜 인스턴스 호출에 사용되는가?

그리고 최종적으로 PyTorch `nn.Module`의 객체 모델에서 전체 구조를 연결할 것이다.

2.5.11 핵심 정리

이번 학습에서는 메서드와 `self`의 관계를 학습했다.

> 클래스에 정의된 일반적인 인스턴스 메서드는 함수 객체로 저장된다.

```
class Person:
    def introduce(self):
        print("Hello")
```

`introduce`는 클래스 네임스페이스에 저장된 함수 객체다.

> 인스턴스를 통해 메서드에 접근하면 바운드 메서드를 얻는다.

```
p.introduce
```

일반적인 경우 Python은 함수 객체와 인스턴스를 연결한 바운드 메서드를 제공한다.

> `self`는 Python의 예약어가 아니다.

인스턴스를 전달받는 첫 번째 매개변수의 관례적인 이름이다.

> 다음 두 호출은 일반적인 인스턴스 메서드에서 같은 효과를 가진다.

```
p.introduce()

Person.introduce(p)
```

단, 클래스의 함수가 재정의되거나 인스턴스 속성에 의해 가려지는 등의 경우에는 항상 같은 동작이라고 단정할 수는 없다.

> 바운드 메서드는 원래 함수와 연결된 인스턴스를 참조한다.

```
method.__func__
method.__self__
```

`__func__`는 원래 함수 객체, `__self__`는 연결된 인스턴스 객체를 참조한다.

> 메서드 바인딩은 디스크립터 프로토콜과 연결된다.

일반적인 함수 객체는 `__get__()`을 제공하며, 이를 통해 바운드 메서드가 만들어질 수 있다.

> 인스턴스 메서드는 일반적으로 클래스에 저장된다.

인스턴스마다 메서드 함수 객체가 복사되는 것은 아니다.

> 메서드 내부에서 다른 인스턴스 메서드를 호출할 때는
>
> `self`를 사용한다.

```
self.add(a,b)
```

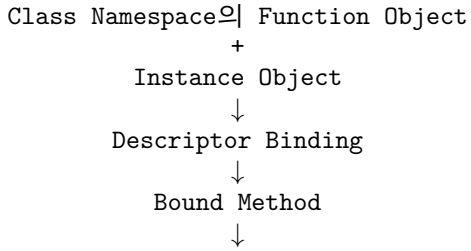
이것은 인스턴스의 속성 탐색과 메서드 바인딩을 이용하는 표현식이다.

> PyTorch의 `forward(self, x)`도 인스턴스 메서드다.

`self`는 모델 인스턴스를 참조하고, `x`는 입력 Tensor를 참조한다.

2.5.12 2.5에서 반드시 이해해야 할 결론

이번 학습의 핵심 구조는 다음과 같다.



호출
↓
Function Execution

즉, self가 자동으로 전달되는 것처럼 보이는 이유는 self라는 이름이 특별해서가 아니라, Python이 클래스의 함수 객체를 인스턴스에 바인딩한 메서드로 제공하기 때문이다.

이 개념은 함수 객체, 속성 탐색, 네임스페이스, 인수 전달이라는 이전 학습 내용이 하나로 연결되는 지점이기도 하다.

다음 2.6에서는 클래스 객체를 호출했을 때 인스턴스가 어떻게 생성되고 초기화되는지, self는 언제 전달되는지, __new__()와 __init__()의 역할이 왜 분리되어 있는지 살펴볼 것이다.

2.6 __new__와 __init__ - 인스턴스 생성과 초기화는 어떻게 다른가?

2.5에서는 클래스에 정의된 함수 객체가 인스턴스에 바인딩되면서 메서드가 호출되는 과정을 학습하였다.

특히 다음 두 호출이 일반적인 인스턴스 메서드에서 같은 효과를 가진다는 사실을 확인하였다.

```
p.introduce()

Person.introduce(p)
```

이번 2.6에서는 그보다 앞선 과정, 즉 인스턴스가 존재하기 전부터 인스턴스가 생성되고 초기화될 때까지 Python에서 어떤 일이 발생하는지 살펴볼 것이다.

출발점은 다음 코드다.

```
class Person:
    def __init__(self, name):
        self.name = name


p = Person("Kim")
```

우리는 흔히 __init__()을 생성자(Constructor)라고 부르지만,

엄밀하게는 __init__()이 인스턴스 자체를 생성하는 것은 아니다.

Python에서는 일반적으로 다음 두 과정을 구분한다.

`__new__()` : 인스턴스 생성

`__init__()` : 생성된 인스턴스 초기화

이번 학습의 핵심은 이 두 메서드가 어떤 순서로 호출되고, 각각 무엇을 반환하며, 그 과정에서 `self`가 어떻게 전달되는지를 이해하는 것이다.

2.6.1 개념 정의 - 생성과 초기화의 차이

먼저 객체의 생성(Creation)과 초기화(Initialization)를 구분하자.

인스턴스 생성

아직 존재하지 않는 인스턴스 객체를 만들어 반환하는 과정이다.

인스턴스 초기화

이미 생성된 인스턴스에 필요한 속성을 설정하거나 초기 상태를 구성하는 과정이다.

예를 들어 다음 코드를 보자.

```
class Person:
    def __init__(self, name):
        self.name = name
        self.age = 0
```

여기서 `__init__()`은 인스턴스에 `name`과 `age` 속성을 설정하고 있다.

하지만 `self.name`과 `self.age`를 설정하려면 그 속성들을 담을 인스턴스 객체가 먼저 존재해야 하지 않을까?

바로 이 질문이 `__new__()`와 `__init__()`을 구분하는 출발점이다.

구분	___new___()	___init___()
역할	인스턴스 생성	인스턴스 초기화
첫 번째 매개변수 관례	`cls`	`self`
첫 번째 인수	생성 대상 클래스	생성된 인스턴스
반환값	생성한 객체 또는 다른 객체	반드시 `None`
실행 순서	일반적으로 먼저	생성 이후
직접 재정의하는 빈도	상대적으로 드묾	매우 흔함

여기서 `cls`와 `self`는 모두 관례적인 매개변수 이름이다.

`self`가 예약어가 아니었던 것처럼 `cls`도 예약어는 아니다.

2.6.2 클래스 객체를 호출하면 무엇이 실행되는가?

다음 표현식을 살펴보자.

```
Person("Kim")
```

2.3에서는 이 표현식이 클래스 객체를 호출하는 것이라고 학습했다.

그렇다면 클래스 객체를 호출할 때 Python은 무엇을 실행할까?

일반적인 Python 클래스는 `type`이라는 메타클래스의 인스턴스다.

```
class Person:
    pass

print(type(Person))
```

실행 결과:

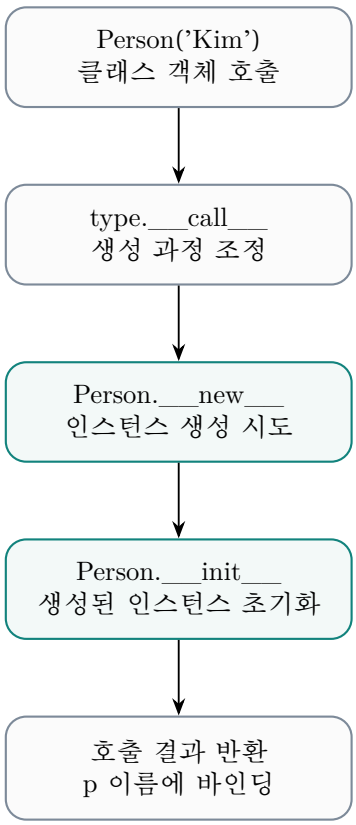
```
<class 'type'>
```

여기서 중요한 사실이 있다.

클래스 객체도 호출 가능한 객체이며, 그 호출 과정은 메타클래스의 호출 메커니즘에 의해 처리된다는 것이다.

일반적인 클래스에서는 `type.__call__()`이 인스턴스 생성 과정을 담당한다.

개념적인 실행 순서는 다음과 같다.



기본 메타클래스 type을 사용하는 일반적인 클래스의 인스턴스 생성 과정

단, 이 그림에는 한 가지 조건이 있다.

`__new__()`가 해당 클래스의 인스턴스 객체를 반환한 경우에만 `__init__()`이 이어서 호출된다.

다른 종류의 객체를 반환하면 `__init__()`이 호출되지 않을 수 있다.

이 예외는 뒤에서 실제 코드로 확인할 것이다.

`type.__call__()`을 개념적으로 표현하면

다음과 비슷하게 생각할 수 있다.

```
def conceptual_call(cls, *args, **kwargs):
    obj = cls.__new__(cls, *args, **kwargs)
```

```
    if isinstance(obj, cls):
        cls.__init__(obj, *args, **kwargs)

    return obj
```

이 코드는 실제 CPython 구현을 그대로 옮긴 것이 아니라, 일반적인 동작 순서를 이해하기 위한 수도 코드다.

실제 구현에서는 특수 메서드 탐색, 타입 검사, 초기화 결과 검증 등의 세부 처리가 추가된다.

또한 위 수도 코드는 `__init__()`의 반환값이 `None`이어야 한다는 검사도 생략하였다.

현재 단계에서 중요한 것은 다음 순서다.

$$\texttt{Class Call} \rightarrow \texttt{__new__} \rightarrow \texttt{__init__} \rightarrow \texttt{Return}$$

2.6.3 `__new__()` - 인스턴스 생성

이제 `__new__()`를 직접 정의해 보자.

```
class Person:
    def __new__(cls):
        print("__new__ 실행")

        instance = super().__new__(cls)

        return instance
```

이 코드에서 가장 중요한 부분은 다음 표현식이다.

```
instance = super().__new__(cls)
```

여기서는 상위 클래스에서 제공하는 인스턴스 생성 기능을 호출한다.

`Person`의 직접적인 상위 클래스는 기본적으로 `object`이므로,

이 예제에서는 `object.__new__()`를 통해 인스턴스를 생성하는 효과가 있다.

`super()`의 정확한 메서드 탐색 규칙은 2.9에서 다룰 것이다.

> `cls`는 무엇을 참조할까?

다음 코드로 확인해 보자.

```
class Person:
    def __new__(cls):
        print(cls)

        return super().__new__(cls)

p = Person()
```

실행 결과:

```
<class '__main__.Person'>
```

즉, cls는 생성 대상인 Person 클래스 객체를 참조한다.

2.5에서 self가 인스턴스 객체를 참조했던 것과 비교해 보자.

매개변수	참조하는 객체
cls	클래스 객체
self	인스턴스 객체

이 차이는 자연스러운 결과다.

__new__()이 호출되는 시점에는 새 인스턴스를 생성하려는 중이므로, 아직 그 인스턴스를 self로 전달할 수 없기 때문이다.

생성한 인스턴스를 확인해 보자.

```
class Person:
    def __new__(cls):
        instance = super().__new__(cls)

        print(type(instance))

        return instance

p = Person()
```

실행 결과:

```
<class '__main__.Person'>
```

`super().__new__(cls)`가 반환한 객체는 `Person` 클래스의 인스턴스다.

그리고 `__new__()`는 그 객체를 반환하고 있다.

여기서 중요한 사실은 `__new__()`가 반드시 새 인스턴스를 반환해야 하는 것은 아니라는 것이다.

다른 객체를 반환하는 것도 가능하다.

다만 그렇게 하면 이후 초기화 과정이 달라질 수 있다.

2.6.4 `__init__()` - 생성된 인스턴스 초기화

이번에는 `__init__()`을 살펴보자.

```
class Person:
    def __init__(self, name):
        print("__init__ 실행")

        self.name = name

p = Person("Kim")
```

실행 결과:

```
__init__ 실행
```

여기서 `self`는 이미 생성된 `Person` 인스턴스를 참조한다.

따라서 다음 코드를 실행할 수 있다.

```
self.name = name
```

2.4에서 배운 것처럼, 이 코드는 `self`가 참조하는 인스턴스에 `name` 속성을 설정하는 동작이다.

실제로 확인해 보자.

```
print(p.__dict__)
```

실행 결과:

```
{'name': 'Kim'}
```

`__init__()`은 객체를 반환하는가?

여기서 주의할 점이 있다.

`__init__()`은 생성된 인스턴스를 초기화하는 메서드이므로 반드시 `None`을 반환해야 한다.

다음 코드를 보자.

```
class Person:
    def __init__(self):
        return 10


p = Person()
```

실행하면 다음과 같은 오류가 발생한다.

```
TypeError: __init__() should return None, not 'int'
```

왜 그럴까?

`__init__()`은 인스턴스 생성 결과를 결정하는 메서드가 아니라, 이미 생성된 인스턴스를 초기화하는 메서드이기 때문이다.

일반적인 함수에서는 반환값을 자유롭게 지정할 수 있지만, Python의 특수 메서드에는 각각의 동작 규약이 존재한다.

`__init__()`도 그 규약을 따르는 것이다.

다음처럼 명시적으로 `None`을 반환하는 것은 가능하다.

```
class Person:
    def __init__(self):
        return None
```

하지만 보통 `return`을 작성하지 않는다.

Python 함수는 명시적인 반환문 없이 종료되면 자동으로 `None`을 반환하기 때문이다.

2.6.5 __new__()와 __init__()의 실행 순서 확인

이제 두 메서드를 함께 정의해 보자.

```
class Person:
    def __new__(cls, name):
        print("1. __new__ 실행")

        instance = super().__new__(cls)

        print("2. 인스턴스 생성 완료")

        return instance

    def __init__(self, name):
        print("3. __init__ 실행")

        self.name = name

        print("4. 인스턴스 초기화 완료")


p = Person("Kim")

print("5. p에 바인딩된 인스턴스:", p.name)
```

실행 결과:

```
1. __new__ 실행
2. 인스턴스 생성 완료
3. __init__ 실행
4. 인스턴스 초기화 완료
5. p에 바인딩된 인스턴스: Kim
```

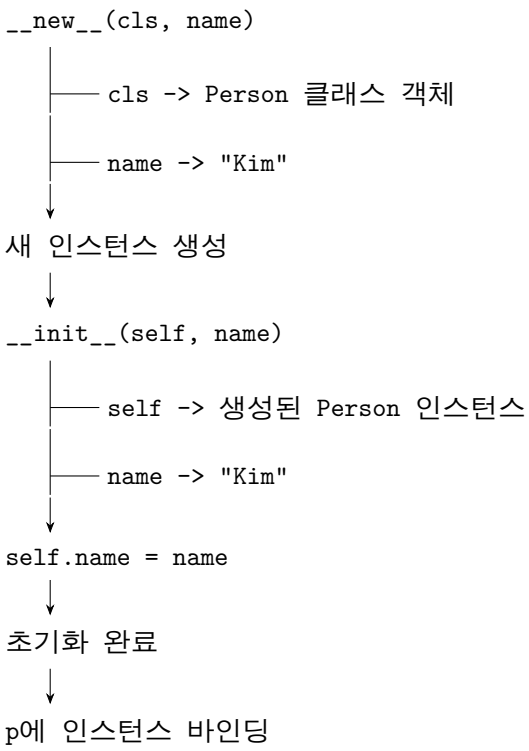
여기서 주목할 점은 __new__()와 __init__() 모두 "Kim"을 전달받았다는 사실이다.

```
def __new__(cls, name):

def __init__(self, name):
```

하지만 첫 번째 매개변수가 참조하는 객체는 서로 다르다.

```
Person("Kim")
    ↓
```



이 구조는 이전에 학습했던 함수의 인수 전달 및 이름 바인딩 규칙과 연결된다.

`cls`, `self`, `name`은 모두 함수 실행 과정에서 객체를 참조하도록 바인딩되는 매개변수 이름이다.

차이는 Python이 특수 메서드를 호출할 때 어떤 객체를 전달하도록 정해 놓았는가에 있다.

2.6.6 `__new__()`가 다른 객체를 반환하면 어떻게 되는가?

앞에서 `__new__()`는 반드시 새로운 인스턴스를 반환할 필요가 없다고 했다.

이번에는 그 의미를 실제 코드로 확인해 보자.

```
class Person:
    def __new__(cls):
        print("__new__ 실행")

        return "Not a Person"

    def __init__(self):
```

```
        print("__init__ 실행")

p = Person()

print(p)
print(type(p))
```

실행 결과:

```
__new__ 실행
Not a Person
<class 'str'>
```

여기서 중요한 사실이 있다.

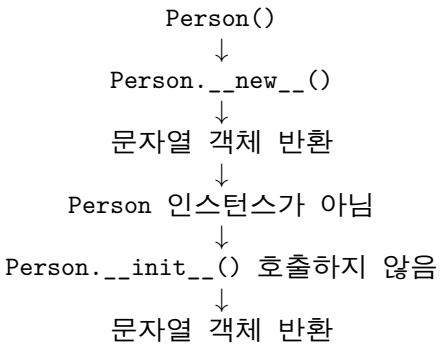
`__init__` 실행 이라는 메시지가 출력되지 않았다.

왜 그럴까?

`__new__()`가 Person 인스턴스가 아니라 문자열 객체를 반환했기 때문이다.

일반적인 `type.__call__()` 동작에서는 `__new__()`의 반환값이 생성 대상 클래스의 인스턴스가 아니면 `__init__()`을 호출하지 않는다.

따라서 이 예제에서는 다음과 같은 과정이 이루어진다.



이 예제를 통해 중요한 사실을 알 수 있다.

클래스 객체를 호출한다고 해서 반드시 그 클래스의 인스턴스가 반환되는 것은 아니다.

일반적으로는 인스턴스를 반환하도록 설계하지만, 특수한 클래스에서는 다른 동작을 구현할 수 있다.

이 점은 클래스 호출이 단순한 객체 복제 문법이 아니라, 정해진 호출 프로토콜을 통해 실행되는 과정이라는 사실을 보여준다.

2.6.7 `__new__()`를 직접 재정의하는 이유

그렇다면 왜 `__new__()`와 `__init__()`을 분리했을까?

일반적인 클래스에서는 `__init__()`만 정의해도 충분한 경우가 많다.

```
class Person:
    def __init__(self, name):
        self.name = name
```

그런데 다음과 같은 상황에서는 인스턴스 생성 과정 자체를 제어해야 할 수 있다.

1. 생성할 객체의 타입을 결정해야 하는 경우
2. 불변 객체의 값을 생성 시점에 설정해야 하는 경우
3. 이미 존재하는 객체를 반환해야 하는 경우
4. 인스턴스 생성을 제한하거나 관리해야 하는 경우

대표적인 예로 불변 객체를 생각해 볼 수 있다.

불변 객체를 상속하는 경우

다음 코드를 보자.

```
class PositiveInt(int):
    def __new__(cls, value):
        if value < 0:
            raise ValueError("양수 또는 0만 허용됩니다.")

    return super().__new__(cls, value)
```

이제 다음처럼 사용할 수 있다.

```
a = PositiveInt(10)

print(a)
print(type(a))
```

실행 결과:

```
10
<class '__main__.PositiveInt'>
```

반면 다음 코드는 오류를 발생시킨다.

```
b = PositiveInt(-5)
```

```
ValueError: 양수 또는 0만 허용합니다.
```

이 예제에서 PositiveInt는 int를 상속받아 새로운 정수형 객체를 정의했다.

int는 불변 객체이므로 정수값 자체는 생성 과정에서 결정된다.

따라서 이런 경우에는 __init__() 보다 __new__()를 재정의하는 것이 적절하다.

이처럼 __new__()는 인스턴스가 만들어지는 과정 자체를 제어해야 할 때 사용된다.

2.6.8 __new__()와 __init__()을 직접 호출하면 어떻게 되는가?

이번에는 두 메서드를 직접 호출해 보자.

먼저 다음 클래스를 정의하자.

```
class Person:
    def __init__(self, name):
        self.name = name
```

__new__() 직접 호출

```
p = Person.__new__(Person)
```

이 코드는 Person의 인스턴스를 생성한다.

하지만 __init__()은 호출하지 않았다.

따라서 다음 코드를 실행하면

```
print(p.__dict__)
```

실행 결과:

```
{}
```

인스턴스는 존재하지만 name 속성은 설정되지 않았다.

__init__() 직접 호출

이제 다음 코드를 실행해 보자.

```
Person.__init__(p, "Kim")
```

그다음 속성을 확인하면

```
print(p.__dict__)
```

실행 결과:

```
{'name': 'Kim'}
```

즉, 직접 호출한 __init__()이 기존 인스턴스를 초기화한 것이다.

이 과정을 정리하면 다음과 같다.

```
p = Person.__new__(Person)

Person.__init__(p, "Kim")
```

이는 일반적인 Person("Kim") 호출에서 일어나는 생성 및 초기화 과정을 이해하기 위한 실험이다.

다만 모든 클래스에 대해 이렇게 수동으로 생성과 초기화를 분리해도 안전한 것은 아니다.

일부 클래스는 생성 과정에서 특별한 불변 조건이나 내부 상태를 설정하기 때문에, 일반적으로는 클래스 객체를 직접 호출하는 방식을 사용해야 한다.

```
p = Person("Kim")
```

왜 이 실험이 중요할까?

이전에는 다음 코드만 봤다.

```
p = Person("Kim")
```

이제는 이 한 줄 안에서 개념적으로 두 가지 다른 동작이 이루어진다는 사실을 확인한 것이다.

$$\texttt{Instance Creation} \neq \texttt{Instance Initialization}$$

2.6.9 __init__()은 인스턴스마다 실행되는가?

다음 코드를 살펴보자.

```
class Person:
    def __init__(self, name):
        print("__init__ 실행", name)

        self.name = name

p1 = Person("Kim")
p2 = Person("Lee")
```

실행 결과:

```
__init__ 실행: Kim
__init__ 실행: Lee
```

일반적인 인스턴스 생성에서는 클래스를 호출할 때마다 새 인스턴스가 생성되고 __init__()이 실행된다.

따라서 p1과 p2는 각각 별도의 속성을 갖게 된다.

```
print(p1.__dict__)
print(p2.__dict__)
```

실행 결과:

```
{'name': 'Kim'}
{'name': 'Lee'}
```

여기서 2.3과 2.4에서 학습한 개념들이 연결된다.

- 클래스 객체는 하나다.

- 인스턴스 객체는 두 개다.
- 각 인스턴스는 독립적인 속성 저장 공간을 가질 수 있다.
- `__init__()`은 각 인스턴스를 초기화하는 과정에서 호출된다.

다만 앞에서 본 것처럼 `__new__()`가 다른 객체를 반환하거나 메타클래스의 호출 동작을 변경하면 이러한 일반적인 흐름이 달라질 수 있다.

2.6.10 PyTorch와의 연결 - `super().__init__()`은 왜 필요한가?

이제 PyTorch 모델 코드로 돌아가 보자.

```
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10,5)

    def forward(self, x):
        return self.linear(x)
```

여기서 가장 중요한 부분은 다음 코드다.

```
super().__init__()
```

왜 이 코드를 작성할까?

`MyModel`은 `nn.Module`을 상속받는 클래스다.

`nn.Module`은 하위 모듈과 파라미터를 등록하고 관리하기 위한 내부 상태를 가지고 있다.

따라서 `MyModel`의 초기화 과정에서는 먼저 `nn.Module`이 필요로 하는 내부 상태를 초기화해야 한다.

그 역할을 수행하는 것이 다음 코드다.

```
super().__init__()
```

이 예제에서는 `super()`가 MRO에 따라 `nn.Module`의 `__init__()`을 찾아 호출하게 된다.

이후 다음 코드가 실행된다.

```
self.linear = nn.Linear(10,5)
```

이때 `nn.Module`의 내부 상태가 초기화되어 있으므로, `nn.Module` 인스턴스를 하위 모듈로 등록할 수 있다.

초기화 순서가 바뀌면 어떻게 될까?

```
class WrongModel(nn.Module):
    def __init__(self):
        self.linear = nn.Linear(10,5)

        super().__init__()
```

이 코드는 일반적인 PyTorch `nn.Module`에서 오류를 발생시킨다.

대표적인 오류 메시지는 다음과 같다.

```
AttributeError: cannot assign module before Module.__init__() call
```

왜 이런 문제가 발생할까?

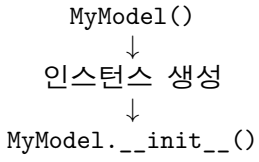
`nn.Module`이 하위 모듈을 등록하기 위한 내부 관리 구조를 아직 초기화하지 않았기 때문이다.

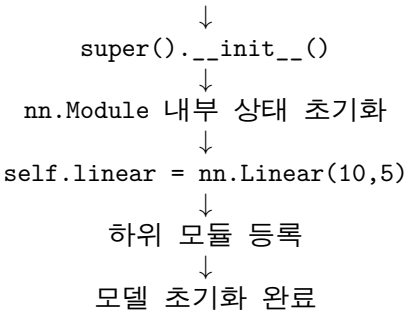
따라서 일반적인 PyTorch 모델에서는 다음 순서를 사용해야 한다.

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10,5)
```

이 과정을 그림으로 나타내면 다음과 같다.





여기서 주의할 점은 `super()`가 무조건 직접적인 부모 클래스만 호출하는 문법은 아니라는 사실이다.

정확히는 현재 클래스와 MRO를 기준으로 다음 메서드를 탐색하는 구조다.

이 부분은 2.9에서 자세히 다룰 것이다.

2.6.11 핵심 정리

이번 학습에서는 클래스 객체 호출과 인스턴스 생성 및 초기화의 관계를 살펴보았다.

> `__new__()`와 `__init__()`은 서로 다른 역할을 한다.

```
def __new__(cls, ...):
    ...
```

인스턴스 생성에 관여한다.

```
def __init__(self, ...):
    ...
```

이미 생성된 인스턴스를 초기화한다.

> 일반적인 클래스 호출은 `type.__call__()`을 통해 처리된다.

```
p = Person("Kim")
```

기본적인 생성 과정에서는 `__new__()`와 `__init__()`이 순서대로 호출된다.

> `cls`와 `self`는 서로 다른 객체를 참조한다.

- cls는 클래스 객체를 참조한다.

- self는 인스턴스 객체를 참조한다.

> __init__()은 반드시 None을 반환해야 한다.

```
def __init__(self):
    self.name = "Kim"
```

반환값을 명시하지 않는 것이 일반적이다.

> __new__()가 다른 타입의 객체를 반환하면
> __init__()이 호출되지 않을 수 있다.

```
def __new__(cls):
    return "Hello"
```

이 경우 일반적인 Person 인스턴스 초기화가 이루어지지 않는다.

> 인스턴스 생성과 초기화는 별도의 과정이다.

```
p = Person.__new__(Person)

Person.__init__(p, "Kim")
```

이 코드는 두 과정의 차이를 이해하기 위한 예제이다.

> __new__()는 생성 과정 자체를 제어할 때 사용한다.

특히 불변 객체의 서브클래싱이나 특수한 객체 생성 전략에서 활용할 수 있다.

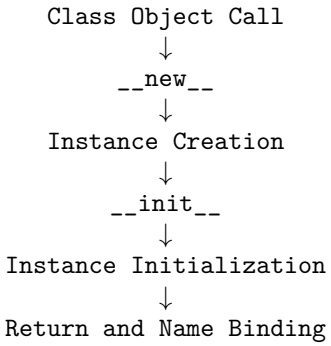
> PyTorch에서는 super().__init__()이 중요하다.

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
```

nn.Module의 내부 상태를 초기화해야 하위 모듈과 파라미터를 정상적으로 등록하고 관리할 수 있다.

2.6.12 2.6에서 반드시 이해해야 할 결론

이번 학습의 핵심은 다음 구조다.



단, 이것은 `__new__()`가 적절한 인스턴스를 반환하고 기본적인 클래스 호출 규칙을 따르는 일반적인 경우의 흐름이다.

이제 다음 코드가 단순히 Person이라는 클래스로 객체를 만든다는 의미를 넘어,

```
p = Person("Kim")
```

클래스 객체 호출 → 인스턴스 생성 → 인스턴스 초기화 → 반환값 바인딩

이라는 실행 과정으로 이해되어야 한다.

다음 2.7에서는 지금까지 학습한 인스턴스 네임스페이스와 메서드 바인딩을 기반으로 Python이 같은 이름의 속성을 어떤 순서로 탐색하는지 살펴볼 것이다.

특히 클래스 속성과 인스턴스 속성의 차이, Mutable 클래스 속성의 공유 문제, 속성 탐색 순서, 디스크립터의 우선순위를 연결해서 설명한다.

2.7 클래스 속성과 인스턴스 속성 - Python은 속성을 어떤 순서로 탐색하는가?

2.4에서는 인스턴스 속성이 일반적으로 `__dict__`에 저장된다는 사실을 학습했고, 2.5에서는 클래스에 정의된 함수 객체가 인스턴스를 통해 접근될 때 바운드 메서드로 변환되는 과정을 살펴보았다.

이번 2.7에서는 두 개념을 연결해 볼 것이다.

클래스와 인스턴스에 같은 이름의 속성이 존재하면 Python은 어떤 값을 선택할까?

그리고 더 근본적으로, 인스턴스에 존재하지 않는 메서드나 속성을 어떻게 찾아 사용할 수 있을까?

이번 학습의 핵심 질문은 다음과 같다.

1. 클래스 속성과 인스턴스 속성은 어디에 저장되는가?
2. 클래스 속성은 모든 인스턴스가 공유하는가?
3. 같은 이름의 속성이 존재하면 어떤 속성이 우선하는가?
4. p.attribute = value와 Person.attribute = value는 어떻게 다른가?
5. Mutable 클래스 속성에서 예상치 못한 공유 문제가 발생하는 이유는 무엇인가?
6. 디스크립터는 일반적인 속성 탐색 순서를 어떻게 변경하는가?
7. 이 구조가 PyTorch의 nn.Module과 어떻게 연결되는가?

2.7.1 개념 정의 - 클래스 속성과 인스턴스 속성

먼저 다음 코드를 살펴보자.

```
class Person:
    species = "Human"

    def __init__(self, name):
        self.name = name

p1 = Person("Kim")
p2 = Person("Lee")
```

여기에는 두 종류의 속성이 존재한다.

클래스 속성(Class Attribute)

```
species = "Human"
```

클래스 본문에서 정의된 속성이다.

Person 클래스 객체의 네임스페이스에 저장된다.

인스턴스 속성(Instance Attribute)

```
self.name = name
```

인스턴스에 설정된 속성이다.

일반적인 사용자 정의 객체에서는 인스턴스의 __dict__에 저장된다.

실제로 확인해 보자.

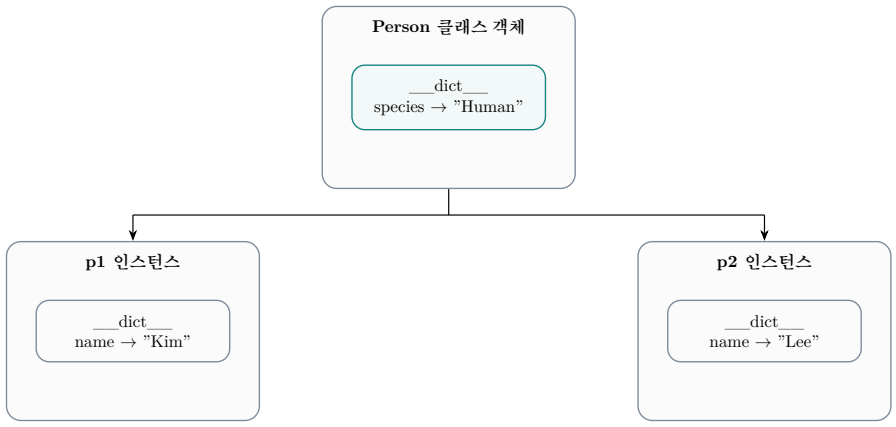
```
print(Person.__dict__["species"])

print(p1.__dict__)
print(p2.__dict__)
```

실행 결과:

```
Human
{'name': 'Kim'}
{'name': 'Lee'}
```

이 구조를 그림으로 나타내면 다음과 같다.



클래스 속성은 클래스 객체에, 인스턴스 속성은 각 인스턴스에 저장된다.

여기서 중요한 점은 속성이 인스턴스마다 복사되는 것이 아니라는 사실이다.

```
print(p1.species)
print(p2.species)
```

실행 결과:

```
Human
Human
```

p1과 p2 모두 species 속성에 접근할 수 있지만, 두 인스턴스의 __dict__에는 species가 존재하지 않는다.

Python이 인스턴스에서 속성을 찾을 때 클래스 객체도 탐색하기 때문에 가능한 일이다.

2.7.2 클래스 속성은 어떻게 공유되는가?

다음 코드를 보자.

```
class Person:
    species = "Human"

p1 = Person()
p2 = Person()
```

여기서 p1.species와 p2.species는 모두 클래스에 저장된 동일한 속성을 통해 값을 얻는다.

```
pritn(p1.species is p2.species)
```

실행 결과:

```
True
```

하지만 이것을 모든 인스턴스가 species라는 인스턴스 속성을 각각 가지고 있다고 이해하면 안 된다.

실제 저장 위치는 클래스 객체다.

```
print(p1.__dict__)
print(p2.__dict__)
```

실행 결과:

```
{}
{}
```

클래스 속성을 변경하면?

```
Person.species = "Homo sapiens"

print(p1.species)
print(p2.species)
```

실행 결과:

```
Homo sapiens
Homo sapiens
```

클래스 속성의 바인딩을 변경했기 때문에, 해당 클래스 속성을 참조하던 인스턴스에서 새로운 값이 관찰되는 것이다.

여기서 이전에 학습했던 이름과 객체의 바인딩 개념을 다시 적용할 수 있다.

```
Person.species = "Homo sapiens"
```

이 표현식은 기존 문자열 객체를 변경하는 것이 아니라, 클래스의 species 속성이 다른 문자열 객체를 참조하도록 재바인딩하는 동작이다.

그렇다면 인스턴스를 통해 변경하면 어떻게 될까?

2.7.3 인스턴스 속성이 클래스 속성을 가리는 현상 - Shadowing

다음 코드를 보자.

```
class Person:
    species = "Human"

p1 = Person()
p2 = Person()

p1.species = "Modified"

print(p1.species)
print(p2.species)
print(Person.species)
```

실행 결과:

```
Modified
Human
Human
```

왜 p1.species만 변경되었을까?

일반적인 속성 설정에서는 다음 코드가

```
p1.species = "Modified"
```

클래스의 species를 수정하지 않고 p1 인스턴스에 같은 이름의 속성을 생성하기 때문이다.

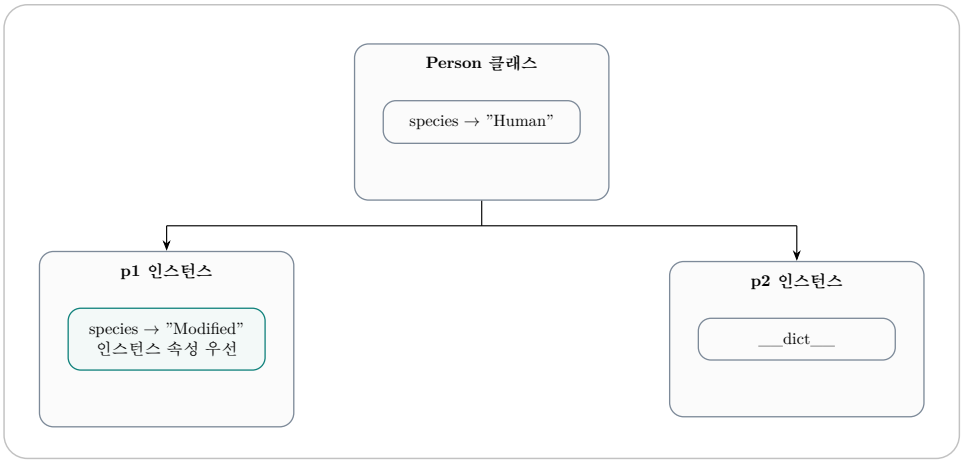
확인해 보자.

```
print(p1.__dict__)
print(p2.__dict__)
print(Person.__dict__["species"])
```

실행 결과:

```
{'species': 'Modified'}
{}
Human
```

이제 속성 저장 구조는 다음과 같다.



클래스 속성 참조

p1에는 이제 자체적인 species 속성이 있다.

일반적인 속성 탐색에서는 클래스 속성보다 인스턴스 속성이 우선된다.

이러한 현상을 **속성 가리기(Shadowing)**라고 한다.

인스턴스 속성을 삭제하면?

```
del p1.species

print(p1.species)
```

실행 결과:

```
Human
```

인스턴스에 저장된 species가 삭제되었으므로, 다시 클래스 속성이 발견되는 것이다.

여기서 한 가지를 정확하게 구분해야 한다.

클래스 속성이 사라졌다가 복구된 것이 아니다.

클래스 속성은 계속 존재했지만, 인스턴스에 같은 이름의 속성이 있을 때 탐색 결과에서 가려졌던 것뿐이다.

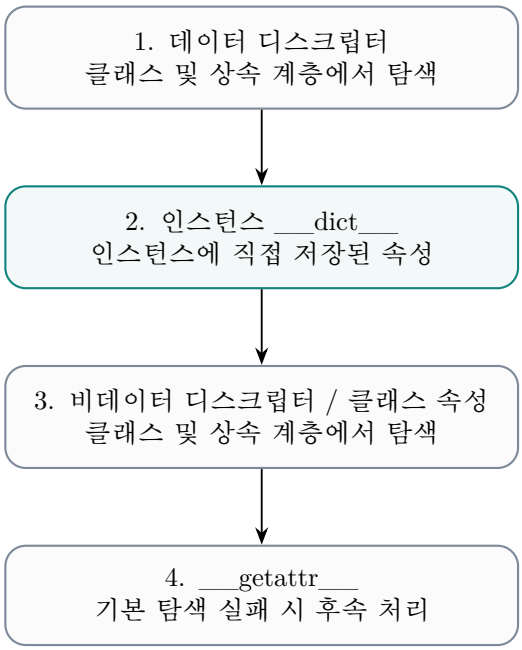
2.7.4 속성 탐색의 정확한 우선순위

지금까지는 인스턴스 속성이 클래스 속성보다 우선한다고 설명했다.

하지만 이 설명은 모든 상황에서 정확하지는 않다.

Python의 기본적인 인스턴스 속성 탐색에는 **디스크립터(Descriptor)**라는 중요한 예외가 있기 때문이다.

일반적인 사용자 정의 객체에서 object.__getattribute__()가 수행하는 속성 읽기를 단순화하면 다음 순서로 이해할 수 있다.



기본 인스턴스 속성 읽기의 개념적 우선순위

여기서 데이터 디스크립터와 비데이터 디스크립터의 차이를 알아야 한다.

구분	제공하는 특수 메서드	기본적인 탐색 우선순위
데이터 디스크립터	`__get__`과 `__set__` 또는 `__delete__`	인스턴스 딕셔너리보다 높음
비데이터 디스크립터	`__get__`만 제공	인스턴스 딕셔너리보다 낮음
일반 클래스 속성	디스크립터 프로토콜 없음	인스턴스 딕셔너리보다 낮음

정확히 말하면 데이터 디스크립터는 `__set__()` 또는 `__delete__()`를 정의한 디스크립터다.

`__get__()`도 제공하는 경우가 일반적이다.

이제 왜 이런 규칙이 필요한지 살펴보자.

2.7.5 비데이터 디스크립터 - 일반 인스턴스 메서드

2.5에서 일반적인 함수 객체는 `__get__()`을 제공한다고 학습했다.

따라서 클래스에 정의된 일반적인 함수 객체는 비데이터 디스크립터다.

```
class Person:
    def introduce(self):
        print("Hello")


p = Person()
```

다음 코드를 실행하면

```
p.introduce()
```

Python은 클래스에 저장된 함수 객체를 찾아 바운드 메서드로 제공한다.

그런데 인스턴스에 같은 이름의 속성을 직접 설정하면 어떻게 될까?

```
p.introduce = "Not a method"

print(p.introduce)
```

실행 결과:

```
Not a method
```

왜 함수가 아니라 문자열이 반환될까?

일반적인 함수 객체는 비데이터 디스크립터이므로, 인스턴스의 `__dict__`에 같은 이름이 존재하면 인스턴스 속성이 우선하기 때문이다.

```
print(p.__dict__)
```

실행 결과:

```
{'introduce': 'Not a method'}
```

이 상태에서 다음 코드를 실행하면

```
p.introduce()
```

오류가 발생한다.

```
TypeError: 'str' object is not callable
```

하지만 클래스의 함수 객체는 여전히 존재한다.

```
Person.introduce(p)
```

실행 결과:

```
Hello
```

이 예제는 두 가지 중요한 사실을 보여준다.

1. 인스턴스 메서드는 일반적으로 클래스에 저장된 함수 객체다.
2. 인스턴스에 같은 이름의 속성을 만들면 그 메서드를 가릴 수 있다.

따라서 메서드 호출 역시 속성 탐색의 결과라는 사실을 이해할 수 있다.

2.7.6 데이터 디스크립터 - property의 동작

이번에는 인스턴스 속성보다 우선하는 데이터 디스크립터를 살펴보자.

Python의 property는 대표적인 예시다.

```
class Person:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        print("getter 실행")
        return self._name
```

이제 다음 코드를 실행해 보자.

```
p = Person("Kim")

print(p.name)
```

실행 결과:

```
getter 실행
Kim
```

여기서 p.name은 단순한 인스턴스 딕셔너리 조회가 아니다.

Person 클래스의 name 속성에는 property 객체가 저장되어 있다.

```
print(type(Person.__dict__["name"]))
```

실행 결과:

```
<class 'property'>
```

그리고 property 객체는 디스크립터 프로토콜을 제공한다.

따라서 p.name에 접근하면 property의 __get__() 메커니즘을 통해 getter 함수가 실행되는 것이다.

> 인스턴스 딕셔너리에 같은 이름을 직접 넣으면?

다음 코드를 보자.

```
p.__dict__["name"] = "Lee"

print(p.__dict__)
print(p.name)
```

실행 결과:

```
{'_name': 'Kim', 'name': 'Lee'}
getter 실행
Kim
```

여기서 흥미로운 결과가 발생했다.

인스턴스 딕셔너리에는 분명히 "name": "Lee"가 존재한다.

그런데 p.name은 "Kim"을 반환한다.

왜 그럴까?

property는 데이터 디스크립터이므로 인스턴스의 __dict__보다 높은 우선순위를 갖기 때문이다.

따라서 p.name을 읽을 때는 인스턴스 딕셔너리의 "name"보다 클래스에 정의된 property가 우선된다.

이것이 단순히 인스턴스 속성이 클래스 속성보다 우선한다고만 설명하면 부족한 이유다.

setter를 정의하면?

```
class Person:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
        self._name = value
```

이제 다음 코드가 가능하다.

```
p = Person("Kim")

p.name = "Lee"

print(p.name)
print(p.__dict__)
```

실행 결과:

```
Lee
{'_name': 'Lee'}
```

여기서 p.name = "Lee"는 단순히 인스턴스 딕셔너리에 "name"을 저장하는 동작이 아니다.

property의 setter를 통해 _name 속성이 변경된다.

따라서 property를 이용하면 속성 접근 문법은 유지하면서 내부적으로 읽기와 쓰기 동작을 제어할 수 있다.

2.7.7 Mutable 클래스 속성의 공유 문제

이번에는 실제 Python 프로그래밍에서 자주 발생하는 문제를 살펴보자.

다음 코드를 보자.

```
class Person:
    hobbies = []


p1 = Person()
p2 = Person()
```

hobbies는 클래스 속성이다.

따라서 두 인스턴스가 같은 리스트 객체를 참조할 수 있다.

```
print(p1.hobbies is p2.hobbies)
```

실행 결과:

```
True
```

이제 다음 코드를 실행해 보자.

```
p1.hobbies.append("Reading")

print(p1.hobbies)
print(p2.hobbies)
```

실행 결과:

```
['Reading']
['Reading']
```

왜 p2.hobbies에도 "Reading"이 추가되었을까?

p1.hobbies와 p2.hobbies가 동일한 리스트 객체를 참조하기 때문이다.

여기서 중요한 것은 append()가 속성을 재바인딩하는 동작이 아니라, 기존 리스트 객체를 변경하는 동작이라는 사실이다.

```
p1.hobbies.append("Reading")
```

이 코드는 다음과 같은 과정을 거친다.

1. p1.hobbies를 통해 클래스 속성의 리스트 객체를 갖는다.
2. 해당 리스트 객체의 append() 메서드를 호출한다.
3. 기존 리스트 객체의 내용이 변경된다.
4. p2.hobbies도 같은 리스트 객체를 참조하므로 변경된 내용이 관찰된다.

> 반대로 재바인딩하면?

```
p1.hobbies = ["Music"]

print(p1.hobbies)
print(p2.hobbies)
```

실행 결과:

```
['Music']
['Reading']
```

이번에는 결과가 달라졌다.

왜냐하면 p1.hobbies = ["Music"]은 일반적인 속성 설정 규칙에 따라 p1 인스턴스에 새로운 hobbies 속성을 만들기 때문이다.

즉, 기존 클래스 속성의 리스트 객체를 변경한 것이 아니라 인스턴스 속성을 새로 설정한 것이다.

코드	동작	다른 인스턴스에 미치는 영향
p1.hobbies .append("Reading")	공유 리스트 객체 변경	같은 리스트를 참조하면 영향 있음
p1.hobbies = ["Music"]	인스턴스 속성 재바인딩	클래스 속성 자체에는 영향 없음
Person.hobbies = []	클래스 속성 재바인딩	클래스 속성을 참조하는 인스턴스에 영향

이 차이는 Python의 Mutable / Immutable, 객체 동일성, 이름 바인딩을 이해하는 데 매우 중요하다.

권장하는 구조

인스턴스마다 독립적인 리스트가 필요하다면 다음처럼 작성해야 한다.

```
class Person:
    def __init__(self):
        self.hobbies = []
```

이렇게 하면 인스턴스가 초기화될 때마다 새로운 리스트 객체가 생성된다.

```
p1 = Person()
p2 = Person()

print(p1.hobbies is p2.hobbies)
```

실행 결과:

```
False
```

따라서 인스턴스마다 독립적으로 변경해야 하는 Mutable 데이터는 일반적으로 인스턴스 속성으로 관리하는 것이 적절하다.

2.7.8 클래스 속성을 변경하는 것과 인스턴스 속성을 변경하는 것

이번에는 속성 변경을 조금 더 체계적으로 비교해 보자.

```
class Counter:
    count = 0
```

여기서 count는 클래스 속성이다.

다음 코드를 실행해 보자.

```
c1 = Counter()
c2 = Counter()

c1.count += 1
print(c1.count)
print(c2.count)
print(Counter.count)
```

실행 결과:

```
1
0
0
```

왜 이런 결과가 발생할까?

`c1.count += 1`은 일반적인 정수 속성에 대해 개념적으로 다음과 같은 읽기와 쓰기 과정을 수행한다.

```
value = c1.count
value = value + 1
c1.count = value
```

처음에는 `c1`에 `count` 인스턴스 속성이 없으므로 클래스 속성 `Counter.count`에서 0을 읽는다.

그다음 1을 더한 결과를 `c1.count`에 설정한다.

따라서 `c1`에 새로운 인스턴스 속성이 생성되는 것이다.

```
print(c1.__dict__)
```

실행 결과:

```
{'count': 1}
```

반면 다음 코드는 클래스 속성을 변경한다.

```
Counter.count += 1
```

이제 다음과 같이 출력될 것이다.

```
print(c1.count)
print(c2.count)
print(Counter.count)
```

실행 결과:

```
1
1
1
```

`c1.count`는 자신의 인스턴스 속성을 가지고 있으므로 여전히 1이고,

c2.count는 클래스 속성을 읽으므로 변경된 1을 반환한다.

이 예제에서는 우연히 같은 값이 출력되지만, 실제 참조 위치는 서로 다르다.

> 클래스 전체에서 공유할 상태라면?

다음과 같이 명시적으로 클래스 속성을 수정할 수 있다.

```
class Counter:
    count = 0

    def __init__(self):
        Counter.count += 1

c1 = Counter()
c2 = Counter()
c3 = Counter()

print(Counter.count)
```

실행 결과:

```
3
```

이 예제에서는 인스턴스가 생성될 때마다 클래스 속성 count를 증가시킨다.

다만 상속 관계까지 고려하면 Counter.count처럼 특정 클래스 이름을 직접 사용하는 방식과 type(self).count처럼 실제 인스턴스의 타입을 사용하는 방식은 서로 다른 동작을 만들 수 있다.

이 차이는 2.8에서 연결해 살펴볼 것이다.

2.7.9 __getattribute__(), __getattr__(), __setattr__()의 역할

지금까지 속성 탐색과 설정 규칙을 살펴보았다.

그렇다면 Python은 이러한 동작을 어떤 메서드로 처리할까?

일반적인 사용자 정의 객체에서 중요한 특수 메서드는 다음과 같다.

특수 메서드	역할
`__getattribute__()`	속성을 읽을 때 호출되는 기본 접근 메커니즘
`__getattr__()`	일반적인 속성 탐색이 실패한 경우 호출되는 후속 메커니즘
`__setattr__()`	속성을 설정할 때 호출
`__delattr__()`	속성을 삭제할 때 호출

`__getattr__()` 예제

```
class Person:
    def __getattr__(self, name):
        return f"{name} 속성은 존재하지 않습니다."


p = Person()

print(p.age)
```

실행 결과:

```
age 속성은 존재하지 않습니다.
```

일반적인 속성 탐색에서는 age를 찾지 못했다.

그런데 `__getattr__()`이 정의되어 있으므로 Python은 후속 처리로 해당 메서드를 호출한다.

따라서 `AttributeError` 대신 문자열을 반환한 것이다.

`__setattr__()` 예제

```
class Person:
    def __setattr__(self, name, value):
        print(f"{name} 속성 설정")

        object.__setattr__(self, name, value)


p = Person()
p.name = "Kim"
```

실행 결과:

name 속성 설정

여기서 object.__setattr__()을 호출한 이유는 기본적인 속성 설정 동작을 수행하기 위해서다.

만약 다음처럼 작성하면 어떻게 될까?

```
class Person:
    def __setattr__(self, name, value):
        self.name = value
```

이 코드는 위험하다.

self.name = value 자체가 다시 __setattr__()을 호출하므로 무한 재귀가 발생할 수 있기 때문이다.

따라서 __setattr__()을 재정의할 때는 기본 속성 설정을 수행하기 위해

object.__setattr__() 등을 적절히 사용해야 한다.

2.7.10 PyTorch와의 연결 - nn.Module은 왜 속성 접근을 재정의하는가?

이제 PyTorch의 다음 코드를 다시 살펴보자.

```
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)
```

여기서 다음 코드는 일반적인 속성 설정처럼 보인다.

```
self.linear = nn.Linear(10,5)
```

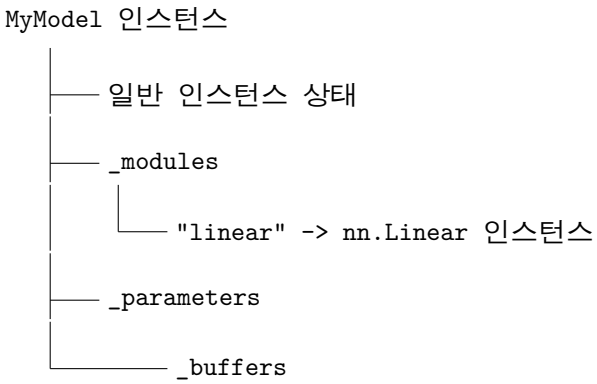
하지만 nn.Module은 속성 설정 메커니즘을 확장해 하위 모듈을 등록하도록 구성되어 있다.

일반적인 사용자 정의 클래스에서는 다음처럼 동작할 수 있다.

```
self.__dict__["linear"] = linear_object
```

반면 nn.Module은 __setattr__()을 재정의하여, nn.Module 인스턴스가 설정될 때 내부 등록 구조인 _modules를 사용해 관리한다.

개념적으로 다음과 같은 구조다.



여기서 중요한 사실은 Python의 일반적인 속성 접근 문법을 그대로 사용하면서 내부 저장 규칙은 클래스가 재정의할 수 있다는 것이다.

따라서 다음 코드도 가능하다.

```
model = MyModel()

print(model.linear)
```

model.linear에 접근하면 PyTorch의 속성 탐색 메커니즘이 등록된 하위 모듈을 찾아 반환한다.

또한 등록된 모듈과 파라미터는 다음 기능들과 연관된다.

```
model.parameters()
model.state_dict()
model.to("cpu")
```

이 기능들이 가능한 이유는 단순히 self.linear라는 이름을 사용했기 때문이 아니다.

nn.Module이 제공하는 속성 등록 및 관리 규약을 따르기 때문이다.

직접 확인해 보자.

```
model = MyModel()

print(model.__dict__.keys())
print(model._modules)
```

`model.__dict__`에는 PyTorch의 내부 관리 구조를 포함한 여러 속성이 저장되어 있고,

`_modules`에는 등록된 하위 모듈이 저장되어 있다.

이처럼 PyTorch는 Python의 객체 모델을 활용해 복잡한 모델 구조를 관리하고 있다.

2.7.11 핵심 정리

이번 학습에서 반드시 이해해야 할 내용을 정리해 보자.

> 클래스 속성과 인스턴스 속성은 서로 다른 네임스페이스에 저장된다.

```
class Person:
    species = "Human"

    def __init__(self, name):
        self.name = name
```

`species`는 클래스 속성이고 `name`은 인스턴스 속성이다.

> 클래스 속성은 인스턴스마다 자동으로 복사되지 않는다.

인스턴스는 속성 탐색을 통해 클래스 속성에 접근할 수 있다.

> 일반적인 속성에서는 인스턴스 속성이 클래스 속성을 가릴 수 있다.

```
p.species = "Modified"
```

이 코드는 보통 클래스 속성을 변경하는 것이 아니라 인스턴스 속성을 생성한다.

> 데이터 디스크립터는
>
> 인스턴스 딕셔너리보다 높은 탐색 우선순위를 가진다.

대표적인 예는 `property`이다.

일반적인 인스턴스 메서드는 비데이터 디스크립터다.

따라서 인스턴스에 같은 이름의 속성이 존재하면 메서드가 가려질 수 있다.

Mutable 클래스 속성은 여러 인스턴스가 공유할 수 있다.

```
class Person:
    hobbies = []
```

이런 구조에서는 한 인스턴스의 리스트 변경이 다른 인스턴스에도 관찰될 수 있다.

속성의 변경과 재바인딩은 서로 다르다.

```
p.hobbies.append("Reading")
p.hobbies = ["Music"]
```

첫 번째는 기존 리스트 객체의 변경이고, 두 번째는 속성의 재바인딩이다.

Python은 특수 메서드를 통해

속성 접근 및 설정 동작을 확장할 수 있다.

```
__getattribute__()
__getattr__()
__setattr__()
__delattr__()
```

이 메서드들은 객체의 속성 관리 방식을 사용자 정의할 수 있게 한다.

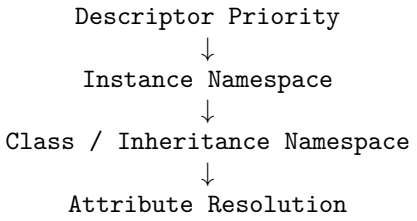
PyTorch의 nn.Module은 이러한 메커니즘을 활용한다.

`self.linear = nn.Linear(...)`는 단순한 속성 설정을 넘어 하위 모듈 등록으로 연결된다.

2.7.12 2.7에서 반드시 이해해야 할 결론

이번 학습의 핵심은 다음과 같다.

Attribute Access
↓



이것은 기본적인 인스턴스 속성 탐색 과정을 단순화한 구조다.

특히 다음 두 코드를 구분할 수 있어야 한다.

```
p.attribute = value
```

```
Person.attribute = value
```

첫 번째는 일반적으로 인스턴스 속성을 설정하고, 두 번째는 클래스 객체의 속성을 설정한다.

또한 다음 두 코드의 차이도 이해해야 한다.

```
p.items.append(value)
```

```
p.items = new_list
```

첫 번째는 참조한 객체를 변경하는 동작이고, 두 번째는 속성에 새로운 객체를 연결하는 동작이다.

결국 Python의 속성 접근은 단순히 객체 안의 값을 읽는 것이 아니라, 객체의 네임스페이스와 클래스 계층, 디스크립터 프로토콜을 결합해 이름을 해석하는 과정이다.

다음 2.8에서는 이 속성 탐색 규칙을 클래스 사이의 관계로 확장해 볼 것이다.

특히 자식 클래스가 부모 클래스의 메서드를 사용할 수 있는 이유, 같은 이름의 메서드를 재정의하면 어떤 일이 발생하는지, 상속이 단순한 코드 재사용을 넘어 어떤 의미를 가지는지 설명할 것이다.

2.8 상속과 메서드 오버라이딩 - 자식 클래스는 부모 클래스의 기능을 어떻게 사용하는가?

2.7에서는 Python이 인스턴스 속성을 읽을 때 인스턴스 네임스페이스뿐 아니라 클래스 네임스페이스도 탐색한다는 사실을 학습했다.

이번 2.8에서는 이 속성 탐색 범위를 하나의 클래스에서 여러 클래스 사이의 관계로 확장한다.

다음 코드를 살펴보자.

```
class Animal:
    def speak(self):
        print("Animal sound")


class Dog(Animal):
    pass

dog = Dog()
dog.speak()
```

실행 결과:

```
Animal sound
```

여기서 중요한 질문이 생긴다.

Dog 클래스에는 speak()가 정의되어 있지 않다.

그런데 왜 dog.speak()를 호출할 수 있을까?

이번 학습에서는 이 질문을 출발점으로 다음 개념들을 연결할 것이다.

- 상속(Inheritance)과 클래스 간 관계
- 부모 클래스와 자식 클래스의 객체 모델
- 상속된 속성과 메서드의 탐색
- 메서드 오버라이딩(Overriding)
- self와 실제 인스턴스 타입
- __init__()의 상속과 초기화
- 다형성(Polymorphism)
- PyTorch nn.Module의 상속 구조

2.8.1 개념 정의 - 상속이란 무엇인가?

상속(Inheritance)은 기존 클래스의 속성과 동작을 활용하여 새로운 클래스를 정의하는 메커니즘이다.

다음 코드를 보자.

```
class Animal:
    def speak(self):
        print("Animal sound")

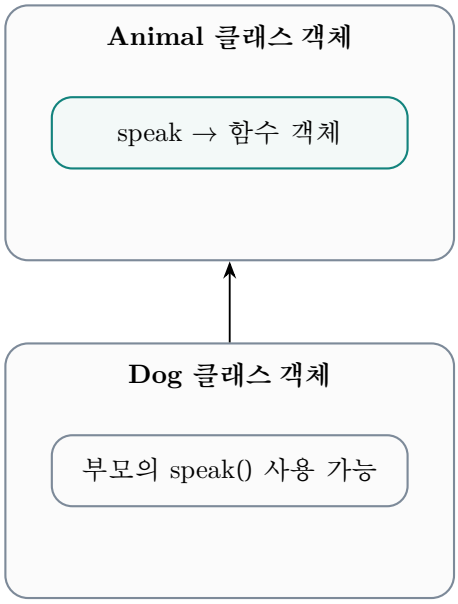

class Dog(Animal):
    pass
```

여기서 `Animal`은 부모 클래스(Base Class), `Dog`는 자식 클래스(Derived Class)이다.

```
class Dog(Animal):
```

이 구문은 `Dog` 클래스가 `Animal`을 상속하도록 정의한다는 의미다.

상속 관계를 그림으로 표현하면 다음과 같다.



상속은 클래스 객체 사이에 속성 탐색 관계를 형성한다.

여기서 가장 먼저 이해해야 할 점이 있다.

상속은 부모 클래스의 모든 속성과 메서드를 자식 클래스의 네임스페이스에 복사하는 과정이 아니다.

자식 클래스는 부모 클래스를 참조하는 상속 관계를 가지고, 속성을 탐색할 때 그 관계를 이용한다.

이 사실은 이후 메서드 오버라이딩과 `super()`를 이해하는 핵심이 된다.

2.8.2 상속하면 실제로 무엇이 생성되는가?

2.2에서는 `class`문을 실행하면 클래스 객체가 생성된다고 학습했다.

상속을 사용해도 마찬가지다.

```
class Animal:
    pass

class Dog(Animal):
    pass
```

이 코드를 실행하면 두 개의 클래스 객체가 생성된다.

- `Animal` 클래스 객체
- `Dog` 클래스 객체

`Dog`가 `Animal`을 상속했다고 해서 두 이름이 같은 객체를 참조하는 것은 아니다.

```
print(Dog is Animal)
```

실행 결과:

```
False
```

두 클래스는 별개의 객체다.

그렇다면 상속 관계는 어디에 기록될까?

Python 클래스 객체에는 직접적인 부모 클래스 정보를 확인할 수 있는

__bases__ 속성이 있다.

```
print(Dog.__bases__)
```

실행 결과:

```
(<class '__main__.Animal'>,)
```

즉, Dog 클래스 객체는 자신의 직접적인 부모 클래스가 Animal이라는 정보를 가지고 있다.

반대로 다음 코드도 확인해 보자.

```
print(Animal.__bases__)
```

실행 결과:

```
(<class 'object'>,)
```

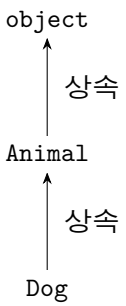
Python 3의 일반적인 사용자 정의 클래스는 별도의 부모 클래스를 명시하지 않으면 기본적으로 object를 상속한다.

따라서 다음 두 클래스 정의는 기본적인 상속 관계에서 동일하다.

```
class Animal:
    pass


class Animal(object):
    pass
```

전체 관계는 다음과 같다.



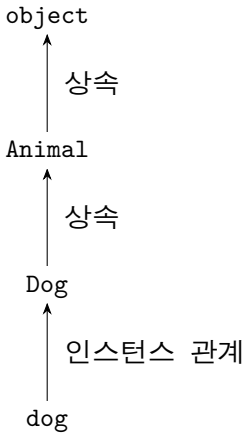
이 그림의 화살표는 클래스 사이의 상속 관계를 나타낸다.

2.3에서 학습했던 클래스와 인스턴스의 타입 관계와는 구분해야 한다.

상속 관계와 인스턴스 관계 비교

```
dog = Dog()
```

이때 두 종류의 관계가 동시에 존재한다.



따라서 다음 결과가 성립한다.

```
print(type(dog) is Dog)

print(isinstance(dog, Dog))
print(isinstance(dog, Animal))
print(isinstance(dog, object))
```

실행 결과:

```
True
True
True
True
```

dog의 실제 타입은 Dog이지만, 상속 관계 때문에 Animal과 object의 인스턴스로도 인정되는 것이다.

2.8.3 부모 클래스의 메서드는 자식 클래스에 복사되는가?

이번 학습에서 중요한 내용 중 하나다.

다음 코드를 보자.

```
class Animal:
    def speak(self):
        print("Animal sound")


class Dog(Animal):
    pass
```

이제 클래스 네임스페이스를 확인해 보자.

```
print("speak" in Animal.__dict__)
print("speak" in Dog.__dict__)
```

실행 결과:

```
True
False
```

Animal의 네임스페이스에는 speak 함수 객체가 존재한다.

하지만 Dog의 네임스페이스에는 존재하지 않는다.

그런데도 다음 코드는 실행된다.

```
dog = Dog()

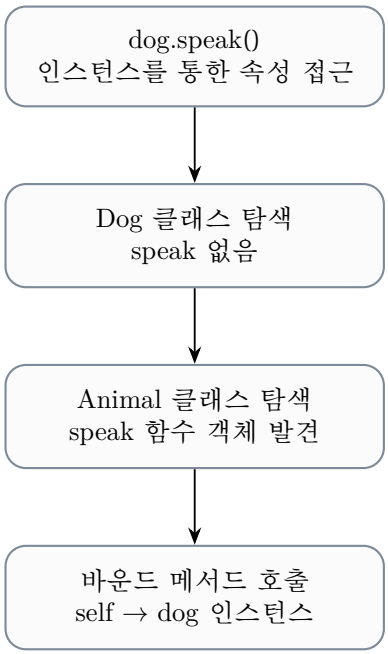
dog.speak()
```

실행 결과:

```
Animal sound
```

왜 그럴까?

Python이 Dog에서 speak을 찾지 못하면 상속 관계에 따라 Animal에서도 탐색하기 때문이다.



일반적인 상속 메서드 탐색 과정을 단순화한 그림

여기서 2.5의 바운드 메서드 개념이 연결된다.

`Animal`에서 발견한 함수 객체를 `dog` 인스턴스에 바인딩하는 것이다.

실제로 확인해 보자.

```
method = dog.speak

print(method.__self__ is dog)
print(method.__func__ is Animal.__dict__["speak"])
```

실행 결과:

```
True
True
```

즉, 부모 클래스에 정의된 함수 객체가 자식 클래스 인스턴스에 바인딩될 수 있다.

이것이 상속 메서드 호출의 핵심 원리다.

2.8.4 부모 메서드의 self는 어떤 인스턴스를 참조하는가?

다음 코드를 살펴보자.

```
class Animal:
    def identify(self):
        print(type(self))


class Dog(Animal):
    pass


dog = Dog()
dog.identify()
```

실행 결과:

```
<class '__main__.Dog'>
```

여기서 주목해야 할 점이 있다.

identify()는 Animal 클래스에 정의되어 있다.

그런데 self의 타입은 Animal이 아니라 Dog다.

왜 그럴까?

self는 메서드가 정의된 클래스를 의미하는 것이 아니라, 메서드 호출에 바인딩된 실제 인스턴스를 참조하기 때문이다.

다음 코드를 보자.

```
animal = Animal()
dog = Dog()

animal.identify()
dog.identify()
```

실행 결과:

```
<class '__main__.Animal'>
<class '__main__.Dog'>
```

두 호출은 같은 함수 객체를 사용하지만, 전달되는 인스턴스가 다르다.

개념적으로 다음과 같다.

```
Animal.identify(animal)

Animal.identify(dog)
```

따라서 함수 본문은 같더라도 `self`가 참조하는 객체에 따라 동작 결과가 달라질 수 있다.

> 부모 메서드에서 자식 인스턴스의 속성에 접근할 수도 있다

```
class Animal:
    def introduce(self):
        print(self.name)


class Dog(Animal):
    def __init__(self, name):
        self.name = name


dog = Dog("Buddy")
dog.introduce()
```

실행 결과:

```
Buddy
```

`introduce()`는 `Animal`에 정의되어 있지만, `self.name`은 `dog` 인스턴스의 속성을 읽는다.

즉, 메서드가 정의된 위치와 메서드가 사용하는 인스턴스의 상태는 구분되는 개념이다.

2.8.5 메서드 오버라이딩 - 자식 클래스에서 메서드를 재정의하기

이제 오버라이딩(Overriding)을 살펴보자.

오버라이딩은 자식 클래스가 부모 클래스에 존재하는 메서드와 같은 이름의 메서드를 정의하여, 자식 클래스에서의 동작을 변경하는 것이다.

다음 코드를 보자.

```
class Animal:
    def speak(self):
        print("Animal sound")


class Dog(Animal):
    def speak(self):
        print("Woof!")
```

이제 인스턴스를 생성해 보자.

```
animal = Animal()
dog = Dog()

animal.speak()
dog.speak()
```

실행 결과:

```
Animal sound
Woof!
```

여기서 Dog는 부모 클래스의 speak()을 변경한 것일까?

정확히는 아니다.

Animal 클래스의 speak 함수 객체는 그대로 존재한다.

Dog 클래스의 네임스페이스에 새로운 speak 함수 객체가 정의된 것이다.

확인해 보자.

```
print(Animal.__dict__["speak"])
print(Dog.__dict__["speak"])

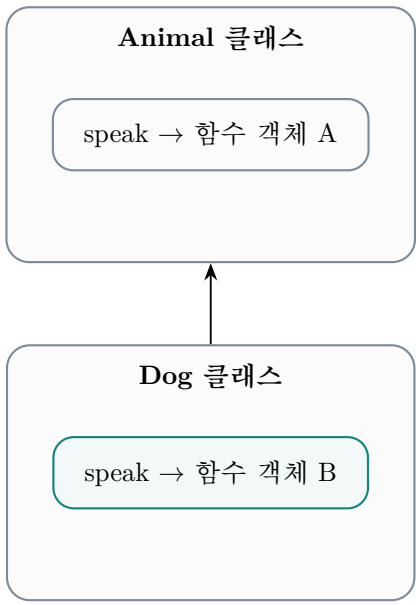
print(
    Animal.__dict__["speak"] is Dog.__dict__["speak"]
)
```

실행 결과의 마지막 줄:

```
False
```

두 함수 객체는 서로 다르다.

따라서 구조는 다음과 같다.



오버라이딩은 부모의 함수 객체를 수정하지 않고 자식 클래스에 같은 이름의 속성을 정의한다.

따라서 `dog.speak()`에서는 `Dog`클래스의 `speak`가 먼저 발견된다.

부모 클래스의 `speak`까지 탐색할 필요가 없는 것이다.

> 오버라이딩은 이름 탐색의 결과다

이전에 인스턴스 속성이 클래스 속성을 가리는 Shadowing을 학습했다.

메서드 오버라이딩도 넓은 의미에서는 비슷한 원리다.

- 인스턴스 속성이 클래스 속성을 가릴 수 있다.
- 자식 클래스의 속성이 부모 클래스의 속성을 가릴 수 있다.

다만 오버라이딩은 주로 상속 관계에서 메서드를 재정의하는 것을 의미한다.

2.8.6 부모 메서드와 자식 메서드를 직접 비교하기

다음 코드를 보자.

```
class Animal:
    def speak(self):
        print("Animal sound")


class Dog(Animal):
    def speak(self):
        print("Woof!")


dog = Dog()
```

일반적으로 다음 코드를 실행하면

```
dog.speak()
```

`Dog.speak()`가 호출된다.

그런데 부모 클래스에 정의된 메서드를 직접 호출할 수도 있다.

```
Animal.speak(dog)
```

실행 결과:

```
Animal sound
```

왜 가능할까?

2.5에서 배운 것처럼 클래스에서 함수 객체에 직접 접근한 뒤 인스턴스를 명시적으로 전달할 수 있기 때문이다.

```
Animal.speak(dog)
```

이 표현식은 `Animal` 클래스에 정의된 함수 객체를 가져와 `dog` 인스턴스를 첫 번째 인수로 전달한다.

따라서 자식 클래스에서 오버라이딩했더라도 부모 클래스의 함수 객체 자체는 여전히 호출할 수 있다.

하지만 실무에서 부모 메서드를 호출할 때는 일반적으로 `super()`를 사용한다.

```
class Dog(Animal):
    def speak(self):
```

```
        super().speak()
        print("Woof!")
```

super()는 단순히 부모 클래스 이름을 대체하는 문법이 아니다.

MRO(Method Resolution Order)를 기반으로 다음 메서드를 찾는 메커니즘이다.

이 부분은 2.9에서 자세히 다룰 것이다.

현재는 오버라이딩이 부모 메서드를 삭제하거나 수정하는 것이 아니라는 점을 기억하면 충분하다.

2.8.7 __init__()도 상속되고 오버라이딩되는가?

2.6에서는 __init__()이 생성된 인스턴스를 초기화하는 특수 메서드라고 학습했다.

그렇다면 __init__()도 상속될까?

그렇다.

__init__() 역시 클래스에 정의된 메서드이므로 상속 관계의 영향을 받는다.

부모의 __init__()을 그대로 사용하는 경우

```
class Animal:
    def __init__(self, name):
        self.name = name


class Dog(Animal):
    pass


dog = Dog("Buddy")

print(dog.name)
```

실행 결과:

```
Buddy
```

Dog에는 __init__()이 정의되어 있지 않다.

하지만 인스턴스를 생성할 때 상속 관계를 통해 Animal.__init__()이 사용된다.

따라서 dog 인스턴스에 name 속성이 설정되는 것이다.

자식이 __init__()을 오버라이딩하는 경우

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed
```

이제 Dog에는 자체적인 __init__()이 있다.

```
dog = Dog("Buddy", "Beagle")
```

실행 결과:

```
{'name': 'Buddy', 'breed': 'Beagle'}
```

여기서는 자식 클래스가 직접 name과 breed를 초기화했다.

그런데 다음 코드에서는 문제가 발생할 수 있다.

```
class Animal:
    def __init__(self, name):
        self.name = name
        self.alive = True


class Dog(Animal):
    def __init__(self, name, breed):
        self.breed = breed

dog = Dog("Buddy", "Beagle")

print(dog.alive)
```

실행 결과:

```
AttributeError: 'Dog' object has no attribute 'alive'
```

왜 이런 문제가 발생할까?

`Dog.__init__()`이 `Animal.__init__()`을 오버라이딩했고, 부모의 초기화 메서드를 호출하지 않았기 때문이다.

자식이 `__init__()`을 정의하면 부모의 `__init__()`이 자동으로 추가 실행되는 것은 아니다.

이 점은 반드시 기억해야 한다.

부모 초기화를 활용하는 경우

```
class Animal:
    def __init__(self, name):
        self.name = name
        self.alive = True


class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

이제 다음 코드가 정상적으로 실행된다.

```
dog = Dog("Buddy", "Beagle")

print(dog.__dict__)
```

실행 결과:

```
{
    'name': 'Buddy',
    'alive': True,
    'breed': 'Beagle'
}
```

`super().__init__(name)`을 통해 상속 계층의 다음 초기화 메서드를 호출했기 때문이다.

이 구조는 PyTorch의 `super().__init__()`을 이해하는 데 직접적으로 연결된다.

2.8.8 다형성 - 같은 메서드 호출이 다른 동작을 수행하는 이유

이제 상속과 오버라이딩을 결합하면 중요한 객체지향 개념이 등장한다.

바로 **다형성(Polymorphism)**이다.

다형성은 같은 인터페이스를 통해 서로 다른 객체가 각자의 동작을 수행할 수 있다는 개념이다.

다음 코드를 보자.

```
class Animal:
    def speak(self):
        print("Animal sound")


class Dog(Animal):
    def speak(self):
        print("Woof!")


class Cat(Animal):
    def speak(self):
        print("Meow!")
```

이제 서로 다른 클래스의 인스턴스를 생성해 보자.

```
animals = [
    Animal(),
    Dog(),
    Cat()
]
```

다음 반복문을 실행하면 어떻게 될까?

```
for animal in animals:
    animal.speak()
```

실행 결과:

```
Animal sound
Woof!
Meow!
```

반복문에서는 항상 같은 표현식을 사용했다.

```
animal.speak()
```

하지만 실제 호출되는 메서드는 인스턴스의 타입에 따라 달라진다.

왜 그럴까?

각 인스턴스가 속한 클래스에서 speak 속성을 탐색하고, 해당 함수 객체를 인스턴스에 바인딩하기 때문이다.

인스턴스	실제 타입	선택되는 메서드
Animal()	Animal	Animal.speak
Dog()	Dog	Dog.speak
Cat()	Cat	Cat.speak

여기서 중요한 사실이 있다.

반복문은 Dog인지 Cat인지 직접 검사하지 않는다.

```
if isinstance(animal, Dog):
    ...
elif isinstance(animal, Cat):
    ...
```

이런 조건문 없이도 객체가 제공하는 speak()을 호출하면 각자의 동작이 실행된다.

이것이 다형성의 핵심적인 활용 방식이다.

> 다형성이 반드시 상속을 요구하는 것은 아니다

Python에서는 다음과 같은 코드도 가능하다.

```
class Robot:
    def speak(self):
        print("Beep!")


class Person:
    def speak(self):
        print("Hello!")
```

두 클래스 사이에는 직접적인 상속 관계가 없다.

하지만 다음 함수는 두 객체를 모두 처리할 수 있다.

```
def make_speak(obj):
    obj.speak()


make_speak(Robot())
make_speak(Person())
```

실행 결과:

```
Beep!
Hello!
```

Python에서는 이런 방식을 흔히 덕 타이핑(Duck Typing)이라고 부른다.

객체의 구체적인 타입보다 필요한 동작을 제공하는가에 집중하는 방식이다.

따라서 다형성은 상속과 오버라이딩으로 구현할 수도 있지만, 반드시 상속을 통해서만 구현되는 것은 아니다.

2.8.9 상속은 언제 사용하는 것이 적절한가?

여기서 상속의 설계 목적을 생각해 보자.

상속은 단순히 중복 코드를 줄이기 위한 문법만은 아니다.

기존 클래스가 제공하는 인터페이스와 동작을 유지하거나 확장하면서, 새로운 타입을 정의하는 수단이다.

예를 들어 다음 관계는 자연스럽다.

```
class Animal:
    pass


class Dog(Animal):
    pass
```

Dog는 Animal의 한 종류라고 볼 수 있기 때문이다.

이를 흔히 IS-A 관계라고 설명한다.

반면 다음과 같은 관계는 상속보다 객체를 속성으로 보유하는 구성이 자연스러울 수 있다.

```
class Engine:
    pass


class Car:
    def __init__(self):
        self.engine = Engine()
```

자동차는 엔진의 한 종류가 아니다.

자동차가 엔진을 가지고 있는 관계이다.

이런 구성을 **합성(Composition)**이라고 한다.

구분	상속	합성
관계	IS-A	HAS-A
예시	Dog(Animal)	Car가 Engine을 보유
구현	부모 클래스를 지정	다른 객체를 속성으로 보유
주된 목적	인터페이스와 동작의 확장·특수화	여러 객체의 기능 조합

실제 프로그램 설계에서는 상속이 항상 합성보다 좋은 것도 아니고, 합성이 항상 상속보다 좋은 것도 아니다.

중요한 것은 두 클래스 사이에 어떤 관계를 표현하려는지를 먼저 결정하는 것이다.

이 구분은 PyTorch에서 특히 중요하다.

2.8.10 PyTorch와의 연결 - nn.Module 상속과 계층의 합성

이;제 다음 모델 코드를 살펴보자.

```
import torch
import torch.nn as nn


class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10, 5)
```

```
    def forward(self, x):
        return self.linear(x)
```

이 코드에는 상속과 합성이 동시에 사용되고 있다.

상속 관계

```
class MyModel(nn.Module):
```

MyModel은 nn.Module을 상속한다.

따라서 MyModel은 nn.Module의 하위 클래스다.

```
print(issubclass(MyModel, nn.Module))
```

실행 결과:

```
True
```

그리고 생성된 모델 인스턴스 역시 nn.Module의 인스턴스로 인정된다.

```
model = MyModel()

print(isinstance(model, nn.Module))
```

실행 결과:

```
True
```

MyModel은 nn.Module의 기능을 활용할 수 있다.

예를 들어 다음과 같은 메서드들이 있다.

```
model.parameters()
model.state_dict()
model.train()
model.eval()
model.to("cpu")
```

이러한 기능들은 MyModel에 직접 정의하지 않았어도 상속을 통해 사용할 수 있다.

합성 관계

다음 코드를 보자.

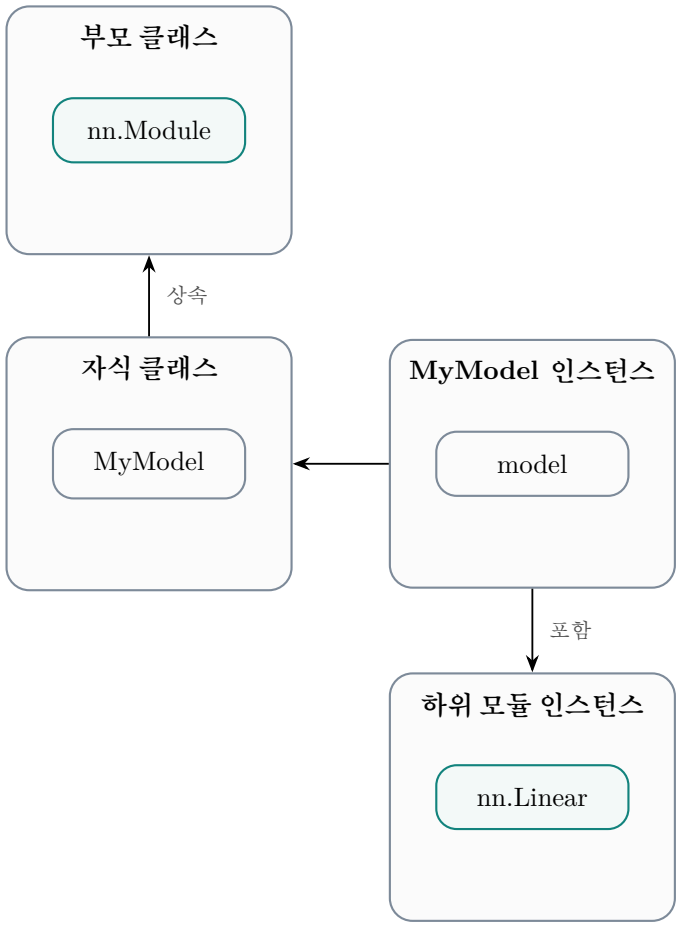
```
self.linear = nn.Linear(10, 5)
```

여기서 `nn.Linear` 인스턴스를 생성해 모델의 하위 모듈로 등록하고 있다.

즉, `MyModel`이 `nn.Linear`를 상속하는 것이 아니다.

`MyModel` 인스턴스가 `nn.Linear` 인스턴스를 포함하는 구조다.

이를 그림으로 표현하면 다음과 같다.



Mymodel은 nn.Module을 상속하고, 생성된 모델 인스턴스는 nn.Linear 인스턴스를 하위 모듈로 포함한다.

forward()는 오버라이딩인가?

```
def forward(self, x):
    return self.linear(x)
```

그렇다.

MyModel은 nn.Module이 제공하는 forward() 인터페이스를 오버라이딩한다.

nn.Module의 기본 forward()는 사용자가 모델에 맞는 계산을 정의하도록 마련된 메서드다.

따라서 MyModel은 자신의 연산을 forward()에 정의한다.

```
x = torch.randn(2, 10)

output = model(x)
```

이때 model(x)는 nn.Module의 호출 메커니즘을 사용하고, 그 과정에서 실제 모델 인스턴스에 정의된 MyModel.forward()가 호출된다.

이것은 상속된 호출 메커니즘과 오버라이딩된 메서드가 결합되는 사례다.

다만 model(x)와 model.forward(x)는 내부 처리 경로가 완전히 같지는 않다.

model(x)는 PyTorch의 Hook 등 추가적인 처리를 포함할 수 있기 때문이다.

2.8.11 핵심 정리

이번 학습의 핵심을 정리해 보자.

상속은 클래스 객체 사이에 관계를 형성한다.

```
class Dog(Animal):
    pass
```

Dog는 Animal을 상속하는 별도의 클래스 객체다.

부모 클래스의 메서드는 자식 클래스에 복사되지 않는다.

자식 클래스는 상속 관계를 따라 부모 클래스의 속성과 메서드를 탐색할 수 있다.

> 상속된 메서드의 self는 실제 호출 대상 인스턴스를 참조한다.

```
dog.speak()
```

부모 클래스에 정의된 메서드라도 self는 dog를 참조한다.

> 오버라이딩은
>
> 자식 클래스에 같은 이름의 메서드를 새로 정의하는 것이다.

```
class Dog(Animal):
    def speak(self):
        print("Woof!")
```

부모의 함수 객체를 수정하는 것은 아니다.

> __init__()도 상속과 오버라이딩의 영향을 받는다.

자식 클래스에서 __init__()을 재정의하면 부모 초기화가 자동으로 추가 실행되는 것은 아니다.

> super()는 상속 계층의 다른 구현을 활용할 때 사용한다.

```
super().__init__()
```

정확한 탐색 방식은 MRO에 따라 결정된다.

> 다형성은 동일한 인터페이스로
>
> 서로 다른 객체의 동작을 호출할 수 있게 한다.

```
animal.speak()
```

실제 객체 타입에 따라 선택되는 메서드가 달라질 수 있다.

> 상속과 합성은 서로 다른 객체 설계 방식이다.

- 상속 : IS-A 관계

- 합성 : HAS-A 관계

> PyTorch는 상속과 합성을 함께 사용한다.

```
class MyModel(nn.Module):
```

nn.Module을 상속하고,

```
self.linear = nn.Linear(10, 5)
```

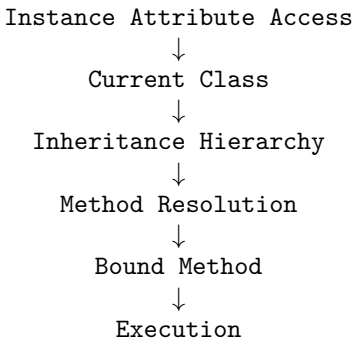
하위 모듈 인스턴스를 포함하며,

```
def forward(self, x):
```

forward()를 오버라이딩한다.

2.8.12 2.8에서 반드시 이해해야 할 결론

이번 학습의 핵심 구조는 다음과 같다.



이것은 일반적인 인스턴스 메서드 탐색을 단순화한 흐름이다.

실제 속성 탐색에는 2.7에서 학습한 디스크립터 우선순위도 적용된다.

가장 중요한 사실은 다음 두 가지다.

첫째, 상속은 부모 클래스의 메서드를 복사하는 것이 아니라,

부모 클래스의 속성을 탐색할 수 있게 관계를 만드는 것이다.

둘째, 오버라이딩은 부모 클래스의 메서드를 수정하는 것이 아니라,

자식 클래스의 네임스페이스에 같은 이름의 새로운 메서드를 정의하는 것이다.

이 두 원리를 이해하면 PyTorch에서 `nn.Module`을 상속하고 `forward()`를 오버라이딩하는 구조를 자연스럽게 해석할 수 있다.

다음 2.9에서는 지금까지 설명한 상속 계층의 메서드 탐색을 더 엄밀하게 다룰 것이다.

특히 다중 상속에서 메서드 탐색 순서가 어떻게 결정되는지,

`super()`가 왜 단순히 부모 클래스를 호출하는 문법이 아닌지,

그리고 `super().__init__()`이 어떤 규칙으로 다음 메서드를 선택하는지 살펴볼 것이다.

2.9 super()와 MRO - Python은 다중 상속에서 메서드를 어떤 순서로 탐색하는가?

2.8에서는 상속이 부모 클래스의 메서드를 자식 클래스에 복사하는 것이 아니라, 클래스 사이에 속성 탐색 관계를 형성하는 것이라고 학습했다.

이번에는 그 관계를 조금 더 엄밀하게 살펴볼 것이다.

특히 다음 코드를 이해하는 것이 목표다.

```
class Dog(Animal):
    def __init__(self, name):
        super().__init__(name)
```

우리는 흔히 `super().__init__()`을 부모 클래스의 `__init__()`을 호출하는 문법이라고 설명한다.

단일 상속에서는 대체로 그렇게 이해해도 결과가 맞다.

하지만 Python의 실제 동작 원리는 조금 다르다.

`super()`는 부모 클래스를 직접 지정하는 것이 아니라, MRO를 기준으로 현재 클래스 다음에 위치한 클래스부터 속성을 탐색하는 메커니즘이다.

이 차이는 다중 상속에서 매우 중요해진다.

2.9.1 MRO란 무엇인가?

MRO는 `Method Resolution Order`의 약자다.

Python이 클래스의 속성이나 메서드를 탐색할 때 사용하는 클래스들의 선형적인 탐색 순서를 의미한다.

다음 코드를 보자.

```
class Animal:
    def speak(self):
        print("Animal")


class Dog(Animal):
    pass
```

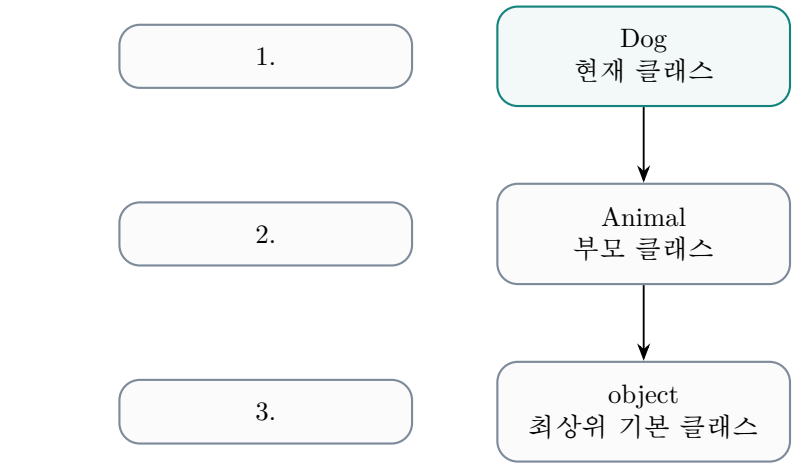
`Dog`의 MRO를 확인해 보자.

```
print(Dog.__mro__)
```

실행 결과:

```
(<class '__main__.Dog'>,
 <class '__main__.Animal'>,
 <class 'object'>)
```

이 결과는 Python이 `Dog` 인스턴스에서 클래스 속성을 탐색할 때 다음 순서를 사용한다는 의미다.



Dog.__mro__에 저장된 클래스 탐색 순서

MRO는 다음처럼도 확인할 수 있다.

```
pritn(Dog.mro())
```

두 표현식의 차이는 반환 형식이다.

표현식	반환값
`Dog.__mro__`	클래스들의 튜플
`Dog.mro()`	클래스들의 리스트

일반적인 클래스에서는 두 표현식으로 동일한 탐색 순서를 확인할 수 있다.

중요한 구분: MRO는 클래스 탐색 순서다

2.7에서 배운 인스턴스 속성 탐색에는 디스크립터와 인스턴스 `__dict__`의 우선순위가 있었다.

MRO는 그 규칙을 대체하는 것이 아니다.

MRO는 클래스 계층에서 속성을 탐색해야 할 때 어떤 클래스를 먼저 살펴볼지 결정한다.

따라서 다음 두 개념을 구분해야 한다.

- 속성 탐색 규칙: 데이터 디스크립터, 인스턴스 딕셔너리 등의 우선순위

- MRO : 클래스와 상속 계층의 탐색 순서

이 둘이 결합되어 실제 속성 접근 결과가 결정되는 것이다.

2.9.2 super()는 무엇을 반환하는가?

다음 코드를 보자.

```
class Animal:
    def speak(self):
        print("Animal")


class Dog(Animal):
    def speak(self):
        super().speak()
        print("Dog")

dog = Dog()
dog.speak()
```

실행 결과:

```
Animal
Dog
```

이때 다음 표현식에 주목한다.

```
super()
```

이 표현식은 Animal 클래스 객체를 반환하는 것이 아니다.

MRO를 기준으로 속성을 탐색하도록 구성된 super 프록시 객체를 반환한다.

이를 직접 확인해 보자.

```
class Dog(Animal):
    def inspect(self):
        proxy = super()

        print(type(proxy))
        print(proxy)
```

```
dog = Dog()
dog.inspect()
```

출력 예시:

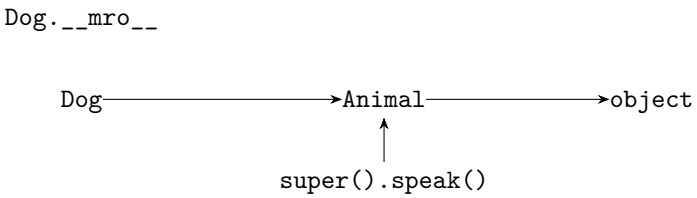
```
<class 'super'>
<super: <class 'Dog'>, <Dog object>>
```

여기서 super() 객체는 두 가지 중요한 정보를 이용한다.

1. 탐색을 시작할 기준 클래스
2. 속성 탐색에 사용할 인스턴스 또는 클래스

이 예제에서는 기준 클래스가 Dog, 연결된 객체가 dog이다.

따라서 super().speak()은 Dog의 MRO에서 Dog다음 위치부터 speak을 찾는 것이다.



단일 상속에서는 그 다음 클래스가 보통 직접적인 부모 클래스이기 때문에 super()가 부모 메서드를 호출하는 것처럼 보이는 것이다.

2.9.3 super()의 두 인수 형태

Python에서는 다음 표현식을 사용할 수 있다.

```
super(Dog, dog)
```

일반적인 인스턴스 메서드 안에서 사용하는

```
super()
```

는 적절한 문맥에서 다음과 같은 효과를 가진다.

```
super(Dog, self)
```

여기서 첫 번째 인수는 MRO에서 탐색을 건너뛸 기준 클래스다.

두 번째 인수는 그 탐색에 사용할 인스턴스다.

직접 확인해 보자.

```
class Animal:
    def speak(self):
        print("Animal")


class Dog(Animal):
    def speak(self):
        super(Dog, self).speak()
        print("Dog")

dog = Dog()
dog.speak()
```

실행 결과:

```
Animal
Dog
```

두 인수를 명시적으로 전달해도 같은 결과가 나온다.

하지만 일반적인 인스턴스 메서드에서는 다음 표현식을 권장한다.

```
super().speak()
```

클래스 이름을 직접 작성하지 않아도 되므로 클래스 이름 변경이나 상속 구조 변경에 더 유연하기 때문이다.

> super()가 무조건 상위 클래스를 의미할까?

아니다.

정확하게는 지정한 기준 클래스 다음부터 MRO를 따라 탐색하는 객체라고 이해해야 한다.

이 사실은 다중 상속에서 분명해진다.

2.9.4 다중 상속 - 부모 클래스가 여러 개인 경우

Python은 다중 상속(Multiple Inheritance)을 지원한다.

즉, 하나의 클래스가 여러 클래스를 동시에 상속할 수 있다.

다음 코드를 보자.

```
class A:
    def speak(self):
        print("A")


class B:
    def speak(self):
        print("B")


class C(A, B):
    pass
```

C는 A와 B를 모두 상속한다.

```
print(C.__bases__)
```

실행 결과:

```
(<class '__main__.A'>, <class '__main__.B'>)
```

그렇다면 C의 인스턴스에서 speak()을 호출하면 어떤 메서드가 실행될까?

```
c = C()
c.speak()
```

실행 결과:

```
A
```

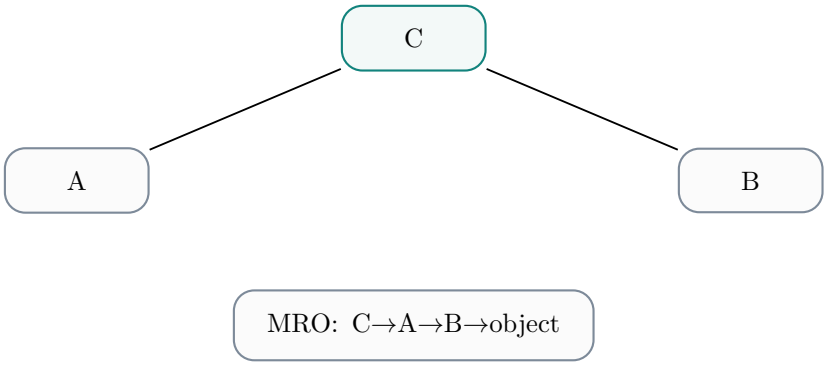
이유는 MRO를 보면 알 수 있다.

```
print([cls.__name__ for cls in C.__mro__])
```

실행 결과:

```
['C', 'A', 'B', 'object']
```

즉 클래스 탐색 순서는 다음과 같다.



상속 구조는 분기하지만, MRO는 하나의 선형 순서를 제공한다.

이 예제에서는 A와 B가 모두 speak()을 정의했다.

Python은 MRO에서 먼저 등장하는 A의 메서드를 선택한다.

여기서 중요한 점은 다음이다.

MRO는 단순히 부모 클래스의 목록이 아니라, 상속 계층 전체를 하나의 일관된 탐색 순서로 정렬한 결과다.

2.9.5 다이아몬드 상속 - 왜 MRO가 필요한가?

이제 조금 더 복잡한 구조를 살펴본다.

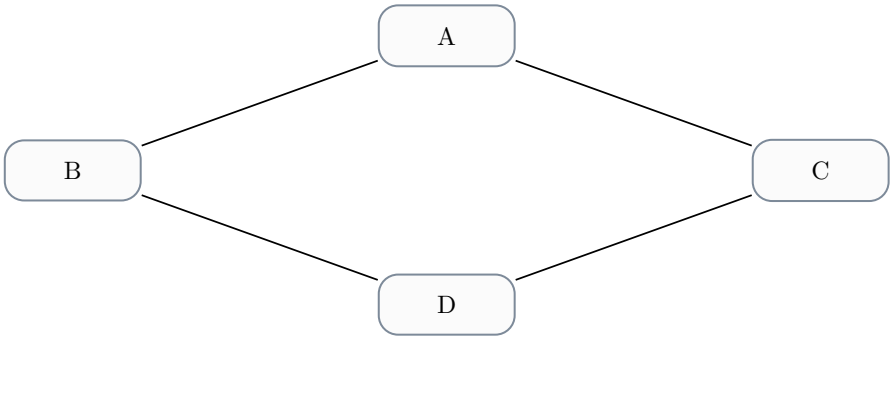
```
class A:
    def method(self):
        print("A")


class B(A):
    pass


class C(A):
    pass
```

```
class D(B, C):
    pass
```

이 구조를 그림으로 나타내면 다음과 같다.



다이아몬드 상속: B와 C가 같은 조상 A를 상속하고, D가 두 클래스를 상속한다.

이 구조를 다이아몬드 상속(Diamond Inheritance)이라고 한다.

여기서 문제가 생길 수 있다.

D는 B를 통해 A에 도달할 수도 있고, C를 통해 A에 도달할 수도 있다.

그렇다면 A는 두 번 탐색해야 할까?

Python은 MRO를 사용해 이 문제를 해결한다.

```
print([cls.__name__ for cls in D.__mro__])
```

실행 결과:

```
['D', 'B', 'C', 'A', 'object']
```

여기서 A는 한 번만 등장한다.

이처럼 MRO는 다중 상속에서 클래스 탐색 순서를 일관되게 유지하는 역할을 한다.

2.9.6 C3 선형화 - MRO는 어떤 규칙으로 결정되는가?

Python 3에서는 일반적인 클래스의 MRO를 결정할 때 C3 선형화(C3 Linearization) 알고리즘을 사용한다.

C3는 단순히 깊이 우선 탐색(DFS)이나 너비 우선 탐색(BFS)을 수행하는 알고리즘이 아니다.

다중 상속에서 여러 제약을 만족하도록 클래스들을 선형 순서로 배치하는 알고리즘이다.

핵심 원칙은 다음과 같다.

1. 자식 클래스는 부모 클래스보다 먼저 탐색된다.
2. 클래스 선언에서 지정한 직접 부모 클래스들의 상대적 순서를 존중한다.
3. 부모 클래스들이 이미 가진 MRO의 상대적 순서를 유지한다.
4. 이 조건들을 만족하는 일관된 순서가 존재하지 않으면 클래스 생성을 거부한다.

이를 수식으로 나타내면 다음과 같다.

$$L[C] = [C] + \text{merge}\left(L[B_1], L[B_2], \ldots, L[B_n], [B_1, B_2, \ldots, B_n]\right)$$

여기서

$L[C]$는 클래스 C의 MRO

$B_1, \ldots, B_n$은 직접 부모 클래스

merge는 C3의 병합 연산

이다.

C3 병합 규칙

병합 과정에서는 각 리스트의 첫 번째 원소를 후보로 살펴본다.

그 후보가 다른 리스트의 첫 번째가 아닌 나머지 부분에 등장하지 않으면 결과에 추가할 수 있다.

후보를 선택했다면 각 리스트에서 해당 원소를 제거하고 같은 과정을 반복한다.

이 규칙이 중요한 이유는 기존 부모 클래스들의 상대적 탐색 순서를 함부로 뒤집지 않기 때문이다.

다이아몬드 구조에 적용하기

앞의 예제를 다시 보자.

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass
```

각 클래스의 MRO는 다음과 같다.

```
A: A → object

B: B → A → object

C: C → A → object

D: D → B → C → A → object
```

D의 MRO를 구성할 때는 다음 리스트들을 병합해야 한다.

$$\text{merge}\left([B, A, object], [C, A, object], [B, C]\right)$$

첫 번째로 B를 선택할 수 있다.

B는 다른 리스트들의 나머지 부분에 등장하지 않기 때문이다.

그다음 C를 선택하고, 이어서 A, 마지막으로 object를 선택한다.

따라서 최종 결과는 다음과 같다.

$$\boxed{L[D] = [D, B, C, A, object]}$$

여기서 중요한 사실은 C3가 클래스 선언의 순서만 단순히 이어 붙이는 것이 아니라, 상속 계층 전체의 순서 제약을 만족하도록 병합한다는 것이다.

일관된 순서를 만들 수 없는 경우

```
class A:
    pass

class B:
    pass

class X(A, B):
    pass

class Y(B, A):
    pass

class Z(X, Y):
    pass
```

이 코드는 Z를 정의할 때 TypeError가 발생한다.

왜냐하면 X는 A가 B보다 먼저 와야 한다고 요구하고,

Y는 B가 A보다 먼저 와야 한다고 요구하기 때문이다.

두 제약을 동시에 만족하는 MRO는 존재하지 않는다.

따라서 Python은 임의로 하나를 선택하는 대신 클래스 정의 자체를 거부한다.

이것이 C3 선형화의 일관성 조건이다.

2.9.7 super()는 다중 상속에서 어떻게 동작하는가?

이제 이번 학습에서 가장 중요한 예제로 들어가 보자.

```
class A:
    def process(self):
        print("A")


class B(A):
    def process(self):
        print("B 시작")
        super().process()
```

```
        print("B 종료")


class C(A):
    def process(self):
        print("C 시작")
        super().process()
        print("C 종료")


class D(B, C):
    def process(self):
        print("D 시작")
        super().process()
        print("D 종료")
```

이제 다음 코드를 실행해 보자.

```
d = D()
d.process()
```

실행 결과:

```
D 시작
B 시작
C 시작
A
C 종료
B 종료
D 종료
```

왜 이런 순서가 나올까?

먼저 MRO를 확인하자.

```
print([cls.__name__ for cls in D.__mro__])
```

실행 결과:

```
['D', 'B', 'C', 'A', 'object']
```

여기서 중요한 부분은 `B.process()` 안에 있는 다음 코드다.

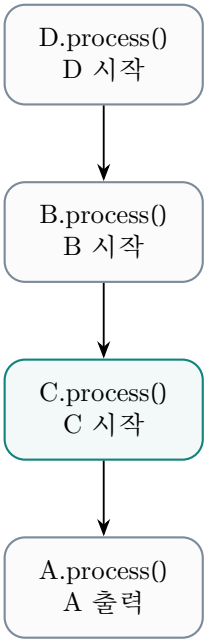
```
super().process()
```

일반적으로 B의 부모는 A이므로 A.process()를 호출할 것이라고 생각하기 쉽다.

하지만 실제로는 C.process()가 호출된다.

왜 그럴까?

super()가 B의 직접 부모 클래스를 호출하는 것이 아니라, 실제 d 인스턴스의 MRO에서 B 다음에 있는 클래스를 탐색하기 때문이다.



각 super().process()는 동일한 인스턴스 d의 MRO에서 다음 구현을 찾아 호출한다.

호출 순서를 조금 더 엄밀하게 표현하면 다음과 같다.

실행 위치	super()의 기준 클래스	다음에 발견되는 메서드
D.process()	D	B.process()
B.process()	B	C.process()
C.process()	C	A.process()

이 표에서 특히 두 번째 행이 중요하다.

B의 직접 부모는 A이지만, D 인스턴스의 MRO에서는 B 다음에 C가 위치한다.

따라서 B.process() 안의 super()가 C.process()를 호출하는 것이다.

그렇다면 B() 인스턴스에서는?

```
b = B()
b.process()
```

실행 결과:

```
B 시작
A
B 종료
```

왜 C를 거치지 않을까?

b의 타입은 B이고, MRO는 다음과 같기 때문이다.

```
B → A → object
```

즉, 같은 B.process() 함수 객체라도 어떤 인스턴스의 MRO를 기준으로 실행되는지에 따라 super()의 다음 탐색 대상이 달라질 수 있다.

이것이 super()를 단순히 부모 메서드 호출이라고만 이해하면 안 되는 가장 중요한 이유다.

2.9.8 협력적 다중 상속 - Cooperative Multiple inheritance

앞의 예제에서는 각 클래스가 다음 코드를 사용했다.

```
super().process()
```

이처럼 각 클래스가 MRO의 다음 구현으로 제어를 전달하도록 설계하는 방식을

협력적 다중 상속(Cooperative Multiple Inheritance)이라고 한다.

여기서 중요한 것은 메서드들이 서로 협력하도록 설계되어야 한다는 점이다.

예를 들어 어떤 클래스가 다음처럼 작성되어 있다면

```
class B(A):
    def process(self):
```

```
        print("B")
        A.process(self)
```

B는 다음 MRO 항목을 찾지 않고 A.process()를 직접 호출한다.

이렇게 하면 다중 상속 구조에서 중간에 위치한 다른 클래스의 메서드를 건너뛸 수 있다.

반면 다음 코드는

```
class B(A):
    def process(self):
        print("B")
        super().process()
```

실제 인스턴스의 MRO에 따라 다음 구현을 호출할 수 있다.

> 하지만 super()만 사용하면 항상 안전할까?

그렇지는 않다.

협력적 다중 상속에서는 다음 조건들도 중요하다.

- 각 메서드가 다음 구현을 호출하도록 설계되어야 한다.
- 메서드들의 인수 전달 규약이 서로 호환되어야 한다.
- 각 클래스가 자신이 담당하는 초기화와 처리를 명확히 구분해야 한다.
- MRO의 끝에서 호출이 자연스럽게 종료되도록 구성해야 한다.

2.9.9 다중 상속에서 __init__()은 어떻게 호출되는가?

다음 코드를 살펴보자.

```
class A:
    def __init__(self):
        print("A 초기화")


class B(A):
    def __init__(self):
        print("B 초기화 시작")
        super().__init__()
```

```
        print("B 초기화 종료")


class C(A):
    def __init__(self):
        print("C 초기화 시작")
        super().__init__()
        print("C 초기화 종료")


class D(B, C):
    def __init__(self):
        print("D 초기화 시작")
        super().__init__()
        print("D 초기화 종료")

d = D()
```

실행 결과:

```
D 초기화 시작
B 초기화 시작
C 초기화 시작
A 초기화
C 초기화 종료
B 초기화 종료
D 초기화 종료
```

여기서 `A.__init__()`은 한 번만 호출된다.

하지만 그 이유를 정확하게 이해해야 한다.

Python이 모든 부모 클래스의 `__init__()`을 자동으로 한 번씩 호출하기 때문은 아니다.

각 클래스의 `__init__()`이 `super.__init__()`을 호출하고, 그 호출이 MRO를 따라 이어지도록 작성되었기 때문이다.

만약 중간 클래스에서 `super().__init__()`을 생략하면 그 뒤의 초기화 체인이 중단될 수 있다.

`object.__init__()`은 무엇을 할까?

MRO의 마지막에는 보통 `object`가 존재한다.

`object.__init__()`은 기본적인 초기화 메서드이며, 사용자 정의 속성을 설정하는 특별한 초기화 작업을 수행하지 않는다.

따라서 협력적 초기화 체인의 마지막 단계로 사용할 수 있다.

다만 `object.__init__()`은 임의의 사용자 정의 키워드 인수를 받아들이지 않으므로, 다중 상속에서 인수를 전달할 때는 각 클래스가 자신에게 필요한 인수를 처리하고 나머지를 다음 메서드로 전달하도록 설계해야 한다.

2.9.10 super()와 일반적인 메서드 호출의 차이

지금까지의 내용을 비교해 보자.

```
class A:
    def method(self):
        print("A")


class B(A):
    def method(self):
        super().method()
```

여기서 다음 세 표현식은 서로 다른 의미를 가진다.

```
self.method()
```

현재 인스턴스에서 일반적인 속성 탐색을 수행한다.

따라서 B 인스턴스에서 호출하면 보통 `B.method()`가 다시 선택된다.

`B.method()` 내부에서 `self.method()`를 호출한다면 재귀 호출이 발생할 수 있다.

```
A.method(self)
```

`A` 클래스의 `method` 함수 객체를 명시적으로 선택해 호출한다.

이 경우 MRO에서 다음 클래스를 자동으로 선택하는 것은 아니다.

```
super().method()
```

현재 메서드의 기준 클래스 다음부터 MRO를 따라 `method`를 탐색한다.

이 차이를 표로 정리하면 다음과 같다.

표현식	탐색 방식
`self.method()`	인스턴스의 일반적인 속성 탐색
`A.method(self)`	`A` 클래스의 함수를 명시적으로 선택
`super().method()`	MRO에서 기준 클래스 다음부터 탐색

이 세 가지를 구분하면 상속 코드의 실행 흐름을 훨씬 정확하게 분석할 수 있다.

2.9.11 PyTorch와의 연결 - `super().__init__()`의 의미

이제 PyTorch 모델 코드를 다시 보자.

```
import torch
import torch.nn as nn


class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)
```

여기서 다음 표현식은

```
super().__init__()
```

단순히 `nn.Module.__init__(self)`를 축약한 문법으로만 이해하면 부족하다.

`MyModel`의 MRO에서 `MyModel` 다음에 위치한 클래스의 `__init__()`을 탐색하는 호출이다.

일반적인 단일 상속 구조에서는 다음과 같다.

```
print([cls.__name__ for cls in MyModel.__mro__])
```

실행 결과:

```
['MyModel', 'Module', 'object']
```

따라서 이 경우 `super().__init__()`은 `nn.Module.__init__()`을 호출하게 된다.

그런데 만약 다른 클래스를 함께 상속하면 MRO가 달라질 수 있다.

예를 들어 다음과 같은 구조를 생각해 보자.

```python
class LoggingMixin:
    def __init__(self, *args, **kwargs):
        print("LoggingMixin 초기화")
        super().__init__(*args, **kwargs)


class MyModel(LoggingMixin, nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10, 5)
```

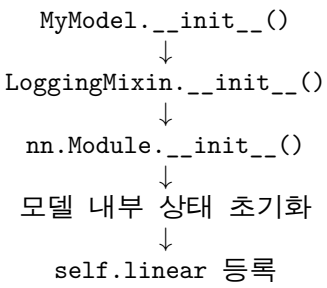
이 클래스의 MRO는 다음과 같다.

$$\texttt{MyModel} \rightarrow \texttt{LoggingMixin} \rightarrow \texttt{nn.Module} \rightarrow \texttt{object}$$

따라서 `MyModel.__init__()`에서 `super().__init__()`을 호출하면 먼저 `LoggingMixin.__init__()`이 실행된다.

그다음 `LoggingMixin.__init__()` 안의 `super().__init__()`이

`nn.Module.__init__()`을 호출하게 된다.



이처럼 `super()`는 상속 구조가 변경되더라도 MRO에 따라 다음 구현을 탐색할 수 있다.

다만 PyTorch에서 다중 상속을 설계할 때는 각 클래스의 초기화 방식이 협력적 상속을 지원하는지 확인해야 한다.

모든 클래스가 임의의 다중 상속 조합을 안전하게 지원하는 것은 아니다.

2.9.12 핵심 정리

이번 학습에서 반드시 이해해야 할 내용을 정리해 보자.

> MRO는 클래스 속성 탐색에 사용되는 선형 순서다.

```
Dog.__mro__
Dog.mro()
```

이를 통해 탐색 순서를 확인할 수 있다.

> super()는 부모 클래스 객체를 반환하지 않는다.

MRO를 기준으로 속성을 탐색하는 super 프록시 객체를 반환한다.

> 일반적인 인스턴스 메서드 안의 super()는 다음과 대응한다.

```
super(CurrentClass, self)
```

단, 이 대응은 정상적인 메서드 문맥에서 사용되는 인수 없는 super()를 설명하는 것이다.

> 다중 상속에서는 직접 부모와 MRO의 다음 클래스가 다를 수 있다.

```
D → B → C → A → object
```

이 구조에서 B 내부의 super()는 C의 메서드를 선택할 수 있다.

> Python은 C3 선형화로 MRO를 결정한다.

C3는 부모 클래스의 상대적 순서와 기존 상속 계층의 순서를 일관되게 유지하려고 한다.

> 협력적 다중 상속은 super() 호출 체인을 활용한다.

각 클래스가 MRO의 다음 구현으로 제어를 전달하도록 설계하는 방식이다.

`__init__()`도 MRO의 영향을 받는다.

부모 초기화가 자동으로 모두 실행되는 것은 아니며,

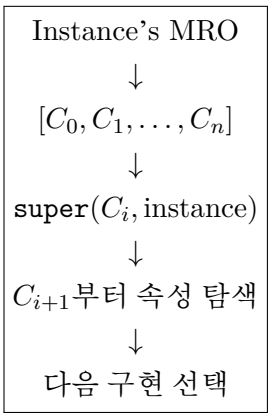
적절한 `super().__init__()` 호출이 필요하다.

PyTorch의 `super().__init__()`도 같은 원리로 동작한다.

일반적인 `MyModel(nn.Module)`에서는 `nn.Module.__init__()`을 호출하지만, 다중상속에서는 중간에 다른 클래스가 위치할 수 있다.

2.9.13 2.9에서 반드시 이해해야 할 결론

이번 학습의 핵심은 다음 구조다.



가장 중요한 문장은 다음과 같다.

`super()`는 직접적인 부모 클래스를 호출하는 문법이 아니라,

MRO에서 지정한 기준 클래스 다음부터 속성을 탐색하는 메커니즘이다.

따라서 같은 클래스에 정의된 `super()` 호출이라도 실제 인스턴스의 MRO가 달라지면 다음에 선택되는 메서드가 달라질 수 있다.

이 원리를 이해하면 Python의 다중 상속뿐 아니라 PyTorch의 `nn.Module`을 상속한 클래스에서 `super().__init__()`을 사용하는 이유도 더 정확하게 설명할 수 있다.

다음 2.10에서는 지금까지 학습한 객체 호출, 속성 탐색, 메서드 바인딩을 연결하여

__call__, __len__, __getitem__, __iter__ 등의 특수 메서드가 Python 문법과 어떻게 연결되는지 살펴볼 것이다.

2.10 특수 메서드 - Python 문법은 객체의 어떤 동작을 호출하는가?

2.9에서는 Python이 MRO를 통해 클래스 계층에서 메서드를 탐색하는 과정과 super()의 동작을 학습했다.

이번 2.10에서는 지금까지 학습한 객체 모델을 Python의 기본 문법과 연결할 것이다.

예를 들어 다음 표현식들을 살펴보자.

```
obj()
len(obj)
obj[0]
for x in obj:
    ...
```

겉으로 보면 서로 다른 문법이다.

그런데 Python은 이러한 연산을 객체의 **특수 메서드(Special Method)**와 연결하는 프로토콜을 제공한다.

이번 학습의 핵심은 다음 질문이다.

Python의 연산자와 내장 문법은

객체가 제공하는 특수 메서드와 어떻게 연결되는가?

특히 마지막에는 PyTorch에서 다음 코드가 실행되는 원리를 연결해 볼 것이다.

```
output = model(x)
```

이 코드를 이해하려면 __call__()과 일반 메서드 호출의 차이를 정확하게 구분해야 한다.

2.10.1 특수 메서드란 무엇인가?

특수 메서드는 Python이 특정 문법이나 내장 연산을 처리할 때 사용하는, 미리 정해진 이름의 메서드다.

일반적으로 이름의 앞뒤에 밑줄 두 개가 붙어 있다.

```
__init__
__call__
__len__
__getitem__
__iter__
__next__
```

이런 이름을 흔히 `Dunder Method`라고도 부른다.

Dunder는 `Double Underscore`의 줄임말이다.

일반적인 메서드와 비교해 보자.

```
class Person:
    def introduce(self):
        print("Hello")
```

`introduce()`는 우리가 직접 이름을 정한 메서드다.

```
p.introduce()
```

이 메서드를 호출하려면 일반적으로 해당 이름을 사용해야 한다.

반면 다음 메서드는 조금 다르다.

```
class Person:
    def __str__(self):
        return "Person instance"
```

이제 다음 코드를 실행하면

```
p = Person()
print(p)
```

실행 결과:

```
Person instance
```

우리는 `p.__str__()`을 직접 호출하지 않았다.

하지만 `print()`가 객체의 문자열 표현을 얻는 과정에서 `__str__()`이 사용된 것이다.

즉, 특수 메서드는 Python이 정해 놓은 연산 프로토콜에 참여하는 메서드다.

대표적인 특수 메서드

Python 문법	관련 특수 메서드	역할
`obj()`	`__call__`	객체 호출
`len(obj)`	`__len__`	길이 반환
`obj[key]`	`__getitem__`	항목 조회
`obj[key] = value`	`__setitem__`	항목 설정
`for x in obj`	`__iter__`, `__next__`	반복
`a + b`	`__add__` 등	덧셈
`a == b`	`__eq__`	동등성 비교
`str(obj)`	`__str__`	문자열 표현
`repr(obj)`	`__repr__`	개발자용 표현
`with obj:`	`__enter__`, `__exit__`	컨텍스트 관리

여기서 중요한 점이 있다.

특수 메서드는 일반 메서드처럼 직접 호출할 수도 있지만, Python의 내장 문법이 이를 처리하는 방식은 일반적인 인스턴스 속성 탐색과 다를 수 있다.

이 차이를 먼저 이해하고 넘어가자.

2.10.2 특수 메서드는 어디에서 탐색되는가?

2.7에서는 다음과 같은 속성 탐색 구조를 학습했다.

```
obj.attribute
```

일반적인 속성 접근은 디스크립터, 인스턴스 네임스페이스, 클래스 MRO 등을 고려한다.

그런데 특수 메서드의 암묵적 호출에는 중요한 차이가 있다.

Python은 일반적으로 `len(obj)`, `obj()` 같은 특수 연산을 처리할 때 인스턴

스 딕셔너리보다 객체의 타입과 그 MRO를 중심으로 특수 메서드를 탐색한다.

다음 예제를 보자.

```
class Box:
    pass


box = Box()

box.__len__ = lambda: 10

print(box.__len__())
```

실행 결과:

```
10
```

인스턴스에 `__len__`이라는 속성을 직접 설정했으므로 명시적으로 호출할 수 있다.

그렇다면 다음 코드는?

```
print(len(box))
```

실행 결과:

```
TypeError: object of type 'Box' has no len()
```

왜 이런 차이가 발생할까?

`len(box)`는 일반적인 `box.__len__` 속성 조회를 그대로 수행하지 않기 때문이다.

이 경우 Python은 `Box` 타입에서 사용할 수 있는 길이 프로토콜을 찾는데, `Box` 클래스에는 `__len__()`이 정의되어 있지 않다.

반대로 다음처럼 클래스에 정의하면 동작한다.

```
class Box:
    def __len__(self):
        return 10


box = Box()
```

```
print(len(box))
```

실행 결과:

```
10
```

이 차이를 정리하면 다음과 같다.

표현식	주요 탐색 방식
`obj.__len__()`	일반적인 속성 접근 후 호출
`len(obj)`	타입의 특수 메서드 프로토콜
`obj.__call__()`	일반적인 속성 접근 후 호출
`obj()`	타입의 호출 프로토콜

이것은 매우 중요한 구분이다.

특히 `__call__()`을 이해할 때 다시 사용하게 될 것이다.

2.10.3 `__call__()` - 인스턴스를 함수처럼 호출하기

이번 학습에서 가장 중요한 특수 메서드다.

2.5에서는 Python에서 함수도 객체이며, 호출 가능한 객체를 `Callable`이라고 부른다고 학습했다.

그렇다면 사용자 정의 클래스의 인스턴스도 Callable이 될 수 있을까?

가능하다.

다음 코드를 보자.

```
class Calculator:
    def __call__(self, a, b):
        return a + b


calc = Calculator()
```

이제 다음 표현식을 실행할 수 있다.

```
result = calc(10, 20)

print(result)
```

실행 결과:

```
30
```

여기서 `calc`는 함수 객체가 아니다.

```
print(type(calc))
```

실행 결과:

```
<class '__main__.Calculator'>
```

`calc`는 `Calculator` 클래스의 인스턴스다.

그런데도 `calc(10, 20)`이라는 함수 호출 형태의 문법을 사용할 수 있다.

그 이유는 `Calculator` 클래스가 `__call__()`을 정의했기 때문이다.

개념적인 호출 과정

```
calc(10,20)
```

일반적인 경우 다음과 같은 효과를 가진다.

```
type(calc).__call__(calc, 10, 20)
```

즉, 객체를 호출하면 그 객체의 타입에 정의된 호출 프로토콜이 사용된다.

여기서 `self`는 `calc` 인스턴스를 참조한다.

```
class Calculator:
    def __call__(self, a, b):
        print(self is calc)
        return a + b


calc = Calculator()
calc(10, 20)
```

실행 결과:

```
True
```

2.5에서 학습한 인스턴스 메서드의 `self` 바인딩 개념이 그대로 연결되는 것이다.

Callable 여부 확인

Python은 객체가 호출 가능한지 확인하기 위해 `callable()`을 제공한다.

```
class A:
    pass


class B:
    def __call__(self):
        return "Hello"


a = A()
b = B()

print(callable(a))
print(callable(b))
```

실행 결과:

```
False
True
```

`B` 인스턴스는 호출 프로토콜을 제공하기 때문에 Callable이다.

다만 `callable(obj)`가 `True`라는 사실이 실제 호출이 반드시 성공한다는 것을 보장하지는 않는다.

인수가 맞지 않거나 `__call__()` 내부에서 예외가 발생할 수 있기 때문이다.

2.10.4 클래스 호출과 인스턴스 호출은 같은 것인가?

이제 2.6에서 학습했던 내용과 연결해 보자.

```
class Calculator:
    def __init__(self):
```

```
        print("__init__ 실행")

    def __call__(self, x):
        print("__call__ 실행")
        return x * 2
```

다음 두 표현식은 서로 다른 동작을 한다.

```
calc = Calculator()

result = calc(10)
```

실행 순서를 확인해 보자.

```
calc = Calculator()
result = calc(10)

print(result)
```

실행 결과:

```
__init__ 실행
__call__ 실행
20
```

여기서 구분해야 할 객체가 두 개 있다.

- Calculator : 클래스 객체
- calc : Calculator의 인스턴스 객체

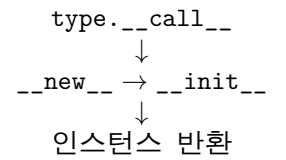
Calculator()는 클래스 객체의 호출이다.

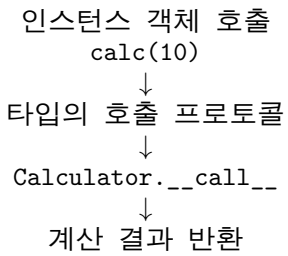
일반적인 클래스에서는 메타클래스 type의 호출 메커니즘을 통해 __new__()와 __init__()이 실행된다.

반면 calc(10)은 인스턴스 객체의 호출이다.

이때는 Calculator.__call__()이 사용된다.

클래스 객체 호출
Calculator()
↓





이 둘은 모두 호출 문법 ()를 사용하지만, 호출되는 객체가 다르기 때문에 실행 경로도 다르다.

클래스의 인스턴스를 생성하는 호출과, 생성된 인스턴스를 함수처럼 사용하는 호출은 구분해야 한다.

2.10.5 __len__() - 길이 프로토콜

다음 코드를 보자.

```
class Dataset:
    def __init__(self, data):
        self.data = data

    def __len__(self):
        return len(self.data)
```

이제 다음처럼 사용할 수 있다.

```
dataset = Dataset([10, 20, 30])

print(len(dataset))
```

실행 결과:

```
3
```

여기서 `len(dataset)`은 Dataset 클래스가 제공하는 길이 프로토콜을 사용한다.

`__len__()`의 반환값은 0 이상의 정수여야 한다.

다음 코드를 보자.

```
class Dataset:
    def __len__(self):
        return -1

dataset = Dataset()
print(len(dataset))
```

실행하면 오류가 발생한다.

```
ValueError: __len__() should return >= 0
```

`__len__()`과 참·거짓 평가

여기서 한 가지 연결되는 동작이 있다.

Python은 객체의 참·거짓을 판단할 때 `__bool__()`을 우선 사용하고, 적절한 `__bool__()`이 없으면 `__len__()`을 사용할 수 있다.

```
class Dataset:
    def __init__(self, data):
        self.data = data

    def __len__(self):
        return len(self.data)


a = Dataset([])
b = Dataset([10, 20])

print(bool(a))
print(bool(b))
```

실행 결과:

```
False
True
```

길이가 0이면 거짓, 0보다 크면 참으로 평가되는 것이다.

따라서 `__len__()`은 단순히 길이를 반환하는 것 외에도 객체의 불리언 평가에 영향을 줄 수 있다.

2.10.6 `__getitem__()`과 `__setitem__()` - 인덱싱과 항목 설정

다음 표현식을 보자.

```
data[0]
```

우리는 리스트나 튜플에서 자주 사용하는 문법으로 알고 있다.

하지만 사용자 정의 객체도 이 문법을 지원할 수 있다.

```
class Dataset:
    def __init__(self, data):
        self.data = data

    def __getitem__(self, index):
        return self.data[index]
```

이제 다음처럼 사용할 수 있다.

```
dataset = Dataset([10, 20, 30])

print(dataset[0])
print(dataset[2])
```

실행 결과:

```
10
30
```

개념적으로 다음 표현식은

```
dataset[0]
```

객체의 타입이 제공하는 `__getitem__()` 프로토콜을 이용한다.

```
Dataset.__getitem__(dataset, 0)
```

여기서 `index`는 단순한 정수만 의미하지 않는다.

슬라이싱을 사용하면 slice 객체가 전달될 수도 있다.

```
print(dataset[0:2])
```

실행 결과:

```
[10, 20]
```

이 예제에서는 내부 리스트에 그대로 전달했기 때문에 슬라이싱도 지원되는 것이다.

> 항목을 수정하려면?

```
dataset[0] = 100
```

이 문법을 지원하려면 `__setitem__()`을 정의해야 한다.

```
class Dataset:
    def __init__(self, data):
        self.data = data

    def __getitem__(self, index):
        return self.data[index]

    def __setitem__(self, index, value):
        self.data[index] = value

dataset = Dataset([10, 20, 30])

dataset[0] = 100

print(dataset[0])
```

실행 결과:

```
100
```

여기서 2.7에서 학습한 속성 설정과 비교해 보자.

표현식	관련 메커니즘
`obj.name = value`	속성 설정, `__setattr__`
`obj[key] = value`	항목 설정, `__setitem__`
`obj.name`	속성 탐색, `__getattribute__` 등
`obj[key]`	항목 조회, `__getitem__`

두 문법은 겉으로 비슷해 보여도 서로 다른 프로토콜을 사용한다.

2.10.7 `__iter__()`와 `__next__()` - 반복 프로토콜

이 부분은 Iterator와 Generator 개념을 연결하는 내용이다.

먼저 다음 코드를 보자.

```
for x in [10, 20, 30]:
    print(x)
```

for문은 내부적으로 반복 가능한 객체에서 Iterator를 얻고, 다음 값을 반복적으로 요청하는 구조를 사용한다.

핵심 함수는 다음 두 가지다.

```
iter(obj)
next(iterator)
```

이 함수들과 연결되는 특수 메서드는 다음과 같다.

```
__iter__()
__next__()
```

Iterable과 Iterator의 차이

- Iterable : iter()를 통해 Iterator를 얻을 수 있는 객체

- Iterator : __next__()로 다음 값을 제공하고,
  __iter__()가 자기 자신을 반환하는 객체

예제를 보자.

```
class NumberIterator:
    def __init__(self, limit):
        self.limit = limit
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= self.limit:
            raise StopIteration

        value = self.current
        self.current += 1

        return value
```

이제 다음처럼 사용할 수 있다.

```
numbers = NumberIterator(3)

for x in numbers:
    print(x)
```

실행 결과:

```
0
1
2
```

여기서 `NumberIterator` 인스턴스는 Iterator다.

`__iter__()`가 자기 자신을 반환하고, `__next__()`가 다음 값을 제공하기 때문이다.

for 문의 개념적 실행 과정

```
iterator = iter(numbers)

while True:
    try:
        value = next(iterator)
    except StopIteration:
        break

    print(value)
```

이 코드는 앞의 for 문이 사용하는 반복 프로토콜을 이해하기 위한 단순화된 표현이다.

실제 for문에는 break, continue, 예외 전파 등 추가적인 실행 규칙이 있다.

Iterator가 소진되면?

```
numbers = NumberIterator(3)

print(list(numbers))
print(list(numbers))
```

실행 결과:

```
[0, 1, 2]
[]
```

Iterator는 소진된 상태를 그대로 유지할 수 있다.

반면 일반적인 리스트는 반복할 때마다 새로운 Iterator를 얻을 수 있기 때문에 재반복이 가능하다.

2.10.8 Iterable과 Iterator를 별도로 설계하는 이유

실제 클래스 설계에서는 Iterable과 Iterator를 구분하는 경우가 많다.

다음 예제를 보자.

```
class Numbers:
    def __init__(self, limit):
        self.limit = limit

    def __iter__(self):
        return NumberIterator(self.limit)
```

이제 다음처럼 사용할 수 있다.

```
numbers = Numbers(3)

print(list(numbers))
print(list(numbers))
```

실행 결과:

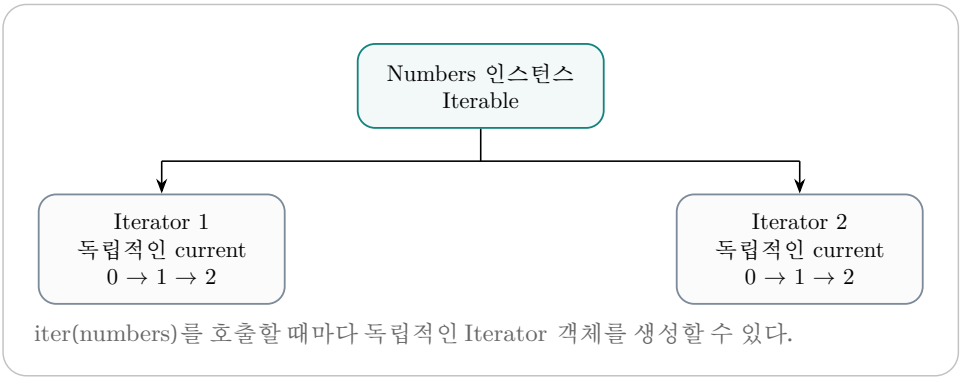
```
[0, 1, 2]
[0, 1, 2]
```

왜 이번에는 두 번 모두 값이 출력될까?

`Numbers.__iter__()`가 호출될 때마다 새로운 `NumberIterator` 인스턴스를 반환하기 때문이다.

즉, `Numbers` 인스턴스는 반복 가능한 객체지만, 그 자체가 Iterator는 아니다.

이 구조를 그림으로 나타내면 다음과 같다.



iter(numbers)를 호출할 때마다 독립적인 Iterator 객체를 생성할 수 있다.

이 설계는 같은 데이터셋을 여러 번 순회해야 하는 상황에서 유용하다.

`__getitem__()`만으로도 반복이 가능한가?

Python은 전통적인 시퀀스 반복 프로토콜도 지원한다.

따라서 `__iter__()`가 없어도 정수 인덱스 0부터 시작하는 `__getitem__()`을 이용해 반복할 수 있는 경우가 있다.

다만 이 방식에서는 적절한 시점에 `IndexError`가 발생해야 반복이 종료된다.

현대적인 사용자 정의 Iterable에서는 의도를 명확하게 표현하기 위해

`__iter__()`를 정의하는 것이 일반적으로 더 적절하다.

2.10.9 `__add__()`와 `__eq__()` - 연산자 오버로딩

Python의 연산자도 특수 메서드와 연결된다.

다음 코드를 보자.

```
a + b
```

일반적으로 덧셈이라고 생각하지만, 객체의 타입에 따라 의미가 달라질 수 있다.

예를 들어 숫자는 산술 덧셈을 수행하고, 리스트는 이어 붙이기를 수행한다.

사용자 정의 클래스에서도 덧셈 연산을 구현할 수 있다.

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector2D(
            self.x + other.x,
            self.y + other.y
        )
```

이제 다음처럼 사용할 수 있다.

```
v1 = Vector2D(1, 2)
v2 = Vector2D(3, 4)

v3 = v1 + v2

print(v3.x, v3.y)
```

실행 결과:

```
4 6
```

이때 `v1 + v2`는 `__add__()`를 활용하는 연산이다.

개념적으로는 다음과 비슷하다.

```
v1.__add__(v2)
```

다만 실제 Python의 이항 연산 처리에는 오른쪽 피연산자의 `__radd__()`와 `NotImplemented`를 이용한 대체 처리 등이 포함될 수 있다.

따라서 모든 `a + b`를 단순히 `a.__add__(b)` 한 번의 호출로 동일시하면 안 된다.

__eq__() - 값의 동등성

2.3에서는 is와 ==의 차이를 학습했다.

is : 객체 동일성

== : 동등성 비교

== 연산은 __eq__() 프로토콜과 연결된다.

```
class Person:
    def __init__(self, name):
        self.name = name

    def __eq__(self, other):
        if not isinstance(other, Person):
            return NotImplemented

        return self.name == other.name

p1 = Person("Kim")
p2 = Person("Kim")

print(p1 is p2)
print(p1 == p2)
```

실행 결과:

```
False
True
```

두 인스턴스는 서로 다른 객체다.

하지만 __eq__()를 통해 이름이 같으면 동등하다고 판단하도록 정의했다.

여기서 중요한 사실은 객체의 동일성과 동등성은 별도의 개념이라는 점이다.

또한 __eq__()를 재정의하면 해시 가능성에도 영향을 줄 수 있다.

기본적으로 __hash__가 비활성화되는 경우가 있으므로, 딕셔너리 키나 집합 원소로 사용할 객체를 설계할 때는 동등성과 해시 규약을 함께 고려해야 한다.

2.10.10 `__str__()`과 `__repr__()` - 객체의 문자열 표현

다음 코드를 보자.

```
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Kim")

print(p)
```

일반적인 출력은 다음과 비슷하다.

```
<__main__.Person object at 0x...>
```

이 표현은 객체의 타입과 식별에 관한 기본적인 정보를 제공한다.

하지만 사용자에게 더 의미 있는 문자열을 제공하고 싶다면 `__str__()`을 정의할 수 있다.

```
class Person:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Person(name={self.name})"

p = Person("Kim")

print(p)
```

실행 결과:

```
Person(name=Kim)
```

반면 `__repr__()`은 주로 개발자가 객체를 확인하고 디버깅할 때 사용할 표현을 제공한다.

```
class Person:
    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return f"Person(name={self.name!r})"
```

```
p = Person("Kim")

print(repr(p))
```

실행 결과:

```
Person(name='Kim')
```

두 메서드의 차이를 정리하면 다음과 같다.

메서드	주된 목적
`__str__()`	읽기 쉬운 문자열 표현
`__repr__()`	개발자에게 유용한 객체 표현

`__str__()`이 정의되어 있지 않으면 기본적으로 `__repr__()`을 활용하는 동작도 있다.

일반적으로 `__repr__()`은 가능하다면 객체의 상태를 명확하게 드러내도록 작성하는 것이 좋다.

2.10.11 `__enter__()`와 `__exit__()` - 컨텍스트 매니저

이전에 파일 입출력을 학습하면서 다음 문법을 사용했을 것이다.

```
with open("data.txt", "w", encoding="utf-8") as f:
    f.write("Hello")
```

여기서 `with`문은 컨텍스트 매니저(Context Manager) 프로토콜을 활용한다.

대표적인 특수 메서드는 다음 두 가지다.

```
__enter__()
__exit__()
```

간단한 예제를 보자.

```
class Resource:
    def __enter__(self):
        print("자원 획득")
```

```
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("자원 해제")

with Resource() as resource:
    print("작업 실행")
```

실행 결과:

```
자원 획득
작업 실행
자원 해제
```

이 과정은 개념적으로 다음과 같다.

1. `Resource()` 인스턴스를 생성한다.
2. `__enter__()`를 호출한다.
3. 반환값을 `resource`에 바인딩한다.
4. `with` 블록을 실행한다.
5. 블록을 빠져나가면서 `__exit__()`를 호출한다.

`with` 블록 내부에서 예외가 발생해도 `__exit__()`이 호출되어 정리 작업을 수행할 수 있다.

또한 `__exit__()`의 반환값이 참이면 해당 예외를 억제할 수 있다.

따라서 `with`문은 단순히 코드를 묶는 문법이 아니라 자원의 획득과 해제를 구조화하는 프로토콜이다.

2.10.12 PyTorch와의 연결 - `model(x)`는 왜 가능한가?

이제 지금까지의 내용을 PyTorch와 연결해 보자.

다음 모델을 정의했다고 하자.

```
import torch
import torch.nn as nn
```

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)
```

모델 인스턴스를 생성해 보자.

```
model = MyModel()
```

여기서 model은 MyModel 인스턴스다.

```
print(type(model))
```

출력:

```
<class '__main__.MyModel'>
```

그런데 다음 표현식이 가능하다.

```
x = torch.randn(2, 10)

output = model(x)
```

왜 그럴까?

MyModel은 nn.Module을 상속하고 nn.Module은 인스턴스를 호출 가능하게 만드는 __call__ 메커니즘을 제공하기 때문이다.

따라서 model은 Callable이다.

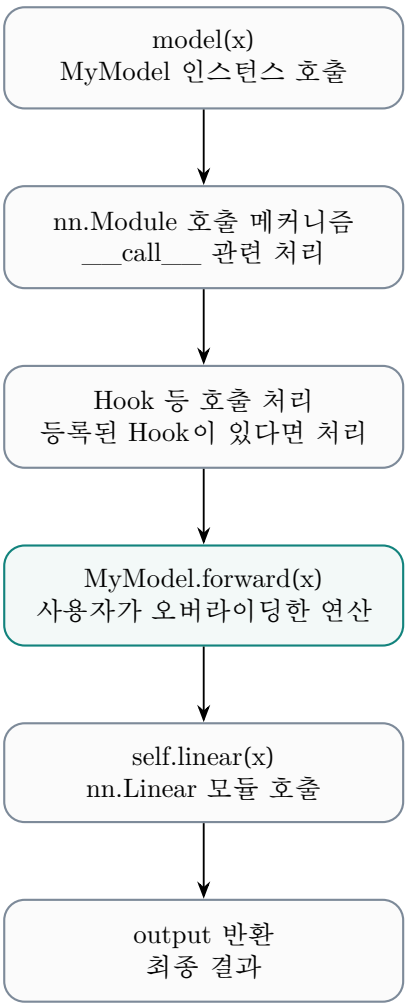
```
print(callable(model))
```

실행 결과:

```
True
```

실행 흐름

일반적인 `nn.Module`의 호출 과정은 다음과 같이 이해할 수 있다.



PyTorch 모델 호출의 개념적 흐름
Hook의 실행 여부와 세부 순서는 등록 상태 및 PyTorch 구현에 따라 달라진다.

여기서 중요한 점은 `MyModel`에 `__call__()`을 직접 정의하지 않았다는 사실이다.

```
class MyModel(nn.Module):
    ...
```

`nn.Module`에서 상속받은 호출 메커니즘을 사용하고 있는 것이다.

반면 `forward()`는 `MyModel`에서 직접 정의했다.

```
def forward(self, x):
    return self.linear(x)
```

이것은 2.8에서 학습한 메서드 오버라이딩이다.

따라서 PyTorch 모델 호출은 다음 개념들이 결합된 구조라고 볼 수 있다.

- 상속: MyModel(nn.Module)
- 메서드 오버라이딩: forward()
- 특수 메서드: __call__()
- Callable 객체: model
- 모듈 합성: self.linear = nn.Linear(...)

model(x)와 model.forward(x)의 차이

다음 두 표현식을 비교해 보자.

```
output1 = model(x)

output2 = model.forward(x)
```

두 코드 모두 모델의 계산 결과를 얻을 수 있다.

하지만 실행 경로는 다르다.

첫 번째는 nn.Module의 호출 메커니즘을 거쳐 forward()를 실행한다.

두 번재는 일반적인 메서드 접근을 통해 forward()를 직접 호출한다.

따라서 두 번째 방식은 nn.Module의 호출 과정에서 수행되는 Hook 등의 처리를 우회할 수 있다.

PyTorch에서는 일반적으로 model.forward(x)보다 model(x)를 사용하는 것이 적절하다.

이 차이는 단순한 코드 스타일의 문제가 아니라, Python의 객체 호출 프로토콜과 PyTorch의 모듈 설계에서 비롯된 차이다.

2.10.13 전체 개념 연결

지금까지 학습한 내용을 하나의 객체 모델로 연결해 보자.

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)


model = MyModel()
output = model(x)
```

이 코드에는 2.2부터 2.10까지의 개념들이 포함되어 있다.

학습 단원	코드에서의 의미
2.2 클래스도 객체인가?	MyModel은 클래스 객체
2.3 클래스와 인스턴스	model은 MyModel 인스턴스
2.4 인스턴스 속성	self.linear 속성
2.5 메서드와 self	forward(self, x)
2.6 __new__, __init__	모델 생성과 초기화
2.7 속성 탐색	self.linear 조회와 등록
2.8 상속과 오버라이딩	nn.Module 상속, forward 재정의
2.9 super()와 MRO	super().__init__()
2.10 특수 메서드	model(x)와 __call__()

이제 PyTorch의 모델 정의는 단순한 라이브러리 사용법이 아니라, Python의 객체 모델 위에서 구성된 설계로 이해할 수 있다.

2.10.14 핵심 정리

이번 학습에서 반드시 기억해야 할 내용을 정리해 보자.

> 특수 메서드는
> Python의 문법 및 내장 연산과 연결되는 프로토콜이다.

```
obj()
```

```
len(obj)
obj[key]
for x in obj:
    ...
```

이러한 문법은 각각 대응하는 특수 메서드와 연결될 수 있다.

> 특수 메서드의 암묵적 호출은
> 인스턴스 속성 탐색과 다를 수 있다.

인스턴스에 `__len__` 속성을 설정했다고 해서 `len(obj)`가 반드시 동작하는 것은 아니다.

> `__call__()`을 제공하는 인스턴스는 Callable이 될 수 있다.

```
class Calculator:
    def __call__(self, x):
        return x * 2
```

> 클래스 호출과 인스턴스 호출은 구분해야 한다.

```
calc = Calculator()
result = calc(10)
```

첫 번째는 클래스 객체 호출이고, 두 번째는 인스턴스 객체 호출이다.

> `__len__()`은 길이 프로토콜을 제공한다.

```
len(obj)
```

> `__getitem__()`과 `__setitem__()`은
> 항목 조회 및 설정을 지원한다.

```
obj[key]
obj[key] = value
```

> `__iter__()`와 `__next__()`은 반복 프로토콜의 핵심이다.

```
iter(obj)
next(iterator)
```

연산자도 특수 메서드와 연결된다.

```
a + b
a == b
```

다만 실제 연산자 처리에는 대체 메서드와 NotImplemented 등의 규칙이 존재할 수 있다.

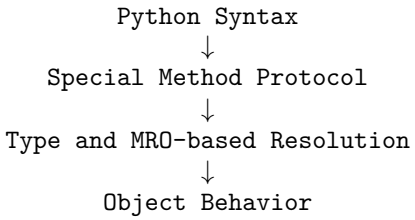
with 문은 컨텍스트 매니저 프로토콜을 사용한다.

```
__enter__()
__exit__()
```

PyTorch의 model(x)는 상속된 호출 메커니즘과
오버라이딩된 forward()가 결합된 구조다.

2.10.15 2.10에서 반드시 이해해야 할 결론

이번 학습의 핵심은 다음과 같다.



Python의 객체 모델에서 중요한 것은 객체가 단순히 데이터와 메서드를 보유하는 데 그치지 않는다는 점이다.

객체가 특정 특수 메서드 프로토콜을 제공하면, Python의 기본 문법 자체가 그 객체의 동작을 사용할 수 있다.

이제 다음 코드를 보면

```
output = model(x)
```

단순히 모델을 실행한다고 이해하는 것을 넘어, 다음과 같이 해석할 수 있다.

model은 MyModel 클래스의 인스턴스 객체다.

MyModel은 nn.Module을 상속한다.

nn.Module은 객체 호출을 위한 __call__ 메커니즘을 제공한다.

model(x)는 그 호출 메커니즘을 사용한다.

호출 과정에서 MyModel이 오버라이딩한 forward()가 실행된다.

forward() 내부에서 하위 모듈인 nn.Linear 인스턴스도 호출된다.

이것이 Python 객체 모델과 PyTorch의 모델 실행 구조가 연결되는 지점이다.

다음 2.11에서는 self, cls, @classmethod, @staticmethod를 비교한다.

특히 2.5에서 학습한 바운드 메서드와 2.7에서 살펴본 디스크립터 프로토콜을 연결하면, 세 종류의 메서드가 서로 다르게 동작하는 이유를 구조적으로 이해할 수 있을 것이다.

2.11 메서드의 종류 - 인스턴스 메서드, 클래스 메서드, 정적 메서드

2.5에서 인스턴스 메서드가 클래스에 저장된 함수 객체이며, 인스턴스를 통해 접근하면 바운드 메서드(Bound Method)가 된다는 사실을 학습했다.

2.7에서 디스크립터 프로토콜을, 2.8과 2.9에서 상속과 MRO를 살펴보았다.

이번 2.11에서는 이 개념들을 연결하여 Python의 세 가지 메서드를 비교해 볼 것이다.

인스턴스 메서드 - self

클래스 메서드 - cls, @classmethod

정적 메서드 - @staticmethod

이번 학습의 핵심 질문은 다음과 같다.

세 메서드는 모두 클래스 안에서 정의되는데,

왜 어떤 메서드는 인스턴스를 자동으로 전달받고,

어떤 메서드는 클래스를 전달받으며,

어떤 메서드는 아무것도 자동으로 전달받지 않을까?

이 차이를 이해하면 단순히 문법을 구분하는 수준을 넘어, Python이 함수 객체를 어떻게 바인딩하는지까지 설명할 수 있다.

2.11.1 세 가지 메서드의 기본 구조

먼저 다음 클래스를 살펴보자.

```
class Person:
    species = "Human"

    def __init__(self, name):
        self.name = name

    def introduce(self):
        return f"My name is {self.name}"

    @classmethod
    def get_species(cls):
        return cls.species

    @staticmethod
    def is_valid_name(name):
        return isinstance(name, str) and len(name) > 0
```

이 클래스에는 세 가지 메서드가 정의되어 있다.

```
p = Person("Kim")

print(p.introduce())
print(Person.get_species())
print(Person.is_valid_name("Lee"))
```

실행 결과:

```
My name is Kim
Human
True
```

겉으로 보면 모두 메서드 호출이다.

하지만 내부적으로 전달되는 첫 번째 인수가 서로 다르다.

구분	정의 방식	자동 전달 대상	주된 역할
인스턴스 메서드	`def method(self)`	인스턴스	인스턴스 상태 사용
클래스 메서드	`@classmethod`	클래스 객체	클래스 상태 및 클래스 기반 생성
정적 메서드	`@staticmethod`	없음	클래스와 관련된 독립적 기능

여기서 중요한 점은 `self`와 `cls`가 Python의 예약어가 아니라는 사실이다.

둘 다 일반적인 매개변수 이름이다.

실제로 중요한 것은 메서드 접근 시 Python이 어떤 객체를 자동으로 바인딩하는가이다.

2.11.2 인스턴스 메서드 - `self`는 어떻게 전달되는가?

다음 코드를 보자.

```
class Person:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        return self.name

p = Person("Kim")

print(p.introduce())
```

실행 결과:

```
Kim
```

2.5에서 배운 것처럼 다음 두 표현식은 일반적인 메서드 호출에서 같은 효과를 가진다.

```
p.introduce()

Person.introduce(p)
```

왜 가능할까?

클래스에 정의된 일반 함수 객체는 디스크립터 프로토콜의 `__get__()`을 제공하기 때문이다.

클래스 네임스페이스 확인

```
print(Person.__dict__["introduce"])
```

출력 예시:

```
<function Person.introduce at 0x...>
```

`Person.__dict__`에 저장된 것은 함수 객체다.

반면 인스턴스를 통해 접근하면 다음과 같다.

```
method = p.introduce

print(method.__self__ is p)
print(method.__func__ is Person.introduce)
```

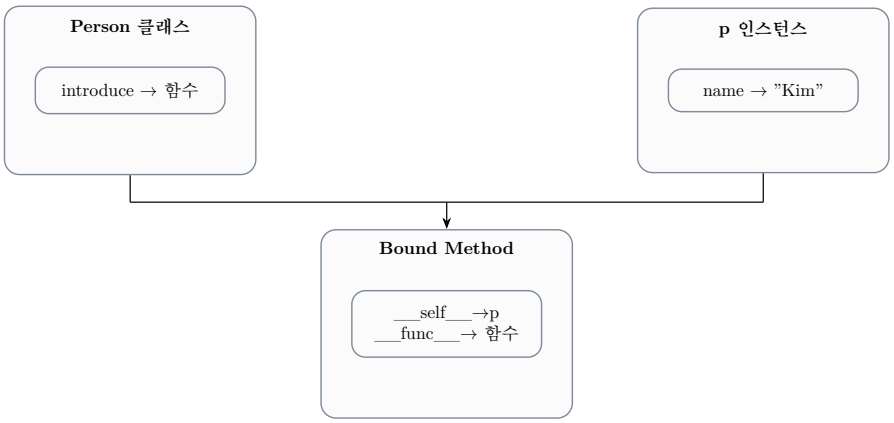
실행 결과:

```
True
True
```

여기서

- `__self__`는 바인딩된 인스턴스
- `__func__`는 원본 함수 객체

를 나타낸다.



p.introduce에 접근하면 인스턴스와 함수 객체가 연결된 바운드 메서드를 얻는다.

따라서 인스턴스 메서드의 핵심은 단순히 `self`라는 이름을 사용한다는 것이 아니다.

함수 객체가 인스턴스에 바인딩되어 호출될 때 그 인스턴스가 첫 번째 인수로 전달된다는 것이 핵심이다.

2.11.3 클래스 메서드 - `@classmethod`는 무엇을 바꾸는가?

이번에는 클래스 메서드를 살펴보자.

```
class Person:
    species = "Human"

    @classmethod
    def get_species(cls):
        return cls.species
```

다음처럼 호출할 수 있다.

```
print(Person.get_species())
```

실행 결과:

```
Human
```

여기서 `cls`는 `Person` 클래스 객체를 참조한다.

따라서 다음과 같은 효과가 있다.

```
Person.get_species()

Person.get_species.__func__(Person)
```

두 번째 표현식은 바운드 메서드에 저장된 원본 함수를 명시적으로 호출한 형태다.

cls는 어디에서 오는가?

`@classmethod`는 일반 함수 객체를 `classmethod` 디스크립터 객체로 감싸는 데코레이터다.

2.5에서 학습했던 데코레이터 변환식을 적용하면 다음과 같이 이해할 수 있다.

```
class Person:
    def get_species(cls):
        return cls.species

    get_species = classmethod(get_species)
```

이는 다음 정의와 대응한다.

```
class Person:
    @classmethod
    def get_species(cls):
        return cls.species
```

중요한 점은 `classmethod`가 원본 함수 객체의 코드를 변경하는 것이 아니라, 클래스를 바인딩하는 방식으로 접근할 수 있게 하는 디스크립터 객체를 생성한다는 사실이다.

클래스 네임스페이스 확인

```
class Person:
    @classmethod
    def get_species(cls):
        return cls.__name__

print(type(Person.__dict__["get_species"]))
```

실행 결과:

```
<class 'classmethod'>
```

일반 인스턴스 메서드와는 다르다.

일반 메서드는 클래스 네임스페이스에 함수 객체가 직접 저장된다.

반면 `@classmethod`를 사용하면 `classmethod` 객체가 저장된다.

바운드 메서드 확인

```
method = Person.get_species

print(method.__self__ is Person)
print(method.__func__ is Person.__dict__["get_species"].__func__)
```

실행 결과:

```
True
True
```

여기서 가장 중요한 부분은 다음이다.

```
method.__self__ is Person
```

인스턴스 메서드에서는 `__self__`가 인스턴스였지만, 클래스 메서드에서는 클래스 객체다.

즉, 두 메서드는 모두 바운드 메서드가 될 수 있지만 바인딩 대상이 다르다는 것이다.

2.11.4 클래스 메서드를 인스턴스에서 호출하면?

다음 코드를 보자.

```
class Person:
    species = "Human"

    @classmethod
    def get_species(cls):
        print(cls)
        return cls.species

p = Person()

p.get_species()
```

출력:

```
<class '__main__.Person'>
```

여기서 `p.get_species()`를 호출했는데도 `cls`는 `p`가 아니다.

`Person` 클래스 객체다.

확인해 보자.

```
print(p.get_species.__self__ is Person)
```

실행 결과:

```
True
```

즉, 클래스 메서드는 클래스 객체로 접근하든 인스턴스로 접근하든 클래스를 바인딩한다.

이것은 인스턴스 메서드와 중요한 차이다.

비교

```
class Person:
    def instance_method(self):
        return self

    @classmethod
    def class_method(cls):
        return cls

p = Person()

print(p.instance_method() is p)
print(p.class_method() is Person)
```

실행 결과:

```
True
True
```

2.11.5 클래스 메서드와 상속 - cls는 어떤 클래스를 참조하는가?

이제 2.8의 상속과 연결해 보자.

```
class Animal:
    species = "Animal"

    @classmethod
    def identify(cls):
        return cls.species


class Dog(Animal):
    species = "Dog"
```

다음 코드를 실행하면 어떻게 될까?

```
print(Animal.identify())
print(Dog.identify())
```

실행 결과:

```
Animal
Dog
```

여기서 중요한 질문이 있다.

`identify()`는 `Animal` 클래스에 정의되어 있다.

그런데 `Dog.identify()`를 호출하면 왜 `Dog`가 출력될까?

클래스 메서드의 `cls`는 메서드가 정의된 클래스로 고정되는 것이 아니라, 접근에 사용된 클래스에 따라 바인딩되기 때문이다.

```
print(Animal.identify.__self__ is Animal)
print(Dog.identify.__self__ is Dog)
```

실행 결과:

```
True
True
```

이 구조는 2.8에서 학습한 인스턴스 메서드의 `self`와 비슷하다.

부모 클래스에 정의된 메서드를 자식 클래스가 사용할 수 있고, 그때 실제 바인딩 대상이 달라지는 것이다.

중요한 비교

구분	인스턴스 메서드	클래스 메서드
바인딩 대상	실제 인스턴스	접근 대상 클래스
관례적 첫 인수	`self`	`cls`
상속 시	자식 인스턴스를 `self`로 받을 수 있음	자식 클래스를 `cls`로 받을 수 있음

이 차이가 클래스 메서드를 활용하는 중요한 이유다.

2.11.6 클래스 메서드의 대표적 용도 - 대체 생성자

클래스 메서드가 특히 유용한 경우는

대체 생성자(Alternative Constructor)를 정의할 때다.

먼저 다음 코드를 보자.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

일반적인 인스턴스 생성은 다음과 같다.

```
p = Person("Kim", 30)
```

그런데 다음과 같은 문자열에서 객체를 생성하고 싶다고 하자.

```
data = "Kim,30"
```

이를 위해 클래스 메서드를 사용할 수 있다.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
    @classmethod
    def from_string(cls, data):
        name, age = data.split(",")

        return cls(name, int(age))
```

이제 다음처럼 사용할 수 있다.

```
p = Person.from_string("Kim,30")

print(p.name)
print(p.age)
```

실행 결과:

```
Kim
30
```

여기서 다음 표현식에 주목하라.

```
return cls(name, int(age))
```

cls는 클래스 객체다.

따라서 cls(...)는 해당 클래스 객체를 호출하여 인스턴스를 생성하는 동작이다.

2.6에서 학습했던 클래스 호출 메커니즘과 연결된다.

> 왜 Person(...) 대신 cls(...)를 사용할까?

상속 관계를 고려하면 차이가 명확해진다.

```
class Student(Person):
    pass
```

다음 코드를 실행해 보자.

```
student = Student.from_string("Lee,20")

print(type(student))
```

실행 결과:

```
<class '__main__.Student'>
```

왜 Student 인스턴스가 생성되었을까?

Student.from_string()으로 접근했기 때문에

클래스 메서드의 cls가 Student로 바인딩되었기 때문이다.

따라서

```
return cls(name, int(age))
```

는 이 호출에서 사실상 다음과 같은 효과를 가진다.

```
return Student(name, int(age))
```

반면 클래스 메서드 내부에서 다음처럼 작성했다면

```
return Person(name, int(age))
```

항상 Person 인스턴스가 생성된다.

자식 클래스의 타입을 유지할 수 없을 것이다.

이것이 클래스 메서드가 대체 생성자에 적합한 중요한 이유다.

단, 자식 클래스가 생성자 인수를 다르게 정의했다면 cls(...) 호출이 실패할 수 있으므로 상속 계층의 생성자 규약도 호환되어야 한다.

2.11.7 정적 메서드 - @staticmethod는 무엇을 하는가?

이번에는 정적 메서드를 살펴보자.

```
class Person:
    @staticmethod
    def is_valid_age(age):
        return isinstance(age, int) and age >= 0
```

다음처럼 호출할 수 있다.

```
print(Person.is_valid_age(30))
print(Person.is_valid_age(-5))
```

실행 결과:

```
True
False
```

여기서 is_valid_age()에는 self도 없고 cls도 없다.

왜 그럴까?

@staticmethod는 인스턴스나 클래스를 자동으로 바인딩하지 않기 대문이다.

즉, 정적 메서드는 클래스의 네임스페이스에 속해 있지만, 호출할 때 자동으로 추가되는 첫 번째 인수가 없다.

데코레이터 변환식

다음 코드는

```
class Person:
    @staticmethod
    def is_valid_age(age):
        return age >= 0
```

개념적으로 다음과 대응한다.

```
class Person:
    def is_valid_age(age):
        return age >= 0

    is_valid_age = staticmethod(is_valid_age)
```

클래스 네임스페이스에는 staticmethod 객체가 저장된다.

```
print(type(Person.__dict__["is_valid_age"]))
```

실행 결과:

```
<class 'staticmethod'>
```

하지만 클래스나 인스턴스를 통해 접근하면 일반적으로 감싸진 함수 객체를 얻는다.

```
p = Person()
```

```
print(Person.is_valid_age is p.is_valid_age)
```

실행 결과:

```
True
```

둘 다 같은 함수 객체를 반환할 수 있다.

인스턴스나 클래스에 바인딩하는 과정이 없기 때문이다.

staticmethod는 왜 필요한가?

다음 함수를 생각해 보자.

```
def is_valid_age(age):
    return isinstance(age, int) and age >= 0
```

이 함수는 독립적으로 존재해도 문제가 없다.

하지만 Person과 관련된 검증 기능이므로 클래스의 네임스페이스 안에 배치하고 싶을 수 있다.

```
class Person:
    @staticmethod
    def is_valid_age(age):
        return isinstance(age, int) and age >= 0
```

이렇게 하면 다음과 같이 기능을 묶을 수 있다.

```
Person.is_valid_age(30)
```

정적 메서드는 클래스와 개념적으로 관련된 함수를 클래스의 네임스페이스 안에 배치하면서, 인스턴스나 클래스를 자동으로 전달하지 않도록 하는 수단이라고 이해하면 된다.

다만 함수가 클래스와 특별한 관련이 없다면 굳이 정적 메서드로 만들 필요는 없다.

모듈 수준의 일반 함수로 정의하는 것이 더 명확할 수도 있다.

2.11.8 세 메서드의 바인딩 구조를 직접 비교하기

이번에는 지금까지의 내용을 하나의 코드로 비교해 보자.

```
class Example:
    def instance_method(self):
        return self

    @classmethod
    def class_method(cls):
        return cls

    @staticmethod
    def static_method():
        return "Hello"

obj = Example()
```

각 메서드에 접근했을 때 어떤 객체를 얻는지 확인해 보자.

```
print(type(obj.instance_method))
print(type(obj.class_method))
print(type(obj.static_method))
```

실행 결과:

```
<class 'method'>
<class 'method'>
<class 'function'>
```

여기서 중요한 결과가 나타났다.

인스턴스 메서드와 클래스 메서드는 모두 바운드 메서드 객체로 접근된다.

반면 정적 메서드는 일반 함수 객체로 접근된다.

__self__ 비교

```
print(obj.instance_method.__self__ is obj)
print(obj.class_method.__self__ is Example)
```

실행 결과:

```
True
```

```
True
```

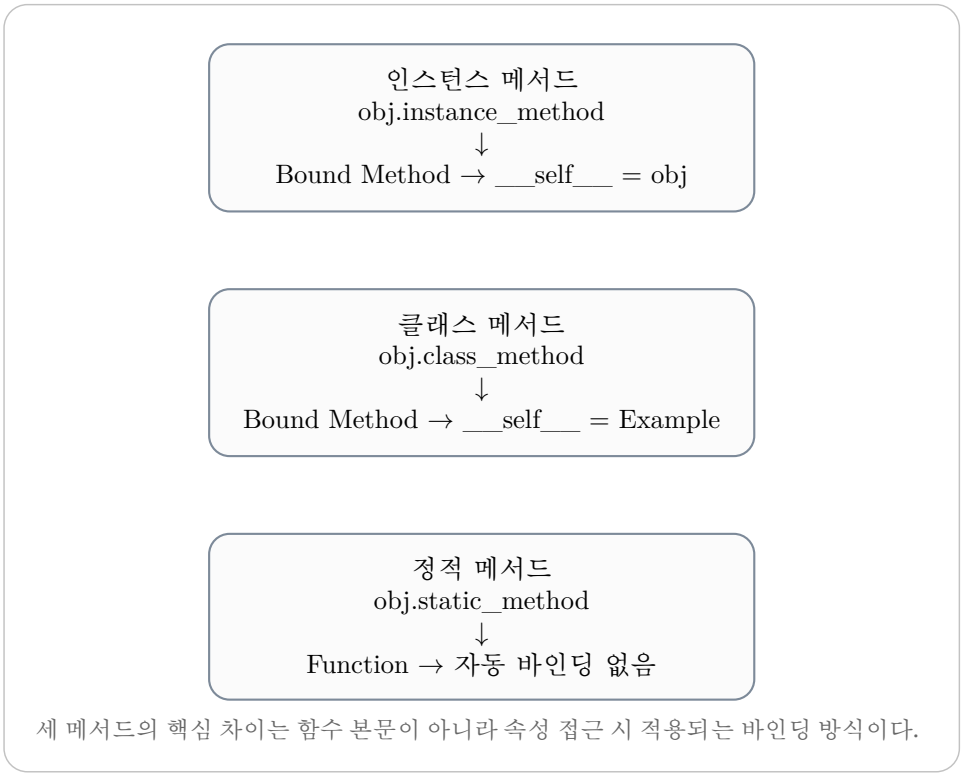
정적 메서드는 일반 함수 객체이므로 바운드 메서드의 `__self__`를 갖지 않는다.

```
print(hasattr(obj.static_method, "__self__"))
```

실행 결과:

```
False
```

세 가지를 그림으로 정리하면 다음과 같다.



세 메서드의 핵심 차이는 함수 본문이 아니라 속성 접근 시 적용되는 바인딩 방식이다.

2.11.9 디스크립터 프로토콜로 세 메서드를 설명하기

이제 2.7에서 학습한 디스크립터 개념을 연결해 보자.

세 메서드의 차이는 클래스 네임스페이스에 저장되는 객체와 그 객체의 `__get__()` 동작으로 설명할 수 있다.

다음 클래스를 정의하자.

```
class Example:
    def instance_method(self):
        pass

    @classmethod
    def class_method(cls):
        pass

    @staticmethod
    def static_method():
        pass
```

클래스 딕셔너리의 객체 타입을 확인해 보자.

```
for name in (
    "instance_method",
    "class_method",
    "static_method"
):
    obj = Example.__dict__[name]
    print(name, type(obj))
```

실행 결과:

```
instance_method <class 'function'>
class_method <class 'classmethod'>
static_method <class 'staticmethod'>
```

이제 구조를 정리할 수 있다.

메서드	클래스 `__dict__`에 저장되는 객체	`__get__()` 결과
인스턴스 메서드	`function`	인스턴스에 바인딩된 메서드
클래스 메서드	`classmethod`	클래스에 바인딩된 메서드
정적 메서드	`staticmethod`	감싸진 함수 객체

실제로 `__get__()`을 직접 호출해 볼 수도 있다.

```
obj = Example()
```

```
instance_descriptor = Example.__dict__["instance_method"]
class_descriptor = Example.__dict__["class_method"]
static_descriptor = Example.__dict__["static_method"]


a = instance_descriptor.__get__(obj, Example)
b = class_descriptor.__get__(obj, Example)
c = static_descriptor.__get__(obj, Example)
```

이제 확인해 보자.

```
print(a.__self__ is obj)
print(b.__self__ is Example)
print(type(c))
```

실행 결과:

```
True
True
<class 'function'>
```

이것이 세 메서드의 차이를 설명하는 가장 중요한 객체 모델이다.

`@classmethod`와 `@staticmethod`는 함수 본문에 특별한 능력을 부여하는 문법이 아니라, 함수가 클래스 속성으로 접근될 때의 바인딩 동작을 변경하는 디스크립터를 만드는 데코레이터다.

2.11.10 `self`, `cls`, 정적 메서드의 선택 기준

이제 실제로 클래스를 설계할 때 어떤 메서드를 선택해야 할지 생각해 보자.

다음 상황을 가정하자.

```
class Person:
    species = "Human"

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

세 가지 기능을 추가하고 싶다.

첫 번째는 자신의 이름을 출력하는 기능이다.

```
def introduce(self):
    return self.name
```

이 기능은 인스턴스의 name 속성을 사용해야 한다.

따라서 인스턴스 메서드가 적절하다.

두 번째는 문자열로부터 인스턴스를 생성하는 기능이다.

```
@classmethod
def from_string(cls, data):
    name, age = data.split(",")
    return cls(name, int(age))
```

이 기능은 실제 접근 대상 클래스를 사용하여 객체를 생성해야 한다.

따라서 클래스 메서드가 적절하다.

세 번째는 나이가 유효한지 확인하는 기능이다.

```
@staticmethod
def is_valid_age(age):
    return isinstance(age, int) and age >= 0
```

이 기능은 인스턴스 상태도, 클래스 상태도 필요하지 않다.

따라서 정적 메서드로 작성할 수 있다.

최종 코드는

```
class Person:
    species = "Human"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        return f"My name is {self.name}"

    @classmethod
    def from_string(cls, data):
        name, age = data.split(",")
        return cls(name, int(age))

    @staticmethod
    def is_valid_age(age):
```

```
        return isinstance(age, int) and age >= 0
```

이제 세 메서드는 사로 다른 목적을 갖게 된다.

```
p = Person.from_string("Kim,30")

print(p.introduce())
print(Person.is_valid_age(p.age))
```

실행 결과:

```
My name is Kim
True
```

2.11.11 클래스 메서드와 클래스 속성 변경에서 주의할 점

2.7에서는 클래스 속성과 인스턴스 속성의 재바인딩 차이를 학습했다.

클래스 메서드에서도 비슷한 문제가 발생할 수 있다.

다음 코드를 보자.

```
class Counter:
    count = 0

    @classmethod
    def increase(cls):
        cls.count += 1

class ChildCounter(Counter):
    pass
```

이제 다음 코드를 실행해 보자.

```
ChildCounter.increase()

print(Counter.count)
print(ChildCounter.count)
```

실행 결과:

```
0
1
```

왜 이런 결과가 발생할까?

ChildCounter.increase()를 호출했기 때문에 cls는 ChildCounter다.

따라서 다음 표현식은

```
cls.count += 1
```

개념적으로 다음 과정을 거친다.

1. ChildCounter.count를 읽는다.
2. ChildCounter에 없으므로
   부모 Counter.count에서 0을 얻는다.
3. 1을 더한다.
4. ChildCounter.count에 결과를 설정한다.

따라서 부모 클래스의 count는 그대로이고, 자식 클래스에 새로운 count 속성이 생성된다.

```
print(Counter.__dict__["count"])
print(ChildCounter.__dict__["count"])
```

실행 결과:

```
0
1
```

이것은 2.7에서 학습했던 속성 가리기와 같은 원리다.

따라서 @classmethod를 사용한다고 해서 모든 클래스가 하나의 공유 상태를 자동으로 사용하는 것은 아니다.

cls가 어떤 클래스 객체를 참조하는지에 따라 속성 설정 위치가 달라질 수 있다.

2.11.12 PyTorch와의 연결 - nn.Module에서는 어떤 메서드를 사용하는가?

이제 PyTorch의 객체 모델과 연결해 보자.

다음 코드를 보자.

```
import torch
import torch.nn as nn


class MyModel(nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()

        self.linear = nn.Linear(
            input_dim,
            output_dim
        )

    def forward(self, x):
        return self.linear(x)
```

여기서 `forward()`는 어떤 종류의 메서드일까?

인스턴스 메서드다.

```
def forward(self, x):
```

`self`는 실제 모델 인스턴스를 참조한다.

따라서 다음 속성에 접근할 수 있다.

```
self.linear
```

이것은 모델 인스턴스마다 등록되ㄴ 하위 모듈이기 때문에 인스턴스 메서드가 적절하다.

> 클래스 메서드를 추가한다면?

예를 들어 설정 딕셔너리에서 모델을 생성하는 기능을 만들 수 있다.

```
class MyModel(nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()

        self.linear = nn.Linear(
            input_dim,
            output_dim
        )
```

```
    def forward(self, x):
        return self.linear(x)

    @classmethod
    def from_config(cls, config):
        return cls(
            input_dim=config["input_dim"],
            output_dim=config["output_dim"]
        )
```

이제 다음처럼 사용할 수 있다.

```
config = {
    "input_dim": 10,
    "output_dim": 5
}

model = MyModel.from_config(config)
```

여기서 `cls`는 `MyModel` 클래스 객체다.

따라서 설정값을 이용해 모델 인스턴스를 생성할 수 있다.

이런 방식은 모델 구조를 설정 파일이나 딕셔너리로 관리할 때 유용하다.

정적 메서드를 추가한다면?

입력 차원의 유효성을 검사하는 함수를 추가할 수도 있다.

```
class MyModel(nn.Module):
    @staticmethod
    def validate_dimension(dim):
        return isinstance(dim, int) and dim > 0
```

이 메서드는 특정 모델 인스턴스나 클래스 상태를 사용할 필요가 없다.

따라서 정적 메서드로 정의할 수 있다.

물론 검증 기능이 여러 클래스에서 공통으로 사용된다면 클래스 밖의 일반 함수로 분리하는 것도 좋은 설계다.

2.11.13 세 메서드의 최종 비교

지금까지 학습한 내용을 하나의 표로 정리해 보자.

구분	인스턴스 메서드	클래스 메서드	정적 메서드
데코레이터	없음	`@classmethod`	`@staticmethod`
관례적 첫 인수	`self`	`cls`	없음
자동 바인딩	인스턴스	클래스	없음
인스턴스 상태 접근	직접 가능	인스턴스가 별도로 필요	인스턴스가 별도로 필요
클래스 상태 접근	`type(self)` 등	`cls`	클래스 이름 등을 명시
상속 시 바인딩	실제 인스턴스	접근 대상 클래스	없음
대표적 용도	객체 동작	대체 생성자, 클래스 단위 동작	관련 유틸리티 함수
클래스 `__dict__`	함수 객체	`classmethod` 객체	`staticmethod` 객체

여기서 한 가지 주의할 점이 있다.

인스턴스 메서드는 클래스에서도 접근할 수 있다.

```
Person.introduce
```

하지만 클래스에서 접근하면 인스턴스가 자동으로 바인딩되지 않으므로, 일반적으로 인스턴스를 명시적으로 전달해야 한다.

```
Person.introduce(p)
```

반면 클래스 메서드는 클래스에서 접근해도 클래스 객체가 자동으로 바인딩된다.

```
Person.from_string("Kim,30")
```

정적 메서드는 어느 쪽에서도 자동 바인딩이 일어나지 않는다.

```
Person.is_valid_age(30)
p.is_valid_age(30)
```

이 차이가 세 메서드를 구분하는 가장 중요한 기준이다.

2.11.14 핵심 정리

이번 학습에서 반드시 이해해야 할 내용을 정리해 보자.

> 세 메서드의 차이는 함수 본문보다 바인딩 방식에 있다.

```
def method(self):
    ...


@classmethod
def method(cls):
    ...


@staticmethod
def method():
    ...
```

> 인스턴스 메서드는 인스턴스에 바인딩된다.

```
p.introduce.__self__ is p
```

> 클래스 메서드는 클래스 객체에 바인딩된다.

```
Person.get_species.__self__ is Person
```

> 정적 메서드는 자동 바인딩을 수행하지 않는다.

클래스나 인스턴스로 접근해도 일반적으로 함수 객체를 반환한다.

> `@classmethod`와 `@staticmethod`는
> 디스크립터 객체를 생성하는 데코레이터다.

클래스 네임스페이스에 저장되는 객체의 종류와 `__get__()` 동작을 변경한다.

> 클래스 메서드는 상속 관계에서 유용하다.

`cls`가 실제 접근 대상 클래스로 바인딩되므로, 자식 클래스의 타입을 유지하는 대체 생성자를 구현할 수 있다.

> 클래스 메서드에서 클래스 속성을 변경할 때는
> 바인딩 대상을 고려해야 한다.

cls.count += 1은 자식 클래스에 새로운 클래스 속성을 만들 수 있다.

PyTorch의 forward()는 인스턴스 메서드다.

모델 인스턴스의 하위 모듈과 파라미터를 사용해야 하기 때문이다.

2.11.15 2.11에서 반드시 이해해야 할 결론

이번 학습의 핵심은 다음과 같다.

Method Type	Binding Target
Instance Method	Instance
Class Method	Class
Static Method	None

즉, Python에서 메서드의 종류를 구분하는 가장 정확한 기준은 어떤 객체가 자동으로 바인딩되는가이다.

이제 다음 코드를 보면

```
class MyModel(nn.Module):
    def forward(self, x):
        return self.linear(x)

    @classmethod
    def from_config(cls, config):
        return cls(**config)

    @staticmethod
    def validate_dimension(dim):
        return isinstance(dim, int) and dim > 0
```

각 메서드가 왜 해당 방식으로 정의되었는지 설명할 수 있어야 한다.

forward()는 모델 인스턴스의 상태를 사용한다.

from_config()는 클래스 객체를 호출하여 인스턴스를 생성한다.

validate_dimension()은 인스턴스나 클래스의 자동 바인딩이 필요하지 않다.

이 세 가지를 구분할 수 있다면 Python의 메서드 모델을 구조적으로 이해한 것이다.

2.2부터 2.11까지 Python의 클래스와 객체 모델을 학습하였다.

이번 2.12는 새로운 문법을 많이 배우는 단계라기보다, 지금까지 학습한 개념들이 실제 딥러닝 프레임워크에서 어떻게 결합되는지 확인하는 단계다.

특히 다음 코드를 단순히 암기하는 것이 아니라, 각 문장이 어떤 객체를 생성하고, 어디에 저장하며, 어떤 탐색과 호출 과정을 거치는지 설명할 수 있게 되는 것이 목표다.

```
import torch
import torch.nn as nn


class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear1 = nn.Linear(10, 20)
        self.relu = nn.ReLU()
        self.linear2 = nn.Linear(20, 5)

    def forward(self, x):
        x = self.linear1(x)
        x = self.relu(x)
        x = self.linear2(x)
        return x


model = MyModel()
x = torch.randn(2, 10)
y = model(x)
```

이 코드에서 우리가 추적할 것은 크게 네 가지다.

1. 클래스와 인스턴스가 어떻게 생성되는가?
2. `self.linear1` 같은 하위 모듈은 어디에 저장되는가?
3. `model(x)`는 어떻게 `forward()`로 연결되는가?
4. PyTorch는 파라미터와 하위 모듈을 어떻게 관리하는가?

2.11.16 nn.Module은 무엇인가?

먼저 `nn.Module` 자체를 살펴보자.

```
import torch.nn as nn

print(type(nn.Module))
```

실행 결과:

```
<class 'type'>
```

nn.Module은 클래스 객체다.

따라서 다음 정의는

```
class MyModel(nn.Module):
    pass
```

Python의 일반적인 상속과 동일한 원리로 동작한다.

```
print(MyModel.__bases__)
print(MyModel.__mro__)
```

출력 예시:

```
(<class 'torch.nn.modules.module.Module'>,)

(<class '__main__.MyModel'>,
 <class 'torch.nn.modules.module.Module'>,
 <class 'object'>)
```

즉, MyModel은 nn.Module을 상속하는 새로운 클래스 객체다.

nn.Module이 가진 메서드와 특수 메서드들은 MyModel의 MRO를 통해 사용될 수 있다.

예를 들어 다음 메서드들이 있다.

```
model.parameters()
model.state_dict()
model.train()
model.eval()
model.to("cpu")
```

이 메서드들을 우리가 MyModel 안에 직접 정의하지 않았더라도 사용할 수 잇는 이유는 상속 때문이다.

nn.Module의 핵심 역할

nn.Module은 단순히 forward()를 호출하는 기능만 제공하지 않는다.

모델 내부의 상태와 구조를 관리하는 기반 클래스다.

대표적으로 다음을 관리한다.

관리 대상	내부 등록 위치	예시
하위 모듈	_modules	nn.Linear, nn.ReLU
학습 파라미터	_parameters	nn.Parameter
버퍼	_buffers	BatchNorm의 running statistics

이 세 가지는 앞으로 모델을 이해할 때 계속 등장하게 될 것이다.

2.11.17 model = MyModel() - 객체 생성 과정

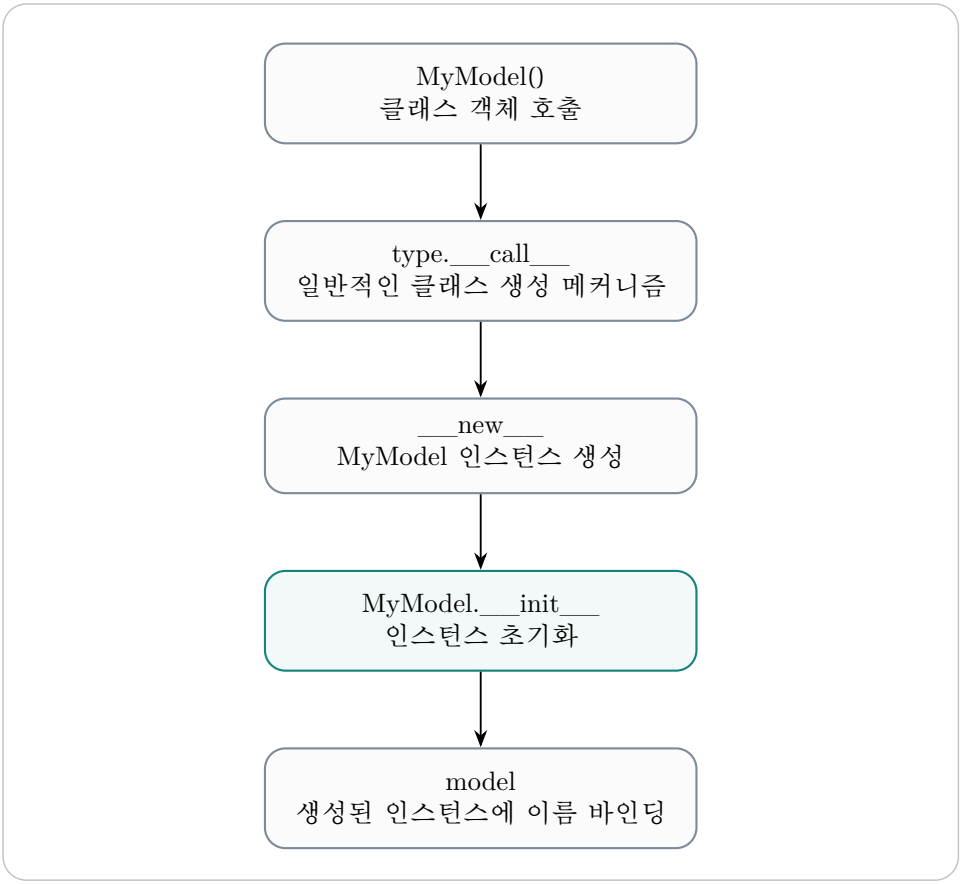
먼저 다음 코드만 살펴보자.

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 5)


model = MyModel()
```

2.6에서 학습한 내용을 적용하면 MyModel()은 클래스 객체를 호출하는 표현식이다.

일반적인 클래스 생성 과정은 다음과 같다.



여기서 `model`이라는 이름은 새로 생성된 `MyModel` 인스턴스를 참조하게 된다.

```
print(type(model) is MyModel)
print(isinstance(model, nn.Module))
```

실행 결과:

```
True
True
```

이것은 일반적인 Python 객체 생성과 다르지 않다.

차이는 초기화 과정에서 `nn.Module`이 제공하는 내부 관리 구조가 설정된다는 점이다.

2.11.18 super().__init__()은 왜 먼저 실행해야 하는가?

다음 코드를 보자.

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 5)
```

2.9에서 배운 것처럼 super()는 MRO를 기준으로 다음 구현을 탐색한다.

이 경우 MRO는 다음과 같다.

$$\texttt{MyModel} \rightarrow \texttt{nn.Module} \rightarrow \texttt{object}$$

따라서 super().__init__()은 nn.Module.__init__()으로 연결된다.

이 초기화 과정에서 PyTorch가 모듈 등록과 상태 관리에 필요한 내부 구조를 준비한다.

대표적으로 다음과 같은 속성들이 있다.

```
self._parameters
self._buffers
self._modules
```

이들은 보통 딕셔너리 형태의 내부 등록 공간이다.

초기화 이후 다음처럼 확인할 수 있다.

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()


model = MyModel()

print(type(model._parameters))
print(type(model._buffers))
print(type(model._modules))
```

일반적인 실행 결과:

```
<class 'dict'>
```

```
<class 'dict'>
<class 'dict'>
```

초기화 순서를 바꾸면?

다음 코드를 보자.

```
class BadModel(nn.Module):
    def __init__(self):
        self.linear = nn.Linear(10, 5)
        super().__init__()

model = BadModel()
```

일반적인 PyTorch에서는 다음과 같은 오류가 발생한다.

```
AttributeError: cannot assign module before Module.__init__() call
```

왜 이런 문제가 발생할까?

`self.linear`에 `nn.Linear` 인스턴스를 대입하는 작업은 단순한 Python 속성 대입이 아니기 때문이다.

`nn.Module`은 하위 모듈을 자동으로 등록하는 메커니즘을 제공한다.

그런데 그 등록을 위한 내부 상태가 아직 준비되지 않았다면 하위 모듈을 등록할 수 없다.

따라서 일반적인 PyTorch 모델에서는 다음 순서를 지켜야 한다.

```
def __init__(self):
    super().__init__()

    self.linear = nn.Linear(10, 5)
```

`super().__init__()`은 단순한 관례가 아니라, PyTorch 모듈의 등록 시스템을 사용하기 위한 초기화 단계다.

2.11.19 `self.lienar = nn.Linear(...)`는 일반적인 속성 대입인가?

이번 학습에서 가장 중요한 부분 중 하나다.

다음 코드를 보자.

```
self.linear = nn.Linear(10, 5)
```

일반적인 Python 클래스라면 다음과 같은 동작을 기대할 수 있다.

```
self.__dict__["linear"] = ...
```

하지만 nn.Module에서는 조금 다르다.

2.7에서 학습했던 __setattr__()이 개입하기 때문이다.

nn.Module은 __setattr__()을 재정의해서 대입되는 객체의 종류에 따라 등록 방식을 다르게 처리한다.

예를 들어 nn.Module 인스턴스를 속성에 대입하면 하위 모듈로 등록할 수 있다.

실제로 확인해 보자.

```
import torch.nn as nn


class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10, 5)


model = MyModel()
```

이제 인스턴스 딕셔너리를 확인해 보자.

```
print("linear" in model.__dict__)
print("linear" in model._modules)
```

실행 결과:

```
False
True
```

linear라는 이름은 일반적인 인스턴스 딕셔너리에 직접 저장되지 않았다.

대신 _modules에 등록되어 있다.

```
print(model._modules)
```

출력 예시:

```
{'linear': Linear(in_features=10, out_features=5, bias=True)}
```

이 결과는 중요한 의미를 가진다.

`self.linear`이라는 속성 접근 문법을 사용한다고 해서 반드시 `linear`라는 이름이 `self.__dict__`에 저장되어 있는 것은 아니다.

2.7에서 학습한 일반적인 속성 탐색 구조가 PyTorch에서는 사용자 정의 속성 접근 메커니즘에 의해 확장된 것이다.

2.11.20 `_modules`에 저장된 객체는 어떻게 `self.linear`로 접근되는가?

여기서 자연스럽게 의문이 생긴다.

`linear`가 `model.__dict__`에 없다면 다음 코드는 어떻게 동작할까?

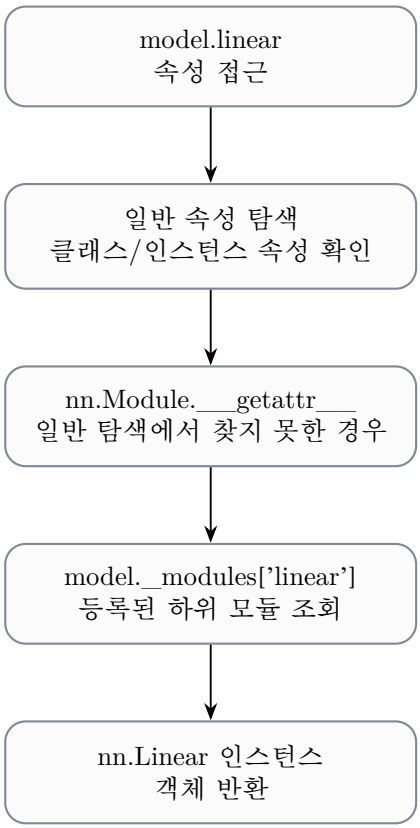
```
model.linear
```

그 이유는 `nn.Module`이 `__getattr__()`도 제공하기 때문이다.

일반적인 속성 탐색으로 원하는 속성을 찾지 못하면 `__getattr__()`이 호출될 수 있다고 배웠다.

PyTorch는 이 메커니즘을 활용해 등록된 파라미터, 버퍼, 화위 모듈을 속성처럼 접근할 수 있게 한다.

개념적인 흐름은 다음과 같다.



일반적인 등록 하위 모듈에 대한 속성 접근을 단순화한 흐름

즉, 다음 두 표현식은 일반적으로 같은 하위 모듈 객체를 참조한다.

```
print(model.linear is model._modules["linear"])
```

실행 결과:

```
True
```

여기서 우리가 학습한 두 특수 메서드가 연결된다.

- `__setattr__()` : 하위 모듈 등록
- `__getattr__()` : 등록된 하위 모듈 조회

이것이 PyTorch가 일반적인 Python 속성 문법을 사용하면서도 내부적으로 모델 구조를 관리할 수 있는 이유 중 하나다.

2.11.21 nn.Linear 인스턴스의 파라미터는 어디에 저장되는가?

이제 다음 코드로 넘어가 보자.

```
linear = nn.Linear(10,5)
```

linear는 nn.Linear 클래스의 인스턴스다.

이 인스턴스에는 대표적으로 다음 파라미터가 있다.

```
linear.weight
linear.bias
```

각각의 타입을 확인해 보자.

```
print(type(linear.weight))
print(type(linear.bias))
```

실행 결과:

```
<class 'torch.nn.parameter.Parameter'>
<class 'torch.nn.parameter.Parameter'>
```

여기서 nn.parameter는 일반적인 텐서와 관련된 특수한 클래스다.

torch.Tensor를 상속하며, nn.Module의 속성으로 대입되면 파라미터로 자동 등록될 수 있다.

내부 등록 위치 확인

```
print("weight" in linear.__dict__)
print("weight" in linear._parameters)
```

실행 결과:

```
False
True
```

즉, weight는 _parameters에 등록되어 있다.

```
print(linear._parameters.keys())
```

실행 결과:

```
dict_keys(['weight', 'bias'])
```

여기서 중요한 점은 모든 텐서가 자동으로 파라미터로 등록되는 것은 아니라는 것이다.

다음 코드를 비교해 보자.

```
class Example(nn.Module):
    def __init__(self):
        super().__init__()

        self.a = torch.randn(3)
        self.b = nn.Parameter(torch.randn(3))

example = Example()

print("a" in example._parameters)
print("b" in example._parameters)
```

실행 결과:

```
False
True
```

일반 텐서 a는 자동으로 학습 파라미터로 등록되지 않는다.

반면 nn.parameter 객체인 b는 파라미터로 등록된다.

이 차이는 오빕마이저와 모델 상태 관리에서 매우 중요하다.

2.11.22 _modules, _parameters, _buffers의 차이

지금까지 두 종류의 등록 공간을 확인했다.

_modules : 하위 모듈

_parameters : 파라미터

여기에 하나가 더 있다.

바로 _buffers다.

버퍼는 모델 상태의 일부이지만 일반적으로 옵티마이저가 학습하는 파라미터는 아닌 텐서를 관리할 때 사용한다.

예를 들어 Batch Normalization의 running mean과 running variance가 대표적이다.

직접 버퍼를 등록할 수도 있다.

```
class Example(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(3, 2)

        self.scale = nn.Parameter(
            torch.ones(2)
        )

        self.register_buffer(
            "running_value",
            torch.zeros(2)
        )

model = Example()
```

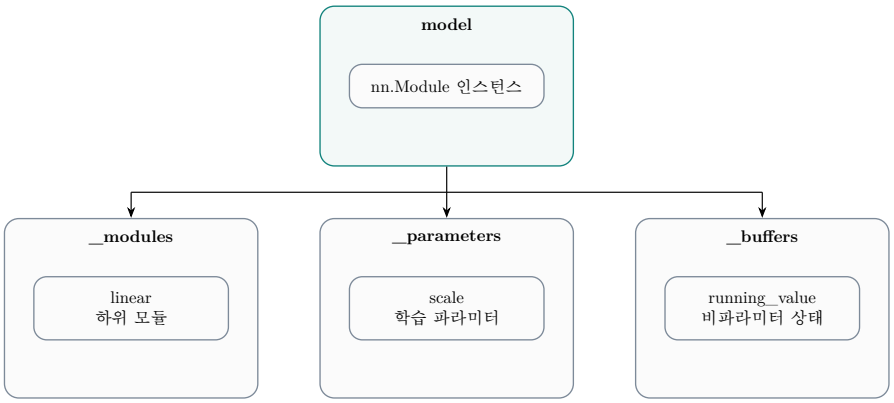
이제 내부 등록 구조를 확인해 보자.

```
print(model._modules.keys())
print(model._parameters.keys())
print(model._buffers.keys())
```

실행 결과:

```
dict_keys(['linear'])
dict_keys(['scale'])
dict_keys(['running_value'])
```

이 구조를 정리하면 다음과 같다.



하위 모듈인 linear 역시 자체 등록 공간을 가진다.
linear._parameters → weight, bias

여기서 중요한 구조가 하나 더 있다.

`model.linear`는 또 다른 `nn.Module` 인스턴스다.

따라서 `model.linear`도 자체적인 `_parameters`, `_modules`, `_buffers`를 가질 수 있다.

이것이 PyTorch 모델이 계층적으로 구성되는 기반이다.

2.11.23 `model.parameters()`는 어떻게 하위 모듈의 파라미터를 찾는가?

다음 모델을 다시 보자.

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear1 = nn.Linear(10, 20)
        self.relu = nn.ReLU()
        self.linear2 = nn.Linear(20, 5)

    def forward(self, x):
        x = self.linear1(x)
        x = self.relu(x)
        x = self.linear2(x)
        return x

model = MyModel()
```

이제 다음 코드를 실행해 보자.

```
for name, parameter in model.named_parameters():
    print(name, tuple(parameter.shape))
```

실행 결과:

```
linear1.weight (20, 10)
linear1.bias (20,)
linear2.weight (5, 20)
linear2.bias (5,)
```

여기서 주목할 점은 MyModel 자체의 _parameters에는 직접 등록한 파라미터가 없을 수 있다는 것이다.

```
print(model._parameters)
```

실행 결과:

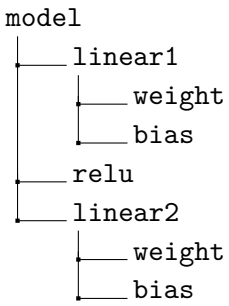
```
{}
```

그런데도 model.named_parameters()는 네 개의 파라미터를 찾는다.

왜 그럴까?

nn.Module이 등록된 하위 모듈들을 재귀적으로 탐색할 수 있기 때문이다.

개념적으로 다음 구조를 탐색하는 것이다.



relu에는 일반적으로 학습 파라미터가 없으므로 named_parameters() 결과에 나타나지 않는다.

중요한 구분

```
model.parameters()
```

는 파라미터를 순회할 수 있는 Iterator를 반환한다.

```
model.named_parameters()
```

는 파라미터 이름과 파라미터 객체의 쌍을 순회할 수 있는 Iterator를 반환한다.

따라서 다음 코드를 사용할 수 있다.

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
```

d ㅗ ㅂ 티마이저는 이 Iterator를 통해 모델의 등록된 파라미터를 전달받는 것이다.

여기서 2.10에서 학습한 Iterator 프로토콜이 다시 등장한다.

2.11.24 state_dict()는 무엇을 저장하는가?

다음 코드를 보자.

```
state = model.state_dict()
```

state_dict()는 모델의 상태를 이름과 텐서 값의 대응 관계로 제공한다.

기본적으로 등록되 ㄴ 파라미터와 영속적(persistent) 버퍼가 포함된다.

예를 들어 앞의 MyModel에서는 다음 키를 얻을 수 있다.

```
print(list(model.state_dict().keys()))
```

실행 결과:

```
['linear1.weight',
 'linear1.bias',
 'linear2.weight',
 'linear2.bias']
```

여기서 중요한 사실은 다음과 같다.

`state_dict()`는 모델 객체 전체를 그대로 저장하는 것이 아니라, 모델의 등록된 상태를 구조화해서 제공하는 메커니즘이다.

따라서 모델 구조를 정의하는 클래스와 그 클래스의 파라미터 값은 개념적으로 구분해야 한다.

예를 들어 다음 코드는 모델 상태를 파일로 저장하는 일반적인 방식이다.

```
torch.save(
    model.state_dict(),
    "model.pt"
)
```

다시 불러올 때는 먼저 같은 구조의 모델 인스턴스를 생성한 뒤 상태를 로드할 수 있다.

```
model = MyModel()

state = torch.load(
    "model.pt",
    map_location="cpu",
    weights_only=True
)

model.load_state_dict(state)
```

여기서 `MyModel()`은 모델 구조를 생성하고,

`load_state_dict()`는 등록된 상태를 해당 모델에 반영한다.

이것이 모델의 구조와 상태를 분리하는 방식이다.

2.11.25 `model(x)` - 호출 메커니즘과 `forward()`

이제 모델 실행으로 넘어가 보자.

```
x = torch.randn(2, 10)

y = model(x)
```

여기서 `model`은 함수 객체가 아니다.

MyModel 인스턴스다.

그런데도 model(x)라는 호출 문법을 사용할 수 있다.

2.10에서 학습한 __call__() 프로토콜 때문이다.

nn.Module은 모델 인스턴스를 호출 간으하게 만드는 메커니즘을 제공한다.

따라서 다음 결과를 얻을 수 있다.

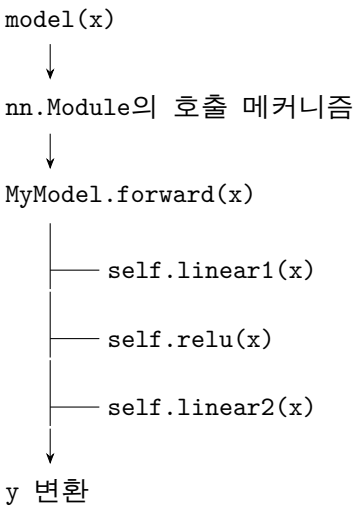
```
print(callable(model))
```

실행 결과:

```
True
```

호출 과정

개념적으로 다음과 같은 흐름이다.



여기서 중요한 것은 forward()를 우리가 직접 호출하지 않았다는 점이다.

model(x)가 상속된 호출 메커니즘을 통해 forward()를 실행하도록 연결된 것이다.

실제 메서드 확인

```
print(model.forward.__self__ is model)
```

실행 결과:

```
True
```

`forward()`는 인스턴스 메서드이므로 `model`에 바인딩된다.

또한 다음처럼 확인할 수 있다.

```
print(model.forward.__func__ is MyModel.forward)
```

실행 결과:

```
True
```

즉, `forward()` 역시 2.5에서 학습한 일반적인 바운드 메서드의 원리로 동작한다.

2.11.26 `forward()`를 오버라이딩했는데 왜 `nn.Module`이 이를 호출할 수 있는가?

이 부분은 2.8의 상속과 오버라이딩을 연결하는 중요한 지점이다.

다음 코드를 보자.

```
class MyModel(nn.Module):
    def forward(self, x):
        return x * 2
```

`nn.Module`은 호출 메커니즘을 제공하고, 자식 클래스는 `forward()`를 오버라이딩한다.

그런데 부모 클래스의 호출 메커니즘은 자식 클래스의 `forward()`를 실행할 수 있다.

왜 그럴까?

호출 메커니즘이 실제 모델 인스턴스의 `forward` 속성을 조회하기 때문이다.

이때 `self`는 실제 `MyModel` 인스턴스다.

따라서 일반적인 속성 탐색 규칙에 따라 `MyModel.forward`가 선택된다.

이것을 간단한 일반 Python 클래스로 표현하면 다음과 같다.

```
class Base:
    def __call__(self, x):
        return self.forward(x)

    def forward(self, x):
        raise NotImplementedError


class Child(Base):
    def forward(self, x):
        return x * 2

obj = Child()

print(obj(10))
```

실행 결과:

```
20
```

여기서 Child는 __call__()을 정의하지 않았다.

하지만 Base.__call__()을 상속했다.

그리고 Base.__call__() 내부에서 다음 코드를 실행한다.

```
self.forward(x)
```

여기서 self는 Child 인스턴스다.

따라서 Child.forward()가 선택된다.

이 구조가 PyTorch의 모델 호출을 이해하기 위한 핵심적인 Python 객체 모델이다.

단, 실제 nn.Module의 호출 메커니즘에는 Hook 처리 등 추가적인 기능이 포함되어 있다.

2.11.27 model.forward(x) 대신 model(x)를 사용하는 이유

다음 두 표현식을 비교해 보자.

```
y1 = model(x)
```

```
y2 = model.forward(x)
```

두 표현식 모두 `forward()`의 계산을 실행할 수 있다.

하지만 실행 경로는 같지 않다.

첫 번째는 `nn.Module`의 호출 메커니즘을 사용한다.

두 번째는 `forward()`를 직접 호출한다.

PyTorch는 모듈 호출 과정에서 여러 Hook을 처리할 수 있다.

예를 들어 다음과 같은 Hook이 있다.

- Forward pre-hook
- Forward hook
- Backward 관련 hook

`model.forward(x)`를 직접 호출하면 모듈 호출 메커니즘을 우회하므로 이러한 처리가 정상적으로 적용되지 않을 수 있다.

따라서 일반적인 모델 실행에서는 다음 형태를 사용해야 한다.

```
y = model(x)
```

이것은 단순한 스타일 문제가 아니라 객체 호출 프로토콜을 통해 프레임워크의 실행 메커니즘을 사용한다는 의미다.

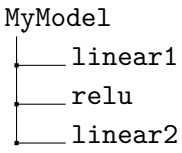
2.11.28 모델 구조와 계산 그래프는 같은 것인가?

여기서 중요한 개념적 구분을 하나 해야 한다.

우리는 지금까지 `nn.Module`이 하위 모듈을 등록하고 관리하는 구조를 살펴보았다.

그런데 이 구조와 자동 미분을 위한 계산 그래프는 동일하지 않다.

모듈 구조



이 구조는 Python 객체와 하위 모듈의 등록 관계를 나타낸다.

모델 인스턴스가 어떤 하위 모듈ㅇ르 가지고 있는지 설명하는 구조다.

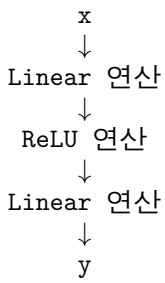
계산 그래프

반면 다음 코드가 실행되면

```
x = torch.randn(2, 10)

y = model(x)
```

텐서 연산이 수행된다.



이 연산들 중 자동 미분 추적 대상이 되는 연산들은 역전파를 위한 그래프에 반영된다.

여기서 중요한 차이는 다음과 같다.

구분	모듈 구조	자동 미분 계산 그래프
기반	Python 객체 및 모듈 등록	텐서 연산
주요 목적	모델 구성 및 상태 관리	미분 계산
생성 시점	주로 모델 초기화 과정	추적 대상 연산이 실행될 때
관리 대상	모듈, 파라미터, 버퍼	연산 관계 및 역전파 정보
동적 변화	모듈 구조 변경 가능	실행 경로에 따라 달라질 수 있음

특히 PyTorch의 일반적인 eager 실행에서는 연산이 수행되면서 자동 미분 그래프가 구성된다.

따라서 모듈 등록 구조가 곧 계산 그래프인 것은 아니다.

이 구분은 이후 Transformer 모델을 분석할 때도 중요하다.

2.11.29 Transformer에서는 이 구조가 어떻게 확장되는가?

다음은 단순화한 Transformer 블록이다.

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model):
        super().__init__()

        self.attention = nn.MultiheadAttention(
            embed_dim=d_model,
            num_heads=8,
            batch_first=True
        )

        self.norm = nn.LayerNorm(d_model)

        self.ffn = nn.Sequential(
            nn.Linear(d_model, 4 * d_model),
            nn.ReLU(),
            nn.Linear(4 * d_model, d_model)
        )

    def forward(self, x):
        attn_out, _ = self.attention(x, x, x)
```

```
        x = self.norm(x + attn_out)

        x = x + self.ffn(x)

        return x
```

이 코드는 지금까지 배운 Python 객체 모델을 그대로 사용한다.

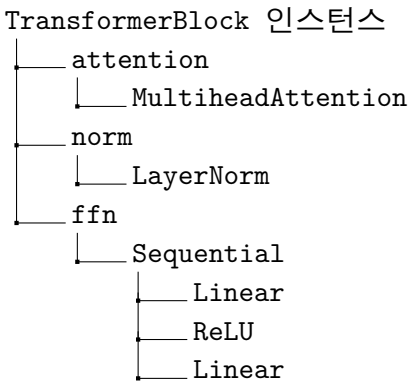
먼저 클래스 객체가 생성된다.

```
TransformerBlock
```

그다음 인스턴스를 생성한다.

```
block = TransformerBlock(512)
```

이 과정에서 하위 모듈들이 등록된다.



이제 다음 코드를 실행하면

```
x = torch.randn(2, 16, 512)

y = block(x)
```

nn.Module의 호출 메커니즘을 통해 ransformerBlock.forward()가 실행된다.

그 내부에서 다시 하위 모듈들을 호출한다.

```
self.attention(x, x, x)
self.norm(x + attn_out)
```

```
self.ffn(x)
```

각 하위 모듈도 nn.Module 인스턴스이므로 동일한 호출 메커니즘을 사요한다.

즉, Transformer는 특별한 Python 객체 시스템 위에서 동작하는 것이 아니라, 우리가 지금까지 학습한 클래스·상속·메서드·특수 메서드·모듈 등록 구조를 계층적으로 조합한 모델이다.

2.11.30 지금까지의 객체 모델을 하나의 그림으로 정리하기

다음 코드를 다시 보자.

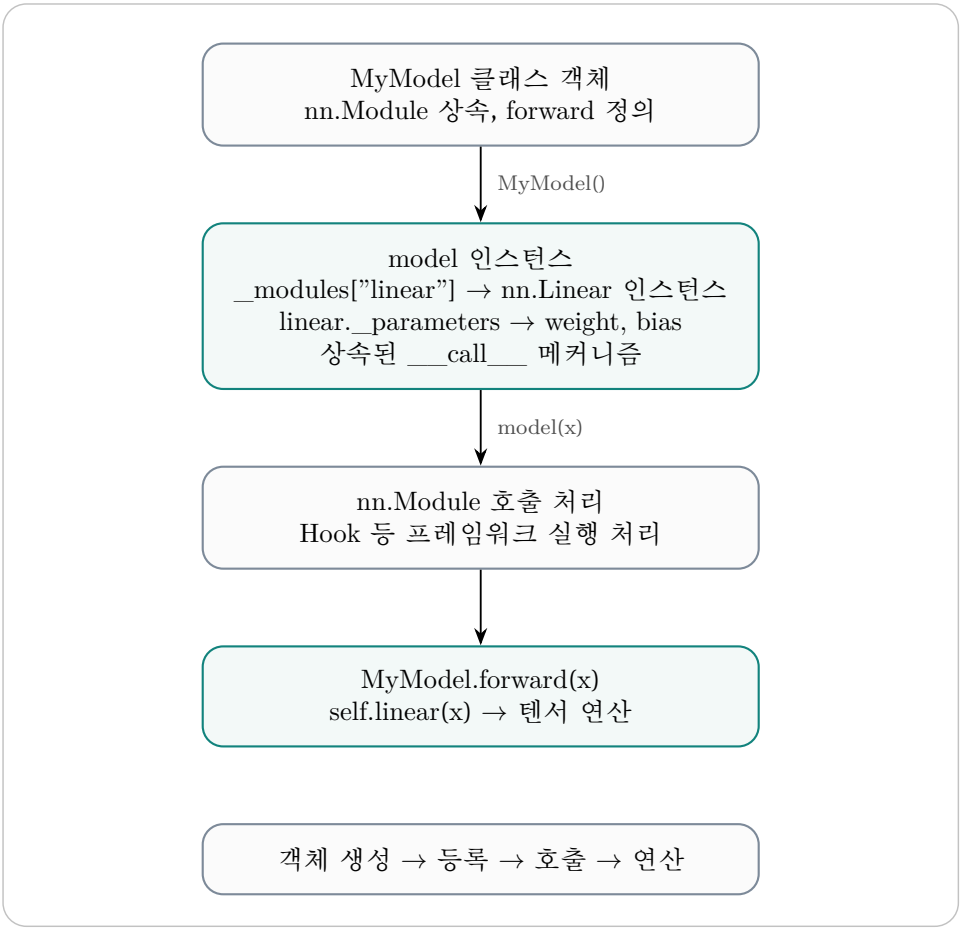
```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)


model = MyModel()
y = model(x)
```

이 코드를 객체 관계와 실행 관계로 구분해 보면 다음과 같다.



이제 코드 한 줄마다 어떤 객체와 메커니즘이 사용되는지 연결할 수 있다.

2.11.31 최종 확인 - 지금까지의 Python 클래스 학습과 연결

2.2부터 2.12까지의 내용을 정리해 보자.

단원	핵심 개념	PyTorch에서의 대응
2.2	클래스도 객체	`MyModel`, `nn.Module`
2.3	클래스와 인스턴스	`model = MyModel()`
2.4	인스턴스 속성과 네임스페이스	`self.linear`
2.5	메서드와 `self`	`forward(self, x)`
2.6	`__new__`, `__init__`	모델 생성 및 초기화
2.7	클래스·인스턴스 속성	`_modules`, `_parameters`, 속성 등록
2.8	상속과 오버라이딩	`nn.Module` 상속, `forward()` 재정의
2.9	`super()`와 MRO	`super().__init__()`
2.10	특수 메서드	`model(x)`
2.11	메서드 종류	`forward`, `from_config`, 검증 메서드
2.12	객체 모델 통합	모델 등록·상태 관리·실행

마지막으로 반드시 구분해야 할 다섯 가지

첫째, 클래스 객체와 인스턴스 객체는 다르다.

```
MyModel
model
```

`MyModel`은 클래스 객체이고, `model`은 그 인스턴스다.

둘째, 속성 접근 문법과 실제 저장 위치는 다를 수 있다.

```
model.linear
```

는 일반적으로 `_modules`에 등록된 하위 모듈을 조회한다.

셋째, 파라미터와 일반 텐서는 등록 방식이 다르다.

```
self.weight = nn.Parameter(...)
self.tensor = torch.randn(...)
```

첫 번째는 학습 파라미터로 등록되고, 두 번째는 일반적으로 자동 등록되지 않는다.

> 넷째, `model(x)`와 `model.forward(x)`는 실행 경로가 다르다.

`model(x)`는 `nn.Module`의 호출 메커니즘을 사용한다.

> 다섯째, 모듈 구조와 계산 그래프는 서로 다른 개념이다.

모듈 구조는 객체의 구성 관계이고, 자동 미분 계산 그래프는 텐서 연산의 관계이다.

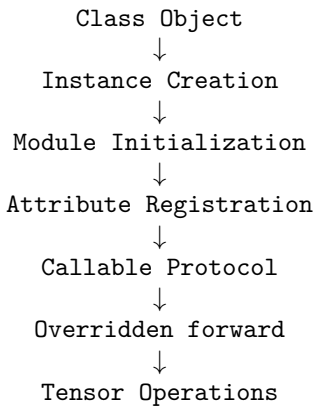
2.11.32 Python 클래스와 객체 모델 학습 완료

이번 2.12까지 학습했다면 다음 코드의 의미를 객체 모델 관점에서 설명할 수 있는 기반이 갖춰진 것이다.

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 5)

    def forward(self, x):
        return self.linear(x)
```

핵심은 다음과 같다.



이제 Python 클래스는 단순히 `class`와 `self`를 사용하는 문법이 아니라, 객체 생성·네임스페이스·디스크립터·상속·MRO·특수 메서드가 결합된 실행 시스템으로 이해할 수 있다.